

VECTOR

Forests in Finland VII ...

- Sasha on R and A+ 56
- Smith on the Dyalog .NET Compiler 65
- Stephen Taylor on XP 81
- Daintree on Web Services 116
- Chastney on Null values 132



*The Journal of the
British APL Association*

ISSN 0955-1433
www.vector.org.uk
Vol.19 No.4 April 2003

A specialist Group of the British Computer Society

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette or via email. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from Vector Production), and MS Word (any version).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 2002–2003

Category	Fee	Vectors	Passes
UK Private	£20	1	1
Overseas Private	£22	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	5	5
Overseas Corporate	£110	5	
Sustaining	£500	10	5
Non-voting Member (Student, OAP, unemployed)	£10	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

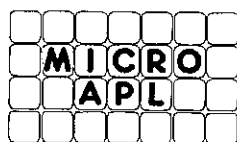
Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to Gill Smith, Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385.

Email: apl385@compuserve.com.

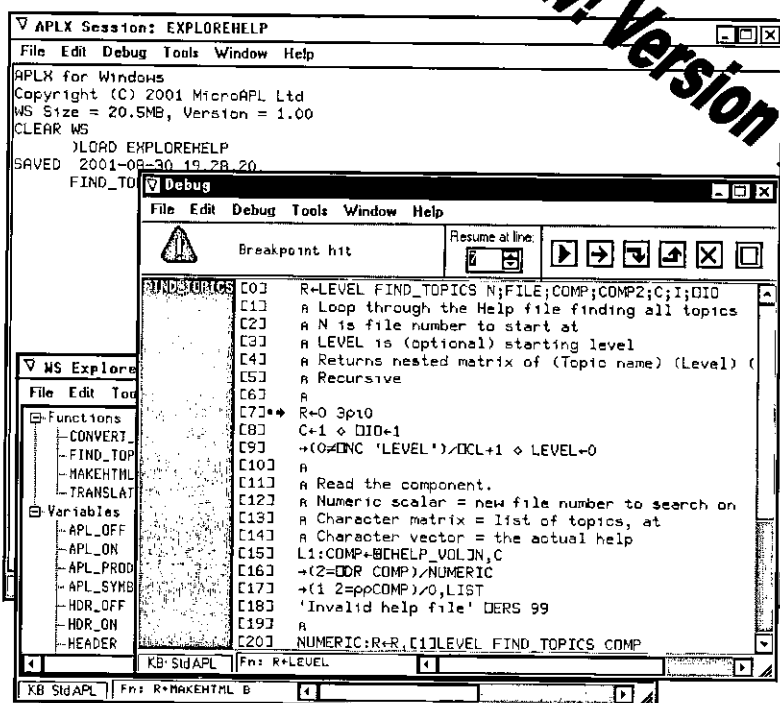
Contents

	Page
Editorial	Stefano Lanzavecchia 3
APL NEWS	
Quick Reference Diary	7
Librarian Proposal	Stefano Lanzavecchia 8
Correspondence	11
British APL Association News	
News from Sustaining Members	15
APL Product Guide	Gill Smith 17
The Education Vector	
Zark Newsletter Extracts	edited by Jon Sandles 32
Numberword in D	Jonathan Barman 40
Crossword	43
J-ottings 36: A Partial Solution to a Partial Problem	Norman Thomson 46
RECENT MEETINGS	
K Watch: Arthur Whitney in London	Steph Johnson 53
Forests in Finland VII	Adrian Smith 55
Experiences with the Dyalog .NET Compiler	Adrian Smith 65
GENERAL ARTICLES	
A Place for Us	Stephen Taylor 77
The Simple Art of Software: XP Techniques in APL	Stephen Taylor 81
TECHNICAL SECTION	
Hacker's Corner: from APL 2 APL2C	Adrian Smith 96
At Play with J: Pick a Card, Any Card	McDonnell & Smith 101
Tools, Part 1. Basics	Dan Baronet 106
Web Services with Dyalog.NET	John Daintree 116
Getting to the Bottom of Things	Phil Chastney 132
Index to Advertisers	143

APLX *for*



Mac OS 9	Windows 95/98/ME	Linux
Mac OS X	Windows NT/2000/XP	AIX



The cross-platform, full-featured, user-friendly,
GUI-enabled, cost-effective APL. *New! Version 1.1*
Check it out at <http://www.microapl.co.uk/apl>

APLX and MicroAPL are trademarks of MicroAPL Ltd. Other trademarks acknowledged.

Editorial

by Stefano Lanzavecchia

"Cyberspace. A consensual hallucination experienced daily by billions of legitimate operators, in every nation, by children being taught mathematical concepts... A graphic representation of data abstracted from the banks of every computer in the human system. Unthinkable complexity. Lines of light ranged in the non space of the mind, clusters and constellations of data. Like city lights, receding...."

William Gibson - *Neuromancer*

A lot has happened since the early days of the World Wide Web explosion: the protocols have evolved, new ones have been introduced, the mark-up languages have changed, the support tools have matured and have become at the same time more accessible and more complex. One of the practical consequences is that ten years ago a surfer of the new virtual wave had to work hard to find a way to put his hands on a Web Browser, no matter which platform or Operating System he was using, whereas today he would hardly find an OS whose standard installation package does not include a full-featured Web Browser. That's certainly good news for software developers. There are many who think that a Web Browser like Microsoft's Internet Explorer or Mozilla are powerful enough platforms to build and deliver full-fledged applications on, and there are many good examples of such applications. In fact for those who have the luck of a permanent and fast Internet connection, the boundary between local computing and remote computing is becoming fuzzier. I will only mention in passing the new challenges that this kind of exposure poses in terms of security and privacy, because what I want to concentrate on are the actual technical benefits of using a Web Browser and a Web Server as the means to build an application's GUI front-end.

Let us start from the browser. The series of articles by Adrian Smith that *Vector* hosted in the past should have convinced our readers that HTML is really simple and that, even if not suited to all kinds of data entry, it can be used to represent information in a possibly primitive but effective way. The quality of the presentation can be readily improved using the so-called Cascading Style Sheets (CSS to friends). I'll also happily fly over the headaches that some cross-browser incompatibilities can cause because my goal here is not to describe a public Web

Application (like Amazon's web site, or Yahoo's portals) but a specialised front-end, therefore I can make assumptions about the platforms my GUI will run on.

So, with a few angular brackets, our data is beautifully laid out and we can now look at the way things look back-stage. Amongst the empty crates of beer, the wasteland of used tea filters (sorry, I don't drink coffee), there sits the mighty Server. The image is purely metaphorical, of course, since, as I said earlier, our application will not run on a dedicated server at our headquarters, but will be shipped at the customer's site, and I am sure we've all experienced the chilling (truly freezing) beauty and neatness of a customer's equipment's vault. Since we are APLers, our data will probably be generated by the clever one-liner we are so proud of. The question is: how do we get it to appear, beautifully formatted in HTML inside the final user's browser? Here is where the big distinctions need to be made in my opinion: it really depends on the kind of data, on the kind of network infrastructure on which the application is deployed, on the expected load, on the required levels of security. Writing a simple (crude) HTTP server is an afternoon's divertissement. To make it safe, fully compliant, fast and scalable is a lifetime's occupation, and it may very well require a degree of intimacy with the TCP socket layer of the host machine which some programming languages don't permit, and APL is no exception. If we envisage heavy load and sophisticated client requirements, we will be better off choosing the released work of those people who have spent the mentioned lifetimes and provided us with powerful all-singing and all-dancing HTTP servers. The only problem left would be to interface our computing engine with the distributing server. Easier said than done, once again, but APL is at no particular disadvantage when compared to other languages. Possibly we could be slowed down only by a lack of publicly available samples to build on, but one can always borrow ideas.

Still, there are situations where our crude APL-based HTTP server can be useful. Let us concentrate on a very precise example: instrumentation of an application server, like a proprietary database engine or an intelligent data cache. Some might consider it a waste of time but my experience in the field tells me that what is known as *instrumenting* a server is quite useful. It seems to be a nature's law that customers are really good at creating problems that all our cleverly designed lab tests did not catch, and their IT infrastructure is no better: flaky networks, shady hardware, unexpected usage scenarios, our nightmares becoming reality. Sometimes it is a real lifesaver to have the possibility of monitoring the inner workings of an application server, even to do simple maintenance without disturbing the users connected to it, users who are not the least concerned about our technical problems and only want to get their jobs done, switch off their PCs on their way out of the office and enjoy this kind Spring sun. Here is where an embedded HTTP server comes to our rescue. Since such a server is hosted directly

by the process running the application, it has access to all of its internal states and can even control it. Useful statistics can be collected based on this privileged knowledge and they can be analysed in real-time to keep the server healthy. Why use an HTML front-end? First, as we said earlier, producing HTML is simple and we can leave all the gory details of the formatting of text paragraphs, data tables and graphs to the Web Browser. Second, the system administrator at the customer's site won't need to install anything in order to access our application monitor. Third, anything that has to do with trendy technologies will have a stronger selling impact, and will please both financial and IT managers.

Similarly, whenever there is the need to expose data without involving the main GUI of our application, if the interaction with the user will be mostly in terms of navigation and not so much in terms of data entry, an embedded Web server is an excellent solution. I like to look at this approach as a sort of symbiosis where the embedded server acts as the interface between the original organism and the new partners. So far, my experience with embedded HTTP Servers has been very positive and I certainly plan to keep exploiting and exploring the possibilities of the technology.

June Cutts

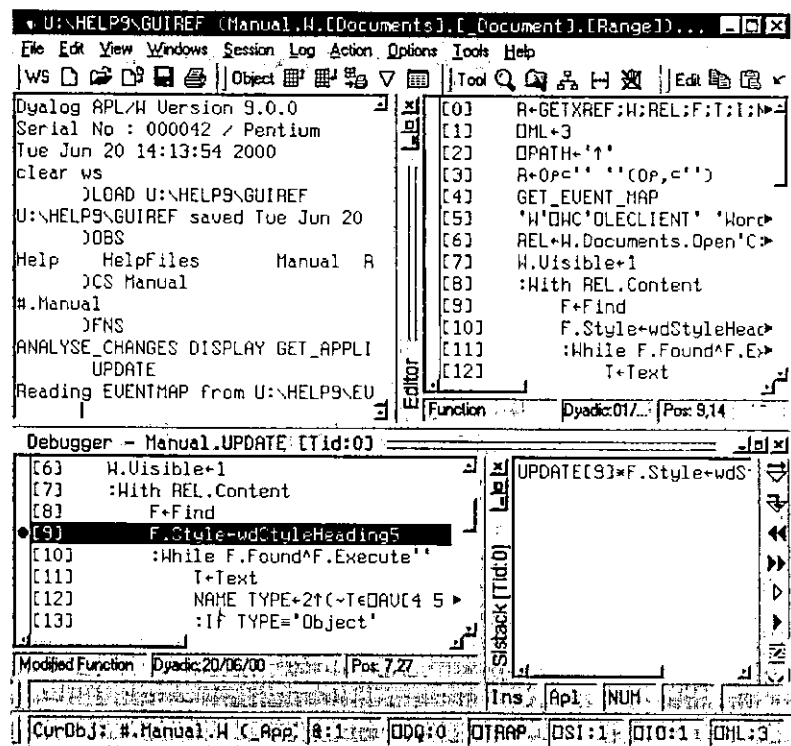
Many members will not have heard much of June Cutts who was appointed to be our mailing house by David Preedy. He became Editor of Vector in 1985, but I think he made the appointment at least a year earlier, when he was acting as assistant to Robert Bittlestone, his predecessor. So June Cutts has done the job from 1983 or 1984 until earlier this year, when the job was taken over by Nicholas Small (who has taken over the job without changeover problems). June postponed her retirement for our benefit at least twice.

We are very grateful for the excellent service we have had over nearly twenty years, and the Committee has sent her a small token of our appreciation. We wish her and her family a happy and fulfilling retirement.

Anthony Camacho, Hon.Sec.

dyalog APL

The Definitive APL for Windows™



Version 9 - an even better IDE(A)

<http://www.dyadic.com>

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing, Basingstoke,
Hants. RG24 7AL, United Kingdom.

Tel:+44 1256 811125 Fax:+44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation

Quick Reference Diary

Date	Venue	Event
11-14 June, 2003	San Diego, USA	FCRC:APL2003 joint event See Call for Papers in Vector 19.2
4-6 Sept, 2003	Gilleleje, Denmark	Proposed Viking APL Conference See http://www.insight.dk for details
2-5 Nov 2003	USA	APL2000 User's Meeting. Naples Beach Hotel.

Dates for Future Issues of VECTOR

	Vol.20 No.1	Vol.20 No.2	Vol.20 No.3
Copy date	20th June	3rd Sept	4th Dec
Ad booking	27th June	12th Sept	11th Dec
Ad Copy	4th July	19th Sept	18th Dec
Distribution	July/Aug 2003	October 2003	January 2004

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ**

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

Please note that Vol.1 No.2 is now out of stock.

An APL Library

proposal by Stefano Lanzavecchia

"O Deep Thought computer," he said, "the task we have designed you to perform is this.

We want you to tell us..." he paused, "the Answer!"

"The Answer?" said Deep Thought. "The Answer to what?"

"Life!" urged Fook.

"The Universe!" said Lunkwill.

"Everything!" they said in chorus.

Deep Thought paused for a moment's reflection.

"Tricky," he said finally.

Douglas Adams – *The Hitchhiker's Guide to the Galaxy*

If day after day your typical task is to delete leading and trailing blanks in simple character vectors, invert small numeric matrices, and calculating the average of planes in arrays representing sales, you'll certainly find that a naked APL interpreter coupled with the *FinnAPL* idiom list is more than adequate. But if your customers or the business requirements demand distributed applications, idiot-proof graphical interfaces, connectivity and the most trivial of the tasks you are faced with is to programmatically download a web page, parse part of it, calculate its CRC, store it in a database, zip it, and attach it to an e-mail, then you might feel let down.

APL interpreters over the years have developed into general purpose development environments with many different interfacing capabilities. So why are the support libraries still lagging behind so badly? Let's take for example another language, a modern and popular one: *Perl*. Well hidden in the acronymic name are the origins of the language: a Practical Extraction and Report Language, that the creator used to do manipulation of text files. How comes that despite its modest origins and its admittedly sub-optimal syntax, it's become one of the most popular languages ever? It's interpreted but it does not offer a development environment like an APL session. It's cross-platform but most of us program on one platform at the time anyway. It is free and open-source but professional developers don't go looking in the bowels of their interpreter every day. The features of the language in itself do not seem to be enough to justify its popularity. We must look somewhere else.

Fire up your favourite web browser, input the URL <http://www.cpan.org/> and check the list of library utilities available. Even if you have a slow modem it's still worth letting the seemingly endless list download. There are more modules to connect to Database Engines than you thought there were databases. There are XML parsers, file compressors, e-mail management tools, web scrapers, and more, much more. Not only that: they are all free and well documented, since these are requirements to make it into CPAN. We can direct our attention to other similarly popular languages, featuring a not

particularly exciting syntax, like Python or PHP, and find the same wealth of free supporting *useful* utility libraries. Even for more obscure but classy languages like ML or Haskell seem to be better supported than our beloved APL.

I believe this to be a serious limiting factor and I expressed my thoughts already in the editorial for Vector's issue 19.1. Because of the positive feedback I have received, the BAA has decided to start a library project. Since the APL community is relatively small and we cannot expect a large amount of support, especially at the beginning, we will proceed in small steps.

At the moment we see the library as a repository for freely available code, decently documented. It would be good if all the different APL (and J and K) interpreters could be supported, and that's definitely a goal, although it could take a while to fill all the gaps. As we begin the collection of what's already available on the world wide web, we will need librarians, each one expert in a different field (networking, protocols, file and file system manipulation, databases, user interface, internationalisation, core utilities, and so on), to sort the code, rationalise the documentation, disentangle the dependencies and finally publish the polished work on a new website. Once the code is available online, we expect people to download it, find it more or less immediately useful, adapt it to their needs, improve it and hopefully make their contribution returning the improved versions to the library. This is how CPAN was born and grew, so we know it's possible, even if it's difficult.

The website hosting the library (what about <http://www.apl.ws/>, (I think all the www.apl.* domains were gone, how about www.apls.ws? domain still available for purchase?) could grow overtime in a information portal, with bug-tracking features, a forum that supports the exchange of messages as well as APL snippets in the full beauty of their symbolic representation, newsletters and more. It could be really cool, as they say.

But to shoot so high now would be foolish, so let's stick to more modest expectations in order not to waste our limited energies and resources. What we need now is a few volunteers to dirty their hands and start the collection and classification of available code before we even start producing new one. Morten Kromberg has offered his services as our Librarian, and he would love to hear from any volunteer "assistant librarians", who should contact him (mkromberg@insight.dk). If you feel that an APL library is a worthy goal, have code to contribute or want to share your expertise in a field by contributing with your time, feel free to contact me at mailto:stf@apl.it and let the fun begin.

"Tricky," he said finally.

"But can you do it?"

Again a significant pause.

"Yes," said Deep Thought, "I can do it."

"There is an answer?" said Fook with breathless excitement.

"A simple answer?" added Lunkwill.

"Yes," said Deep Thought, "Life, the Universe, and Everything. There is an answer.

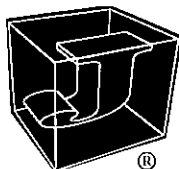
But," he added, "I'll have to think about it."

Douglas Adams - The Hitchhiker's Guide to the Galaxy

**Download
from the
web for
free!**

Jsoftware™ Version 5.01

- ♦ **Session Manager, window driver, IDE and GUI support in Unix and Linux**
- ♦ **GUI event support extended to more closely match VB and Java**
- ♦ **New Debug facilities**
- ♦ **New Documentation**
- ♦ **Performance Improvements**



**Jsoftware Inc.
P.O. Box 330
Excelsior, MN 55331
Info@jsoftware.com
Tel (952) 470-7345
Fax (952) 470-9202**

The programming language
ideal for complex analytics
and data manipulation.

www.jsoftware.com

CORRESPONDENCE

Hidden meaning inside APL

From: Gianluigi Quario

February 10th 2003

An application programmer usually bears in mind the meaning of the data inserted in the APL structures he is using: a vector of prices, a table of names and surnames, the nested array representing the tree of summarisation groups, and so on.

But the structures also have an inner meaning of their own.

Let us consider for example a vector of numbers. It is not just a simple collection of numbers. *It has an implicit order.*

Between the original meaning attributed to a vector and the vector itself there is another "semantic" layer, which is directly provided by the language.

This appears to be the case whenever a natural or formal language is used.

The use of a language to represent something meaningful "pollutes" the original meaning by adding some further meaning to it or even introducing a bias into it.

What happens is that, not only the exchange of information through a language (natural or formal) produces a kind of entropy, but also that the use of a language modifies the signification. The structure of the language is *never neutral* and the use of a particular language originates different views regarding the intrinsic meaning of what we (or a formal language) are talking about.

For example: the diversity of the Greek and Latin languages gave rise to different theological interpretations in the eastern and western churches of Christianity in the first millennium of our era and similarly the use of national languages by the countries ruled by the Roman law contributed to the diversification of national legislations.

The data structures of APL (and other parallel languages) sometimes engender the attribution of further strong meaning to the information lodged by the application programmer inside the data.

When a programmer defines a vector like:

```
girls ← 'Mary' 'Lucy' 'Anne' 'Rose'
```

perhaps one just wants to use a tool for pointing – by means of a single noun – at those four girls.

When one defines a vector of survivorship group like:

```
L ← 99890 99835 99802 ....
```

one wants to tell APL that – from an initial number of 100,000 – at age 1 the number of survivors is 99890, at age 2 is 99835 .. and so on.

In both cases APL adds automatically another meaning to those vectors: *Mary is the first, Lucy the second ...99890 is the first, 99835 the second ...*

but the meaning added by APL to `girls` is arbitrary: Mary is the eldest or the shortest or the cutest?

On the contrary 99890 is the actual number of survivors at the first anniversary, 99835 at second anniversary, and so on.

The vectors of APL implicitly contain an idea of ordering, which may or may not correspond to the programmer's will.

Another clear instance in which the designers of APL implicitly thought that a vector has a more complex native structure is to be found in the behaviour of the primitive function "*index of*" (dyadic iota).

Let us consider:

```
girls ι ⋈ 'Emma' ↔ ⍳10 + ρgirls
```

The result should be DOMAIN error, because Emma is not in our set called "*girls*".

Why is Emma the fifth? My answer is: because the vector "*girls*" looks like having an extra element at its right end. This extra element is not defined elsewhere in APL and I would call it:

"something matching everything".

Only this extra semantic meaning can explain the result that Emma is the fifth girl of the group.

Some say that, when the right argument is out of domain, the result of the primitive function "*index of*" is the best possible result because it is the most useful for practical programming activities.

May be it is so, but we should be aware that coherence is not to be found everywhere in APL and that sometimes it would be better to work around APL's constraints.

I have given a few examples about vectors; in a similar manner we can also expect the other data structures (arrays or nested arrays) to engender some implicit relationships inside the data.

In general we can pose these questions:

Is the programmer using a language (whatever it is) really able to attribute an unambiguous meaning to the data inserted in the structures he has created?

Should the best language have the fewest hidden meanings?

But the most important question for APL is the following:

Do parallel languages have a heavier hidden semantic overload ?

I do not know the answer. Can anybody help?

APL and the Pocket PC

From: Philip Bath

February 4th 2003

The reviews by Ray Cannon and Adrian Smith on Dyadic's version of Dyalog for the PocketPC [1,2] were interesting and I promptly downloaded "PocketAPL" onto my Compaq iPAQ. Whilst the beta version of the product clearly works (and comes with some neat little utilities), sadly it will be of little use to me as a APLX user. I have little time or inclination to change large amounts of code on my Mac for porting over to the iPAQ. In this respect I was concerned at Adrian's comment "If we can all share the same interpreter, so much the better" [2]. Whilst it is superficially attractive to only have one APL interpreter on all hardware (not just the Pocket PC) with the aim of removing the very problem I describe above, I believe that this would lead to the demise of APL. Like English, APL is dynamic and evolving and one of the spurs to this is the competition between the vendors and their supporters. So, a single dialect of APL, perhaps from a single vendor, would not only probably lead to price increases (lack of competition) but would stifle APL evolution. As we all know, those things that do not evolve eventually die. So I hope that MicroAPL produce a Pocket PC version of APLX, not just for the benefit of their current users, but to maintain healthy competition in the APL market. The same applies to other vendors and their dialects of APL.

Philip Bath

Email: philip.bath@doctors.org.uk

1. Cannon R. *Dyalog APL for Window CE (beta). A hacker's first impression.* Vector 2002;19:(2),46-51.
2. Smith A. *If you go down to the woods today ... First experiences with PocketAPL.* Vector 2002;19:(2),

[Reply] I am sorry for the misunderstanding. What I meant to say was that all the Dyalog applications for the Pocket can share the same installation, as the Dyalog runtime is a separately installable component from Handango. This means that if you post (say) a card-trick simulator workspace (see the Esmerelda simulator in the article by Richard Smith and Gene McDonnell on page 101) it can be a really small download. Adrian

BAA: Sustaining Members' News

Dyalog Ltd

As of 31st March 2003, the Dyalog APL Business has been transferred from Dyadic Systems Limited to Dyalog Limited

What has changed?

Dyalog APL is now owned, developed, marketed and supported by a new company called Dyalog Limited.

Dyalog Limited is owned and operated by Peter Donnelly, John Scholes and Pauline Brand, the founders of Dyadic Systems. All members of staff who were associated with the support and development of Dyalog APL at Dyadic have transferred to the new company and are now employed by Dyalog Limited.

Dyadic Systems is now owned by Syan Limited and is no longer involved with or connected with Dyalog APL.

Why has it changed ?

Previously, Dyadic Systems Limited had two separate divisions. One division, the original business, was dedicated to the development and support of Dyalog APL of which Dyadic was the author. The second division, which sold hardware, had grown into a highly respected and successful IBM Business Partner providing high-availability computing solutions to major UK companies based upon IBM pSeries, Storage and Software.

It was becoming increasingly difficult to manage and grow the two divisions as a single business, and the founders of Dyadic decided to sell the IBM Reseller business to Syan Limited, IBM's iSeries Partner of the year in 2002.

Structurally, it was sensible to first move the Dyalog APL business (including the APL development team and the copyright and intellectual property rights for Dyalog APL) into a new company, leaving Dyadic with just the IBM Reseller business prior to its sale.

How will it affect Dyalog APL ?

In the immediate future, Dyalog Limited and Dyadic Systems Limited will share the Basingstoke offices and cooperate at a business and personal level, just as the two divisions have operated previously.

Effectively, as far as our APL customers are concerned, all that has changed is the company name. However, by concentrating solely on APL, Dyalog Limited will be able to provide an even better service to its Dyalog APL customers in the future. This change will have only a beneficial effect on the future of Dyalog APL.

The official Dyalog APL web site is now *www.dyalog.com*, although the old site will continue to run in parallel for some time. The email addresses at Dyalog Limited are as follows:

Technical support: support@dyalog.com
Sales enquiries: sales@dyalog.com

Dyalog APL Version 10

Dyalog Limited is pleased to announce the availability of Dyalog APL Version 10.

One of the many enhancements in Version 10 is a performance boost provided by the implementation of *idiom recognition* and other features. One Dyalog APL customer, a major US oil company, has already reported a speed-up of 35% in an application they have developed.

Causeway Graphical Systems Ltd

This last few months has been a case of running hard to stand still. We are close to a release of RainPro for APLX (it will require Version 2.0 as I rely on some of the more obscure stuff in the +Win Draw method) and by the time you read this, there will be an update of the CPro workspace for Dyalog 10. Unfortunately, Version 10 was not quite upward compatible from Versions-8 and 9, so anyone with the up-to-date password for December's build will have free access to the workspace for V10. The good news is that (having made a few simple fixes to the engine) I can't fault the APL. Everything just runs as before on V10.

The main addition to RainPro is *Tables* – these allow you to add a neat table of summary information to any existing graph (in the same way as a Note) with full control over the formatting of the row/column labels and cells. Also (really just for completeness) we have added the *Venn Diagram*, so at long last RainPro can do everything that the old IBM PGF package did in 1979. Such is progress.

We have also been working hard to meet a customer requirement for 'close to WYSIWYG' HTML output from NewLeaf. You can now run your report as normal and choose the rendering (paper, PDF, HTML) for the completed report. This facility will be available via CUSP for both APL+Win and Dyalog APL customers.

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

*Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.
Please help us keep it up to date!*

All contributions and updates to the Vector Product Guide should be sent to:
Gill Smith, Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385,
Email: apl385@compuserve.com

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Borealis Inc.	Dyalog APL	poa	Distributor of Dyalog APL products from Dyadic
	APL2000	poa	Distributor of APL2000 products
APL Systems IDC SL	APL+Win v3.6 Full version	1895	A 32-bit Windows-hosted interpreter that runs under Windows 95/98/ME/NT/XP
	Upgrade to v3.6 from v3.x	474	
	to v3.6 from APL+Win v2.x	1034	
	to v3.6 from APL+Plus III v1.x	1249	
	Migration to v3.6 from APL+Plus II or APL+Plus PC	1464	
	APL+Unix		(Please contact us for details.)
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	
Dittrich & Partner	APL+Win	poa	Cognos/APL2000 Inc products
	Dyalog APL	poa	
	IBM APL products	poa	
Dyalog	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	
	Dyalog APL for Unix	995-12,000	
DynArray	DICE for Windows	poa	Software development kit which includes an APL interpreter as a DLL and the ability to run and link existing and new APL code to non APL code such as VB, C/C++, Java and integration with various Windows software applications and database packages such as MS Office.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	
	I-APL/Archimedes	8	
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	Workstation APL2 V2 Version 2	\$1500	
	APL2 Version 2	poa	
	APL2 Application Envi Vn2	poa	
Insight Systems	Cognos/APL2000 Inc	poa	Leading distributor of APL2000 products in Denmark
	Dyalog Ltd.	poa	
	IBM	poa	
J Austria	J	poa	Distributor for Austria and Switzerland
	Dyalog APL	poa	
	Causeway Products	poa	

	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
JSoftware Inc.	J on the Web online registration ...		
	J Professional (online reg.)	\$895	includes manual set and one year of updates
	J Standard (online reg.)	Free	Free for download only
	Books and accessories		
	J Dictionary	\$50	
	J Phrases	\$50	
	J Primer	\$50	
	Fractals, Visualization and J	\$80	
	Concrete Math	\$40	
	Exploring Math	\$50	
Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also distributes in Switzerland and Belgium. Call for price quotes.
	APL+Unix	poa	
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APL/W	poa	French distributor for Dyalog
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APLX for Windows/MacOS	499	Cross-platform APL development environment with GUI programming facilities. Interpreter modelled on APL2. Available for Windows 95/98/ME/NT/2000/XP, Mac OS 9 and Mac OS X.
	APLX Server Edition	poa	For running large multi-user APL applications on x86 Linux, RS/6000 AIX, and Windows NT/2000.
	APLX for Linux (commercial)	499	Linux desktop edition of APLX, compatible with Windows and MacOS versions, with full development environment and GUI programming facilities. Runs under RedHat, Debian, Mandrake and SuSe Linux distributions.
	APLX for Linux (personal)	Free	Full version of APLX, can be downloaded free for personal use.
Oasis	Dyalog APL	poa	Dyalog Ltd
	APL*PLUS	poa	Manugistics
	APL.68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Omega	Zero	poa	A "small simple and fast" alternative to APL
Optima	Dyalog APL/W	995	Dyalog's best, leading edge APL interpreter
RE Time Tracker Oy	APL+PC (APL*PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.
	APL+DOS (APL*PLUS II)		
	APL+Win (APL*PLUS III), APL+Link		
	APL+UNIX		
	APL*PLUS Sharefile		
Soliton Associates	SHARP APL for OS/390	poa	for IBM OS/390 mainframes
	SHARP APL for UNIX	poa	for SunOS and IBM AIX
	SHARP APL for Linux	poa	for Intel Linux
Strand Software	Canada		
	All APL*PLUS products	poa	All APL*PLUS products including upgrades and educational.
	Dyalog and JSoftware products	poa	

USA

Dyalog and JSoftware products poa

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADAPTA Software	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum	Adaytum e.Planning	poa	Adaytum e.Planning offers a Web-based solution that combines planning, forecasting, budgeting, modelling and reporting in a single, integrated application.
APL Software\Services	APL Utilities	poa	Software: mostly .AWS for DOS, utilities for most APL interpreters. Public domain APL*Plus v10 with on-screen documentation and interactive tutorials. APL Conference Software. Books: APL user manuals for STSC, IBM, and Sharp. Request email catalog from dick.holt@juno.com.
APL Systems IDC SL	APL+Linkpro 3.0 New System	648	Database Access SQL Client for OBDC
	Upgrade to v3.0		
	from APL+Linkpro-32	280	
	from APL-Link or APL+Linkpro	474	
	GraphX (Includes ChartX)	599	
	GraphX Lite (No distribution)	340	
	SPREAD (An APL grid control)	86	
	QPLOT (x,y plotting in APL)	47	
Beautiful Systems	QWIN	215	(Runs APL+DOS functions in APL+Win)
	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
Causeway	CausewayPro for Dyalog/W	400	Causeway application development platform for Dyalog APL/W.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in NewLeaf reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.

CODEWORK	3-way TANGRAM	poa	3-way TANGRAM is a DSS-OLAP product, basically a powerful and versatile handler of multi-way tables (also known as hypercubes). It entails 46 analysis modules, including computations, cube merging, sensitivity analysis, time intelligence, free format queries, HTML and LaTeX outputs. Current version is 7.0. At the time of writing, the product is available on Dyalog APL 8.3.1 for Windows 95/98/NT. Future plans include an APL+WIN version and later a LINUX version.
CoSy	K.CoSy	\$30 % yr sub	K.CoSy is a general purpose computing and programming environment constructed, all in open code, in Arthur Whitney's very high level, yet structurally transparent Array Programming Language, K, and its tightly coupled User Interface. K.CoSy is an extremely productive environment in one of the most powerful and fastest of APL's progeny, and therefore, likewise, of all languages. K.CoSy provides a workspace-like interactive development environment previously impossible in K. Because of its unique open construction within the language itself, this environment is clearly competitive in a large domain with the APLs from the other vendors. K.CoSy notepad nature, interactivity, and open K code vocabulary make learning Arthur Whitney's K far less daunting and far more productive than its raw console, or any external scripting method. If you are a client of Kx Systems, or are investigating the possibilities, Contact us. See CoSy/K/CoSy for more information.
DynArray	DynaWeb Server	poa	A web server providing web based access to applications running on the DICE interpreter from DynArray, or on an IBM mainframe running APL2.
	DynaHarry	poa	A DSS system which offers the next generation capabilities for current APLDI, IC/E and IC/I users. It comes with ROLAP capabilities, multisystem access to a wide variety of databases and data warehouses.
	DynaLink	poa	An ODBC client interface for DICE and IBM APL2 programs.
Entropy Software	Personal Information Management	poa	Entropy specialises in Personal Information Management Systems (PIMS) which run on a PC. Most systems are inadequate in some way. This is an attempt to fulfil all the needs of George MacLeod who has been an unhappy user of a variety of systems for many years. The product is being developed in modules (Calendar, Address List, To Do List, etc) which will be available at no charge. When all the modules are complete they will be integrated together. There will be a charge for the integrated product.
		poa	
HMW	4XTRA	poa	Networked, Windows/Unix based Front End and Middle Office Foreign Exchange and Money Market Dealing System. Scalable from 1 user to 120+.
	Inca	poa	Software Change Management System. Enables the user to co-ordinate development work from several sources, resolve clashes, promote work items for testing and configure releases to a live environment.
	Maya	poa	APL code file manager. A comprehensive suite of tools giving a multi-window IDE style interface to file based APL code. Offers features such as copying from file to file, object comparison, string search, style formatting, hot-spot editor for filed objects (including variables), etc.
	Aztec	poa	System shell for APL development. Manages real-time and batch applications across multiple platforms. Offers standardised error trapping, job scheduling, task communication and recovery/restart features.
	Olmec	poa	APL GUI environment, providing menu bar, tool bar, status area, navigation sidebar (with treeviews & listviews) and client area. All are configured by simple text files and require no programming. Client area has a "tab wizard" option to provide ordered transaction processing
	Nazca	poa	fast, flexible and reliable static database and editor.

I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
IBM APL Products	A Graphical Statistical System	\$250	for DOS, Product Number 5764-009
Insight Systems	Causeway	poa	Leading distributor of Causeway products in Denmark <i>All our old products are now either OEM'd, in the public domain, out of date, or all of the above. We'll be back!</i>
JAD Software	JAD SMS	poa	JAD SMS is a multi-user software management system for Dyalog APL™ based on shared, hierarchical databases. JAD SMS databases let you keep historical versions of apl items as well as attributes such as timestamp, user name and documentation. The software includes a graphical user interface as well as specialized functions for inclusion in applications. No charge for single-user version; \$100/user for multiple users
Lescasse Consulting	APL+Win Monthly Training	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC interface for APL+Win
	SQAPL Pro	poa	ODBC interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	QWIN	poa	Legacy DOS Windowing support for APL+Win
	ODBC/127	poa	IBM AP127-like ODBC Interface for APL+Win and Dyalog APL/W
Optima	ServiceLine	poa	A property management system which keeps a record of all outstanding tasks, produces an up to date list of work to-do and Scheduled reminders plus automated standard letters and basic financial control.
	TravelLine	poa	A system designed to control the workload allocation for a fleet of chauffeur vehicles plus a reminder system for fleet management and Local Authority requirements.
	BPA	poa	"Brand Performance Analysis" allows for the modelling of product/brand performance over time and comparison with competitor products.
	DBI	poa	"Database Interrogator" allows for the non technical user to ask sensible questions of a large database such as a questionnaire and obtain results tables and graphs quickly, easily and accurately.
Qualedi	Qualedi	\$850-\$5,500	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
RE Time Tracker Oy	UIT/W	poa	Comprehensive high-level Windows User Interface library for APL+Win and +II v 5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.

	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via DDE.
	NEWT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMS together with a sample +Win workspace.
	DB+	poa	Database interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server.
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Weighahead Systems	Weighahead Windows Weighing System (3WS)	poa	Recipe Weighing System for Manufacturing Industries. Pharmaceutical, Cosmetics, Foods etc. Works without keyboard or mouse. Uses Electronic Balances, Laser scanners, bar codes and label printers.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES (£)	DETAILS
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Andrews	Consultancy	poa	APL programming and analysis, algorithms, tree processing and design programs for craft work.
APL Borealis Inc.	Support and Development	poa	APL Software Support and Development. Specialists since 1979 in Sharp APL, APL*Plus, APL+Win, Dyalog APL
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
APL Systems IDC SL	Consultancy	poa	Consultancy and maintenance available by retainer or on call.
AUSCAN Software	Consultancy and Training	poa	APL software development, training
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ian Clark	Consultancy	poa	Interfacing APL, VB, C/C++, ActiveX/COM. Screen design and documentation. National language porting.
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	250-500	24-hour programmer: APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: info systems for top executives, Internet applications.

CoSy	Consultancy	poa	CoSy.com, Coherent Systems, provides rapid development in the K language and associated data base products, with a particular interest in quantitative (financial) modelling.
David Crossley	Consultancy	poa	Experienced in large APL system developments since 1989 for PC or mainframe.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.
Dittrich & Partner	Consultancy	poa	APL programming and analysis; APL workshops and training on the job
Dyalog	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
DynArray	Consultancy	poa	DynArray offers consulting in the areas of DSS, Y2K and APL programs upgrade/conversion to modern Web enabled platforms.
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
First Derivative Analytics Ltd.	Consultancy	poa	Analysis, design, prototyping, development & testing of APL (especially financial) applications: Sharp, Dyalog APL/W.
General Software	Consultancy	from 200	Over 20 years experience with every version of APL, large mainframe systems and small PC based programmes.
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
HMW	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
Michael Hughes	Consultancy	poa	APL consultant with 20 years experience with all versions of APL. I can create your dynamic Web sites using the full power of APL working with Microsoft IIS (Internet Information Service) on Windows NT or 2000. I also undertake System design, Programming and Maintenance on all platforms, particularly MS Windows.
INFOSTROY	Consultancy	poa, competitive	Broad experience in various APL platforms. Special skills and knowledge in developing complex applications for investment, financial and construction markets. Implementation of hybrid solutions based on APL, Delphi, C++, VBA, SQL servers.
Insight Systems	Consultancy	poa	We have experience with just about every APL system and platform in common use during the last 20 years, from SHARP APL under MVS or Linux to APL+Win and in particular Dyalog APL under Windows 9x, NT or 2000. If you have decisions to take about adapting your APL application to take advantage of emerging technologies, or would like your strategy reviewed, give us a call. We have extensive experience in all areas of APL development, from legacy systems, up, down and sideways migrations, to the development and support of shrink wrapped solutions based on APL. Even if we don't have time to do the work ourselves, we will know where to find someone who is an expert in your version of APL and your application area, on your continent.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
KJK	Consultancy and software development	poa	APL-based data management: conversions, ad hoc-analyzing tools, well-interfaced methods for defining, processing and browsing of multi-dimensional reports. Rapid custom software development based on proven modular toolset approach.
Lambent Technology	Consultancy	poa	APL programming, consulting & training; web design and construction.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.

Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APLW. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.
Lucas Solutions	Consultancy	poa	Rates depend on task and location.
Mackay Kinloch Ltd	Consultancy	poa	Design, analysis and programming for banking, insurance and pensions, financial planning and modelling, corporate performance and legal reporting
MasterWork Software	Consultancy	poa	Consulting and J programming for econometrics and statistics in public policy, health and food industries.
Milinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Windows programming.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Oasis	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning; for all APL versions (10+ years experience)
Omega Computing	Consultancy	poa	APL consultancy, programming, etc.
Optima	Consultancy, Training & Development	poa	We have been in business since 1990 and have a team of APL professionals with many years experience in pharmaceutical, industrial and financial systems on both PC and Mainframe platforms. In addition to pure APL we can also offer assistance with network integration, ACTI customisation and issues concerning the internet and/or web design.
RadSys Technologies	Consultancy	poa	Areas of expertise: financial systems, risk analysis systems, healthcare systems.
Resources & Results	Consultancy	poa	Knowledge management company builds decision support, data warehouse, data mining and strategic planning systems for CFO's, CEO's, and senior management, using our Rapid Application Development (RAD) methods and tools. Extensive experience in large-scale system development and ad hoc executive support for Fortune 500 clients.
RE Time Tracker Oy	-Consultancy	poa	APL application conversions, APL Windows interfaces, APL to API-level interfacing to any system under Windows, TCP/IP network and database connectivity.
Rex Swain	Consultancy	poa	Independent consultant, 25 years experience. Custom software development, PC and/or mainframe.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.
Snake Island Research Inc	Consultancy	poa	APL Interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
SovAPL	Consultancy	poa	Offshore APL development service.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
Weighahead Systems	Consultancy	poa	Specialising in industrial systems. Links to PLCs, laser scanners, bar codes, weigh scales, label printers etc. Also programmable hand held scanners.

Stephen Wynn	Consultancy	poa	Most experience of financial planning, and mathematical areas: operational research, quality control, experimental design.
--------------	-------------	-----	--

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adlee	Employment	poa	Contractors and permanent employees
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification.
APL Borealis Inc.	APL Training	poa	Hands-on courses in Introductory, Intermediate, Advanced and Windows APL. Courses are customized and flexible, and may be delivered on-site, with strong emphasis on methods for efficient and maintainable APL systems development.
Atomic Vector	RELCELL	poa	Details: Report Writer, Data Mining, Database Visualization, and Export to Excel from relational databases. Offered in English, Spanish, Italian, Portuguese, French and German. With this product, you don't have to know English in order to use SQL. Point and click with easy to use Wizards.
	APL Games Volume 1	\$15	Details: Volume 1 - A collection of four APL games on cd, saved from past yearly issues of the APL Planet. Great way to demonstrate the capabilities of APL, and to entice the younger generations into APL. Written entirely in Dyalog APL.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	Books	poa	I-APL stocks books written to go with the I-APL Interpreter and some APL Press books. For a list write to 11 Auburn Road, Bristol BS6 6LS, ring 0117 973 0036 or email 100612.1057@compuserve.com.
Oasis	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Optima	IBM Compatible	poa	Complete business solutions including hardware, software and support plus additional network installation and maintenance if required.
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Right Seat Software	Vox Proxy	\$199 (comm) \$69.95 (edu)	Vox Proxy is authorware for PowerPoint(r) 2000 or 2002 which allows the use of Microsoft Agent Technology (3D talking animated characters) within slide shows. VP appears on PowerPoint's main menu and provides editing side-by-side with slides. Automated script-writing provides control of PowerPoint, allowing the use of characters for live presentations or fully-automated tutorials, demos, or training programs. Optional CD Prep program allows the user to create auto-starting CD's that will play on any version of PowerPoint or without PowerPoint.

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Germany e.V.	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL		FF350 (private) FF2800 (Company)
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	Vector provided	Mini-congress, APL ShareWare Initiative	

FinnAPL	Helsinki, Finland	FinnAPL Newsletter Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat) 10,000yen to join
NY SigAPL	New York, USA	Big Apple APL	Monthly meetings \$35/\$25(ACM)
Rome/Italy SIG	Roma, Italy		
SE APL Users Grp	Atlanta, Georgia	The APL Planet	Semi Annual meetings \$13 (US), \$25 (non-US)
SovAPL	Moscow, Russia	-	Seminars and Annual Meeting
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars SEK 75
SWAPL	Texas, USA	SWAPL	\$18
Swiss APL (SAUG) Bern	Bern	Part of Qlty SI-Info	SF60 (SI) + SF20 (SAUG)
Toronto SIG	Toronto, Canada		Occasional meetings, APL Skills Database, Toronto Toolkit

ADDRESSES

ORGANISATION CONTACT

ADDRESS, TELEPHONE, FAX, EMAIL etc.

ACM SigAPL	David Siegel	ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
ADAPTA Software GmbH	Michael Baas	Stellinger Weg 19, 20255 Hamburg, Germany. Tel: +49 40 40170951 Fax: +49 40 40170952. Email: info@adapta.de
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: hillrj@melbpc.org.au
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
Andrews	Dr Anne D Wilson	12 Thorny Hills, Kendal, Cumbria LA9 7AL, UK. Tel: 01539-731205 Email: ADWilson@kencomp.net
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ, UK. Tel: 01439-788385 Email: apl385@compuserve.com
APL2000 (Europe)	Fred Honea	see APL Systems IDC SL.
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: jonesca@vnet.ibm.com
APL Borealis Inc.	Richard Procter	381 Manor Road East, Toronto, Ontario M4S 1S7, Canada. Tel: (416) 457-7828 Fax: (416) 482-6582 Email: info@aplborealis.com
APL Club Austria	Harald F. Nelson	c/o N-TECH, Siebenbrunnfeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
APL Germany e.V.	Dieter Lattermann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-63469 Compuserve: 100332,1461
APL Software/Services	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7624; Fax: +1 (703) 528-7617; Email: dick.holt@juno.org
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: aplsl@starpower.net
APL Systems IDC SL	Fred Honea	Alfredo Marquerie, 12 - 2 F, 28034 Madrid, Spain. Tel: +34 91 730 7008 (Office) +34 60 680 5949 (Mobile). Fax: +1 775 743 6131. Email: uksales@apl2000.net
Association Francophone pour la promotion d'APL	Ludmila Lemagnen	174 Boulevard de Charonne, F-75020 Paris, FRANCE Email: lemagnen@aol.com
AtomicVector Productions Inc.	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA. Tel: +1 (770) 972-3755 Email: E-mail: mailto:seapldoc@aol.com
AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@ca.inter.net
BACUS	Joseph De Kerf	Rocinberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 5, Jenkintown, PA 19046, USA Tel: +1 (215) 886-2636; Fax: +1 (215) 886-4888 Email: BeautifulSystems@goff.net
Carnacho	Anthony Carnacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. email: acam@blueyonder.co.uk

Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK. Tel: 01252-874697 Email: ray_cannon@compuserve.com
Causeway Graphical Systems Ltd	Adrian Smith	The Maltings, Castlegate, MALTON, North Yorks YO17 7DP, UK. Tel: 01853-696760 Fax: 01653-697719 Email: adrian@causeway.co.uk
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB, UK. Tel: 020 7404 5401. Compuserve: 100343,3210
Cinerea AB	Rolf Komemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email rolf@cinerea.se
Ian Clark	Ian Clark	1205 Deer Creek Drive, Plainsboro, NJ 08536, USA. Tel: +1 609 716 8832 Email: earthspot2000@hotmail.com
CODEWORK	Mauro Guazzo	Corso Lanza 105, 10133 Torino, Italy. Tel: +39 11 6601560 Fax: +39 11 6601560 Email: codework@codework-it.com
ComLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 260-1435 Email: jeff@softmed.com
CoSy.com	Bob Armstrong	42 Peck Slip #4B, New York, NY 10038-1725, USA. Tel: +1 212-285-1864 Fax: +1 212-285-1864. E-mail: bob@CoSy.com.
David Crossley	David Crossley	187 Le Tour du Pont, 84210 ST DIDIER, France. Tel: +33.4.90.66.08.87 Email: crossley@au-village.com
Danish User Group	Helene Boesen	c/o Insight Systems ApS, Nordre Strandvej 119G, Hellebæk, Denmark
Dinosoft Oy	Pertti Kalliojärvi	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dinosoft@dinosoft.fi
Dittrich & Partner Consulting GmbH	Axel Holzmüller	Kieler Strasse 17, D-42697 Solingen, Germany. Tel: +49 212-260 660 Fax: +49 212-260 6666; Email: info@dpc.de
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dyalog Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL, UK. Tel: 01256-811125 Fax: 01256-811130 Email: sales@dyalog.com; support@dyalog.com
DynArray Corporation	Dr James Brown	16360 Monterey Rd. Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408)-782-6648 Fax: +1 (408)-782-6627 Email:info@DynArray.com
Entropy Software Limited	George MacLeod	Calle Viena 13, Residencia Larrosa, Guardamar, 03140 Alicante, Spain. Tel: +34 61 00 41 940 Email: george.macleod@nlworld.com
Evestic AB	Olle Evero	Berteliusvagen 12A, S-146 38 Tullinge, Sweden Tel & Fax: +46 778 4410 Email: olle.evero@mail.com
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi
First Derivative Analytics Ltd.	Ken Chakahwata	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-339620. Email: KenChakahwata@compuserve.com
General Software Ltd	M.E. Martin	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 749365 E-mail: martin@gsoft.freeserve.co.uk
Godin London Incorporated	Gaëtan Godin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-6381 Email: info@godin.on.ca
HMW Computing	Chris Hogan	Hamilton House, 1 Temple Avenue, London EC4Y 0HA, UK. Tel: 0870-1010-469; Email:HMW@4xtra.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominique, Salisbury Road, Woking, Surrey, GU22 7UR, UK. Tel: 01483-771028. Fax: 01483-837324 Email: Bob.Hoekstra@HoekstraSystems.ltd.uk
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL, UK. Tel: 01536-770998 Email: Michael@Hughes.uk.com
I-APL Ltd	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. Email: 100612.1057@compuserve.com
IBM APL Products	Nancy Wheeler	APL Products, IBM Silicon Valley Lab, Dept H36/F40, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 [+1 (408) 463-2752] Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com
INFOTROY	Alexey Miroshnikov	3 S. Tulenln Lane, St. Petersburg 191186 Russia. Tel: +7 812 325-9797 Fax: +7 812 311-2184 Email:aim@infotroy.ru
Insight Systems ApS	Helene Boesen	Nordre Strandvej 119G, DK-3150 Hellebæk, Denmark Tel: +45 70 26 13 26 Fax: +45 70 26 13 25 Email: info@insight.dk
JAD Software	David Crossley	175 East 98th St., Apt. 17G, New York, NY 10128 Country: USA Tel: +1 (212) 369-6713 Fax: +1 (212) 761-0124 Email: jadsms@usa.net

Japan APL Assoc	Toshio Nishikawa	1-8-13 Masujima Buld.6F Higashi Gotanda Shinagawa-ku, Tokyo Japan 141-0022. Tel: +81 (03) 3280-0411 Fax: +81 (03) 3280-0418 Email: KYY00361@niftyserve.or.jp
J Austria	Joachim Hoffmann	Hochsteingasse 13/26, 8010 Graz, Austria. Tel:0043 (0)316 91 42 51 Mobile: 0043 (0)699 1 91 42 51 2. Email: joachim.hoffmann@gmx.at
JSoftware Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@jsoftware.com
KJK-tieto Oy	Kimmo Kekäläinen	Merikasarminkatu 10 B 56,00160 Helsinki, Finland. Tel: +358 50 55 27 207; Email: Kimmo.Kekalainen@pp.htv.fi
Lambent Technology Ltd	Stephen Taylor	81 South Hill Park, London NW3 2SS, UK. Tel: +44(0)20 7813 3786. Email: stj@lambenttechnology.com
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 8JY, UK. Tel: 01992-633807 Fax: 0121-359 0375 Email: phil.last@net.ntl.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33.1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Lingo Allegro USA, Inc	Walter G. Fil	203 N. LaSalle Street, Suite 2100, Chicago, Illinois 60601, USA. Tel:+1 (800) 546 4621 E-mail: lingo-allegro@visto.com
Lucas Solutions	Jim Lucas	Stubbedamsvej 9C, 3.tv., 3000 Helsingør, Denmark Tel: +45 49 26 52 42. Email: j@danbbs.dk
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0)131 228 5235 Email: alastair.kinloch@btinternet.com
MasterWork Software Ltd	Fraser Jackson	PO Box 56-036, Tawa, Wellington, New Zealand. Tel: +64 (4) 232-4440 Fax: +64 (4) 232-4452 Email: fraser.jackson@xtra.co.nz
Mercia Software Ltd.	Gareth Brentnall	Mercia House, Ashted Lock, Aston Science Park, Birmingham, B7 4AZ, UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	Richard Nabavi	The Roller Mill, Mill Lane, Uckfield, E.Sussex TN22 5AA Tel: 01825 768050. Fax: 01825 749472 Email: MicroAPL@microapl.demon.co.uk
Millinta Inc.	Dan Baronet	Contact Dan Baronet at danb@millinta.com
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01730-263843 Email: Ellis@mrilfrn.demon.co.uk
NY Sig	David Siegel	PO Box 2697, New York, NY10163-2697, USA. Email: NYSIGAPL@ACM.ORG
Oasis b.v.	Louis Rijkse	Lekstraat 4, 3433 ZB Nieuwegein, Holland. Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: rijkse@oasis.nl
Omega Computing Inc	Alan Graham, Andrew Chou	3 Columbus Avenue, Edison, NJ 08817, USA. Tel: +1 (732) 985 9519 Email: alangraham@mindspring.com
Optima Systems Ltd	Paul Grosvenor	Optima House, Mill Court, Spindle Way, Crawley, West Sussex, RH10 1TT, UK. Tel: 01293 562700 Fax: 01293 562699 Email:paul@optima-systems.co.uk
Qualedi Inc.	Nicole Schless Georges Brigham	121 West Main Street, Milford, CT 06460, USA. Tel: +1 (203) 874-4334 Fax: +1 (203) 876-9083. Email: sales@qualedi.com; info@qualedi.com
RadSys Technologies AB	Randolph Schrab	Lövsångarv. 18, S-756 52 Uppsala, Sweden. Tel: +46 18 32 41 53 Fax: +46 708 1996 11 Email: randolph.schrab@radsys.se
Renaissance Data Systems Ed Shaw		P.O. Box 511, Botsford, CT 06404, USA. Tel: +1 (203) 270-9729 Email: aplbooks@earthlink.com
Resources & Results	Frank Rhodes	2438 W. Northgate Dr., Irving, TX 75062, USA. Tel: +1 972 523 5117 Email: frank@decision-support.net
RE Time Tracker Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
Right Seat Software, Inc.	Tom Atkins	1110 12th Street, Golden, CO 80401, USA. Tel:+1 303 278 2244. Fax: +1 303 278 6967. Email: info@voxyproxy.com
The Rochester Group Inc.	Robert Miller	600 Park Avenue, Rochester, NY 14607-2926, USA. Tel: +1 (716) 271-1110. Fax: +1 (716) 271-1230
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00149-Roma Trullo, Italy Email: mario.sacco@tin.it
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapldoc@aol.com
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 8th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 Email: ashepp@compuserve.com

Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@interlog.com
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA. Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Soliton Associates	Benoit Paquin	Soliton Denmark, Havsgårdsvej 4, 2900 Hellerup, Denmark Email: sales@soliton.com
SovAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region 249020, Russia Tel: +7(08439)47109 Fax: +1 (530) 6885510 Email: askom@obninsk.com
Russian Chapter of SIGAPL		
Strand Software Inc	Anne Faust	P.O. Box 330, Excelsior, MN 55331 USA Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@strandsoft.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rex@rexswain.com
SwedAPL	Christer Ulfhielm	Novator Consulting Group AB, Svårdvägen 11C, S-182 33 Danderyd Sweden, Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CServer: 100341,404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: sig@ifi.unizh.ch
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: info@torontoapl.org
Welghahead Systems	Philip Bulmer	Camberley House, 1 Portesbery Road, Camberley, GU15 3RB, UK. Tel +44 1276 20789 Email: sales@welghahead.com
Stephen Wynn		8 Clarence Gardens, Brighton, Sussex BN1 2EG, UK. Tel: 01273-327238 Email: centre@cwcom.net
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

FTP SITES

IBM APL2	ftp.software.ibm.com/ps/products/apl2
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascii

WORLD WIDE WEB SITES

ACM SigAPL	www.acm.org/sigapl/
Adapta Software	www.adapta.de/
Adaptable Systems	www.assuredsystems.com.au/
AFAPL	www.afapl.asso.fr/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.demon.co.uk/apl385/
APL Journal, Germany	www.rhombos.de/apljourn.htm
APL Borealis Inc.	www.aplborealis.com
APL Systems IDC SL	www.apl2000.net
AUSCAN	www.interlog.com/~rjp/auscan/
Eke van Batenburg	www.bio.LeidenUniv.nl/~Batenburg/index.html
Causeway	www.causeway.co.uk/
CODEWORK	www.codework-it.com/tangram/eng/
CoSy (Bob Armstrong)	CoSy.com/
Dinosoft Oy	www.dinosoft.fi/
Dittrich & Partner	www.dpc.de ; www.apl-online.de
DMOZ - Open Directory	http://dmoz.org/Computers/Programming/Languages/APL/
Dyalog Ltd	www.dyalog.com/
DynArray	www.dynarray.com/
FinnAPL	www.pyr.fi/apl/

Godin London Inc	www.godin.com/
Houben (IQL)	www.apl.olap.club.tip.nl
Hoekstra Systems	www.HoekstraSystems.ltd.uk/
IBM APL2	www.ibm.com/software/ad/apl
Infostroy	www.infostroy.ru
Insight Systems ApS	www.insight.dk/
Japan APL Association	www.naska.co.jp/JAPLA/
JSoftware Inc	www.jsoftware.com/
KJK-tieto Oy	www.kjk-tieto.com
Lambent Technology	www.lambenttechnology.com
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo.com/
Mackay Kinloch	http://mackaykinloch.ltd.uk/
MicroAPL Ltd	www.microapl.co.uk/apl
Milinta Inc	www.milinta.com
Oasis b.v.	www.oasis.nl/
Optima Systems Ltd	www.optima-systems.co.uk
Qualedi, Inc.	www.qualifiedi.com
Renaissance Data	www.aplbooks.com/
Resources & Results	www.decision-support.net
RE Time Tracker Oy	www.rett.fi/
Right Seat Software, Inc.	www.voxproxy.com
The Rochester Group Inc.	www.rochgrp.com/
Shepp & Associates	www.digitravel.com/
SigAPL	www.acm.org/sigapl/
Soliton	www.soliton.com/
Snake Island Research Inc.	www.snakeisland.com
Strand Software Inc.	www.strandsoft.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.org/
Jim Weigang	www.chilton.com/~jimw/
Weighahead Systems	www.weighahead.com

ZARK Newsletter Extracts

introduced by Jon Sandles

Here is another Zark newsletter extract. As always, don't hesitate to send in any solutions you may have that you may be inspired to investigate. There may be new more elegant ways of solving an old problem, which may also turn out to be faster. Please don't be afraid to try and shed "new" light on the "old" problems. Once again, thanks to Jonathan Barman for helping typeset this column.

Utility Corner: Exporting Numbers

(The purpose of this column is to make you more productive by introducing you to utility functions. Think of utility functions as APL functions that have names instead of symbols. By expanding your function vocabulary, you'll be able to write APL code that's more concise, more efficient, and more readable.)

In last issue's *Limbering Up* column, you were asked to define the utility functions TOCOMMA and FROMCOMMA. These functions facilitate your ability to communicate with the outside (non-APL) world.

Suppose you have a matrix of number that you want to export from APL to a non-APL application. The function TOCOMMA translates the numeric matrix to a "comma-delimited text file" (a character vector) that can be read by the non-APL application.

For example, if this is the numeric matrix to convert,

```
      NMAT
123  5000 60 90793
124   400 60 31593
125 39900 71 42293
```

The TOCOMMA function returns the following comma-delimited character vector:

```
      TOCOMMA NMAT
123,5000,60,90793

124,400,60,31593

125,39900,71,42293
```

(The lines appear double-spaced because of the inclusion of both carriage return and line feed characters at the end of each line. ASCII files typically contain both of these characters at the end of each line.)

FROMCOMMA does the inverse of TOCOMMA. Given a comma and carriage return/line feed delimited character vector as its argument, it returns the numeric matrix of the numbers within the character vector. For example,

```

      CVEC+TOCOMMA NMAT
      FROMCOMMA CVEC
123  5000 60 90793
124   400 60 31593
125 39900 71 42293

```

Obviously, TOCOMMA and FROMCOMMA do not solve the problem entirely. The character vector must still be written to or read from a file. In APL*PLUS PC or APL*PLUS II, this can be accomplished as follows:

Exporting a numeric matrix:

```

A Convert the numeric matrix to a
A comma-delimited character vector:
  CVEC+TOCOMMA NMAT

A Create a "native" (DOS) file:
  'C:\FILENAME.TXT'⎕NCREATE ~1

A Write the character vector to the
A file and deactivate the file
  CVEC ⎕NAPPEND ~1
  ⎕NUNTIE ~1

```

Importing from a comma-delimited file:

```

A Activate the "native" (DOS) file:

  'C:\FILENAME.TXT'⎕NUNTIE ~1

A Read its contents as a character
A vector, and deactivate the file:
  CVEC+⎕NREAD ~1,82,⎕NSIZE ~1
  ⎕NUNTIE ~1

A Convert the comma-delimited charac-
A ter vector to a numeric matrix:
  NMAT+FROMCOMMA CVEC

```

Let's look at the FROMCOMMA function first, since its logic is somewhat simpler:

```

▽ NMAT←FROMCOMMA CVEC;D;N;IO
[1]  A Converts comma-delimited char vec
[2]  A CVEC, whose records are delimited
[3]  A (followed) by carriage return/line
[4]  A feed paris, into a numeric matrix
[5]  A with one row per record.
[6]  IO←1
[7]  A Grab the first record (IOTC[2]=
[8]  A IOCNL=carriage return):
[9]  N←(1+CVectIOTC[2])÷CVEC
[10] N[(N=',' )/1pN]←' ' A Repl ',' by ' '
[11] N←N A ...and cvt to num (or IOFI)
[12] N←pN A This many columns in result
[13] A Replace all delimiters by blanks
[14] A (IOTC[3]=IOCLF=line feed), and also
[15] A blank out the EOF char (IOAV[27]):
[16] D←',' ,IOTC[2 3],IOAV[27]
[17] CVEC[(CVEC÷D)/1pCVEC]←' '
[18] NMAT←,CVEC A Convert to numeric
[19] A Reshape to a matrix:
[20] NMAT←((pNMAT)÷N),N)pNMAT
▽

```

On lines [9] to [12], FROMCOMMA determines how many numbers there are per record. It grabs the first record (up to the first carriage return character), replaces the commas by blanks, converts the characters to numbers by executing them (though the APL*PLUS system function IOFI would do the job just as well, and probably more efficiently, since it's simpler), and checks the shape of the result. FROMCOMMA makes the assumption that all records contain the same number of numbers. The result of $\div$ is ravelled to handle the case in which the record contains just one number. The logic on line [12] assumes that N is a vector.

On lines [16] to [18], the entire character vector is converted to a numeric vector in one fell swoop. To do this, the non-digit characters (commas, carriage returns, line feeds, and EOF [end-of-file] markers) are replaced by blanks, and the result is executed. Again, IOFI could be used here in place of $\div$. The EOF marker (IOAV[27]) is included at the end of DOS ASCII/text files by most non-APL applications.

On line [20], the numbers are reshaped into a matrix, with one row per delimited records. The number of records is determined by dividing the total number of numbers by the number of numbers per record.

To TOCOMMA function is a bit trickier:

```

      V CVEC←TOCOMMA NMAT;I;N;R;⍺IO;⍺PP
[1]  A Converts numeric matrix NMAT into
[2]  A a comma-delimited character vector.
[3]  A Each row of NMAT becomes one record
[4]  A in the result, where the records
[5]  A are delimited (followed) by a
[6]  A carriage return/line feed pair.
[7]  ⍺IO←1
[8]  N←⌈1⌈pNMAT A No. of columns in NMAT
[9]  ⍺PP←17 A Set print precision to max
[10] A Num mat → char vec, with one blank
[11] A after each number:
[12] CVEC←(⌘,NMAT), ' '
[13] I←(CVEC=' ')/1pCVEC A Inds of blanks
[14] CVEC[I]←', ' A Insert comma delims
[15] A Inds of trlmg comma for each rec:
[16] I←((pI)ρ(-N)⌈1)/I
[17] A I←I[N×1(pI)÷N] A Alternative
[18] R←(pCVEC)ρ1 A Replication vector
[19] R[I]←2 A Double trailing commas
[20] CVEC←R/CVEC A ... for CR/LF
[21] A Adjust indices for expanded vec:
[22] I←I+1pI A Indices of LF's
[23] A Replace comma-pairs by CR/LF pairs
[24] A (⍺TC[2 3]=⍺TCNL,⍺TCFL=CR/LF):
[25] CVEC[I÷.-1 0]←((pI),2)ρ⍺TC[2 3]
      V

```

On line [12], the numeric matrix is ravelled and converted to a character vector all at once. Before doing so, though, the value of αPP is set to its largest setting (16 on some APL systems) so numbers with decimal places will be formatted to the maximum possible precision. For example, the number from $22 \div 7$ will be formatted as 3.1428571428571430, instead of the default 3.142857143.

On lines [13] and [14], the blank delimiters are replaced by commas. Since monadic $\lceil$ returns one blank between each number, and since a blank is catenated to the end of the vector on line [12], there is now one comma after each number.

On line [16] (or [17] if that algorithm is more efficient on your machine), the vector of indices of the comma delimiters is compressed to retain just the indices of those commas that mark the end of each record. Each of these commas must be replaced by a carriage return(CR) and a line feed (LF).

To replace single commas by CR/LF pairs, the character vector must be expanded. On lines [18] to [20], replicate (/) is used to perform this expansion by doubling up each of the end-of-record commas.

Once the character vector is expanded, the indices that point to the end-of-record commas are no longer valid. On line [22], the indices are adjusted (adding 1 to the first, 2 to the second, and so on) so they point to the second of the two commas that now exist at the end of each record.

On line [25], outer product is used to build a two-column matrix of the indices of both end-of-record commas. A corresponding two-column matrix of CR/LF pairs is constructed and inserted. This direct insertion of a matrix into a vector is possible because indexed assignment ($A[B] \leftarrow C$) permits its index argument and its assigned argument to be of any rank, as long as they are scalar compatible (i.e. as long as the expression $B=C$ would not cause an error).

ToComma and FromComma

from Jon Longley and Mahesh Navani - Jennison Associates

```

▽ R←FROMCOMMA X;COLS;ROWS
[1] COLS←(ROWS++/' ','=X)÷ROWS++/□TC[2]=X
[2] ((,Xε','',(□TC[2 3]))/,'X)←' '
[3] R←(ROWS,COLS)⍉X
▽

▽ K←TOCOMMA K

[1] K←∘,/(-1↓'',/(⊘K),'','),''=□TC[2 3]

▽
```

ToComma and FromComma

from Adrian Smith - Causeway

This one bothers me a little, because it seems to understate (quite substantially) the problem. The real challenge is in dealing with the mess that Excel throws out when you ask to save a CSV file and choose comma rather than tab as the separator. Things get a lot worse when there is a chance that someone may mess with the file by hand, but even in the 'simple case' you must deal with garbage like this:

This is a nasty file!

```
When,How much,Who
15-Sep-2000,$4.00,Fat cat
17-Sep-2000,$3.00,"7, Castlegate"
19-Sep-2000,$4.50,"Who put that '""' there?"
21-Sep-2000,"$1,500.00"
23-Sep-2000,$6.00
25-Sep-2000,$7.00
27-Sep-2000,"$23,016.45"
29-Sep-2000,$5.00
01-Oct-2000,"$4,321,000.00"
```

My 'bitter experience' code from GraPL seems to deal with most of what users have thrown at it over the last 2 years, and reads files up to around 10,000 rows by 20 columns in a reasonable time (faster than Excel can, anyway). It looks awfully over-engineered by comparison, but I think I can claim that in the real world (see Stephen Taylor on XP) it may be "the simplest thing that possibly works" ...

```
▽ r←ParseTxt crv;tb;n1;lf;mat;dd;qq;cols;flds;rows;pad;xle
[1] A Handle tab and comma-separated data, returning a matrix of text cells
[2] A Space-separated takes any run of spaces as the delimiter - June 2002
[3] A Returns the number of separators in each record for later filtering
[4] (tb n1 lf)←JAV[10 4 3]
[5]
[6] :If n1←crv A Not a Unix file
[7]   crv←~lf
[8] :End
[9]
[10] :While n1~1←crv
[11]   crv←~1←crv
[12] :End
[13]
[14] :If ~tb←crv A Turn CSV into TSV
[15]   dd←crv←','
[16]   :If ~v/dd A No commas either!
[17]     A GNU_Plot accepts blank-separated lists!
[18]     A Also handy for APL-formatted matrices!
[19]     crv←tb WhiteSpaceToDelim crv
[20]     dd←crv←tb
[21] :End
[22] qq←~\crv←'"'
[23] ((dd^~qq)/crv)←tb
[24] (('"'⊖crv)/crv)←'o'
[25] (('o'⊖crv)/crv)←'"'
[26] crv←~'"'
[27] ((crv='o')/crv)←'"'
[28] :Else A Just clear out quotes (Excel uses these in CSV)
[29] (('"'⊖crv)/crv)←'o'
[30] (('o'⊖crv)/crv)←'"'
[31] crv←~'"'
[32] ((crv='o')/crv)←'"'
[33] :End
[34]
```

```

[35] A Sort out dates which arrive in 'Jan-01' format (Excel throws these)
[36]   crv←(tb,nl,'.-/ ')FixDates crv
[37] A Sort out numbers which arrive in '$23.56' format
[38]   crv←(tb,nl,'-')FixNums crv
[39] A Now split it on the newlines and tabs
[40] A First count the tabs in each record (classic partitioned sum)
[41]   dd←2÷0,(1,⌊crv=nl)/+⌊(0,⌊crv=tb) ∘ cols÷1÷f/dd
[42] A Now split on either delimiter
[43]   mat←tb n! csvton crv
[44]   flds←1+dd
[45]
[46] A Special case for Excel nulls - treat as zero for now
[47] A The '@' version comes from old 123 sheets which Excel still supports
[48]   xle←'#N/A' '#VALUE!' '#REF!' '#DIV/0!' '#NUM!' '#NAME?' '#NULL!' '@NA'
[49]   ((matexle)/mat)+,c,'0'
[50]
[51] :If fldsv.≠cols A Expand mask time (but beware overtake) ....
[52]   pad←c''
[53]   mat←1+(1,ecolst"fldsp"1)\pad,mat
[54] :End
[55]
[56]   rows←⌊(pmat)÷cols
[57]   mat←(rows,cols)pmat
[58]   r←(mat dd)

```

▽

I won't bore readers with the subfunctions to fix up Excel-formatted dates, or numbers with currency decorators. The workspace is free to anyone who could use it - email me at Causeway for the latest (probably even more defensive) copy.

Limbering Up: Numbers to Words

(The purpose of this column is to work some flab off your APL midsection. Like muscles, your APL skills can atrophy if not exercised with adequate frequency and variety. This column presents a task for you to perform. Set aside a few minutes from your busy schedule and work the task. Mail in your solution and stay tuned for the results.)

When your pay check comes in, the amount is generally expressed both in terms of digits and in terms of words. Like so:

\$1,234.56

One thousand two hundred and thirty four and 56/100 dollars.

As APL programmers, we don't have much trouble formatting the amount as a bunch of digits. For example:

```

      AMT←1234.56
      A APL*PLUS
      'P<£>CF10.2' □FMT AMT
£1,234.56

```

```

      A APL2:
      '£55,553.50' *AMT
£1,234.56

```

The words, however, present more of a programming challenge.

Your task is to define a utility function named NUMBERWORDS, which takes a single positive integer as its right argument and returns a character that describes the number. For example:

```

      NUMBERWORDS 1234
one thousand two hundred thirty four
      NUMBERWORDS 1007
one thousand seven
      NUMBERWORDS 0
zero
      NUMBERWORDS 1003011
one million three thousand eleven

```

Given such a function, the task of formatting a cheque amount into words is straightforward:

```

      V R←CHECKAMOUNT N;C;D
[1]  A Format a $ amount (e.g. 1234.56)
[2]  A into words (e.g. 'One thousand two
[3]  A hundred thirty four and 56/100
[4]  A dollars'). Needs NUMBERWORDS.
[5]  A
[6]  D←[N A Dollars
[7]  C←100×N-D A Cents
[8]  D←NUMBERWORDS D A Dollars to words
[9]  A Capitalize first letter:
[10] D[□IO]+'EFNOSTZ'['efnostz'∘D[□IO]]
[11] A PUT it all together:
[12] R←D,' and',(0≠C),'/100 dollars'
      V

```

```

      CHECKAMOUNT 1234.56
One thousand two hundred thirty four and 56/100 dollars

```

The NUMBERWORDS function can be fast or slow, elegant or inelegant, short or long. All that matters is that it produces the correct result.

Send your solution to:

Vector Production
 Brook House
 Gilling East
 York YO62 4JJ
 UK email apl385@compuserve.com

The notable functions and their authors' names will be printed in the next issue of *Vector*. Good luck and happy limbering.

Reprinted with kind permission from Zark APL Tutor News, a quarterly publication of Zark Incorporated, 23 Ketchbrook Lane, Ellington, CT06029, USA.

Numberwords with DFns

a provocation to readers from Jonathan Barman

This entry for the NUMBERWORDS Zark Limbering Up challenge demonstrates a way of using Dyalog Direct Definition Functions (D-Fns) and compares this with a traditional APL solution.

```

∇NumberWords←{
[1]  A Spell a scalar integer in words.
[2]  A Optional left argument of 1 to include 'and'
[3]    □IO←1 ∘ □ML←1
[4]    0⇒ω:'zero'      A Return if zero
[5]    α←0             A Default left argument
[6]    and←α/'and '    A 'and' text, if required
[7]    htu←{ A Fn to spell numbers of less than 1000
[8]      u←'one ' 'two ' 'three ' 'four ' 'five ' 'six '
        'seven ' 'eight ' 'nine ' 'ten ' 'eleven '
        'twelve ' 'thirteen ' 'fourteen ' 'fifteen '
        'sixteen ' 'seventeen ' 'eighteen ' 'nineteen '
[9]      t←'ten ' 'twenty ' 'thirty ' 'forty ' 'fifty '
        'sixty ' 'seventy ' 'eighty ' 'ninety '
[10]     A Hundreds to words
[11]     r←{ω≥100:((⌊ω÷100)≠u),'hundred ' ∘ ''}ω
[12]     A Tens and units
[13]     n←100|ω
[14]     n>19:r,and,((⌊n÷10)≠t),{ω>0:ω≡u ∘ ''}10|n
[15]     n>0:r,and,n≡u  A Teens

```

```

[16]      r}          A Exact hundred
[17]      mag←{ A Fn to spell numbers according to magnitude
[18]      rtxt tval←ω A Rt Arg. Result, text and value
[19]      mtxt mval←α A Lt Arg. Magnitude, text and value
[20]      mval>tval:ω A Magnitude greater so return rt arg
[21]      A Return result text and new test value
[22]      (rtxt,(htu|tval≠mval),mtxt)(mval|tval)}
[23]      A Use Reduction / to apply <mag> for each magnitude.
[24]      A Remove leading 'and'. Drop trailing space.
[25]      -1+{((pand)×'a'≡ω)+ω}>=mag/' ' 'thousand ' 'million '
[25]      'billion ' 'trillion ' ''{α ω}''(10*0 3 6 9 12),=ω
[26] }
∇

```

All the action occurs on line [25] where Reduction (/) is used to loop through each order of magnitude, starting with trillions and ending with units. The {α ω}'' on that line is a D-Fns idiom which converts the two vectors into a vector of pairs of items.

The function mag is first applied between the rightmost two elements of the vector of pairs, effectively

```
('trillion ' (10*12)) mag '' ω
```

The 'and' option can be used to write English style cheques rather than the American checks given in the Zark challenge:

```
ChequeAmount 1234.56
One thousand two hundred and thirty four pounds 56 pence
```

The following traditional (non-nested) APL solution has exactly the same structure as the D-Fns solution, though without the 'and' option:

```

∇ R=NUMBERWORDS N;MVAL;MTXT;UNITS;TENS;X;I;IO
[1]  IO←1
[2]  A Exit if zero
[3]  →(N≠0)/L1
[4]  R←'zero'
[5]  →0
[6]  A Set up names
[7]  L1:UNITS←19 10p190↑'one      two      three      four
    five      six      seven      eight      nine      ten
    eleven     twelve     thirteen    fourteen    fifteen    sixteen
    seventeen  eighteen  nineteen'
[8]  TENS←9 8p72↑'ten      twenty    thirty    forty    fifty
    sixty     seventy    eighty    ninety'
[9]  MTXT←5 9p45↑'trillion billion million thousand'
[10] MVAL←10*12 9 6 3 0

```

```

[11] A Loop for each magnitude
[12] R←'' A Set up result
[13] I←1 A Loop counter
[14] A Skip if value less than magnitude
[15] L2:→(N<MVAL[I])/L6
[16] X←[N÷MVAL[I] A Number of mag
[17] →(X<100)/L3 A Hundreds?
[18] R←R,UNITS[[X÷100;],' hundred '
[19] L3:X+100|X A Tens and units
[20] →(X<20)/L4 A Teens?
[21] A 20 or more, so tens and units
[22] R←R,TENS[[X÷10;],UNITS[10|X;]
[23] →L5
[24] A 19 or less. Check not zero
[25] L4:→(X=0)/L5
[26] R←R,UNITS[X;] A Teens
[27] A Append magnitude text
[28] L5:R←R,MTXT[I;]
[29] A Remove high order numbers processed
[30] N←MVAL[I]|N
[31] A Carry on with loop
[32] L6:→((pMVAL)≥I+1)/L2
[33] A Remove redundant spaces
[34] R←~1+(X<1,~1+X+R≠' ')/R

```

▽

Lines [15] to [30] of the traditional function are the equivalent of the mag D-Fn, with lines [17] to [26] matching the htu D-Fn called by mag.

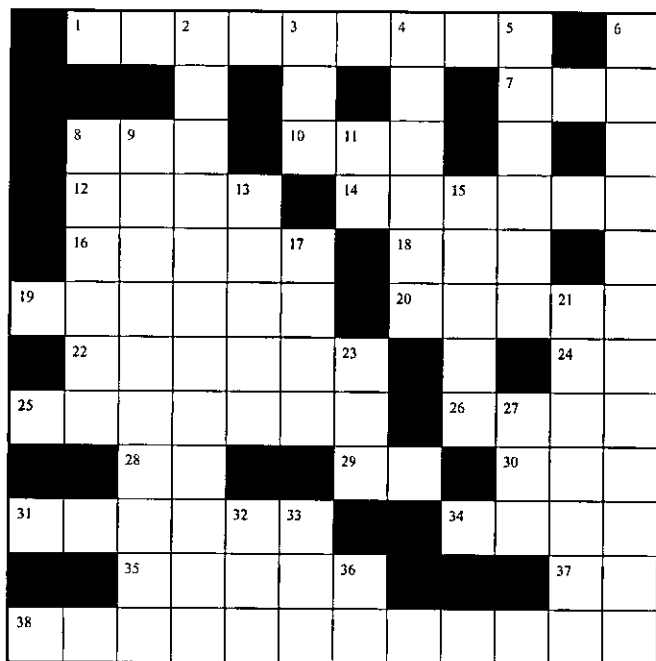
Which way is best? I can't make up my mind. D-Fns can be too 'clever' in which case they are difficult to maintain. Traditional APL tends to have lots of branching which can be equally difficult to maintain.

New Crossword and Solution for 19.3

Solution to Crossword in 19:3

	P	I	V	≠	1	↓	P	I	V	,	0
V	E	C	1	N	↑	M		W	A		€
S	←	R	L	R	E		(	ρ	M	)	ρ
↓		×	/		,		2	-	A	D	V
V		?	(	ρ	V	)	×	S	T	R	E
1	≠	R	V	0		S	ρ	D	,	O	C
	T		>			A	V	×	A	P	T
1	+	(	9	≠	N	V	)	1	1		O
0	R		9			E	ρ	B	Q	φ	R
1	S		)	S	I		1		T	L	,
⌈	/	+	/	^	\	'		'	=	C	M
E	L	E	V	E	N		0		A	P	L

New Crossword



Across

1. Smallest positive number in vector V
7. Cumulative product of vector V
8. Branch to line labelled JV
10. The larger of M or C
12. $(T+1), (T+2), \dots, (T+5)$ for scalar T
14. Convert indices in $SO1$ to origin 0
16. The remainder of $V2 \div GF$
18. All but first P elements of vector R
19. $M=M$ for any array M
20. $V[1;Yp1]$
22. $\sim(LB \div 2) * K + K$
24. $,*/p$
25. 10 20 30 40 ... 1230

26. ('MAT+12 11p-1'~'A1 p-')[2 3 1 4]
28. ~1+1+'APL2'
29. !4
30. V1[(1+pV1)-1pV1]
31. (14),5+14
34. ____pA is first element of A, as a scalar
35. ,(T-[T]),[1.5]T-[T] for vector T
37. __A returns A, but promotes scalar A to a vector
38. Distinct elements of character array CV

Down

2. Distinct elements of vector V, whose length is L
3. M[;1]>M[;2] for two-column M
4. Bring objects into active workspace without overwriting existing objects
5. (V*-R1)+(V*J)
6. Distinct elements of sorted,non-negative, numeric vector V
8. Branch to line labelled L0 if bit TF is 1
9. J-(MB*MB*P92A*P92A)*0.5
11. The next larger integer at or above S
13. K1[S2p1pK1]
15. Number of columns in matrix MT
17. G1+G1
21. Number of dimensions in matrix MV01A
23. (1+1+1+1+1)~' '
27. __X is (2+1X-2),12
32. ^/~
33. C-5*[C÷5]
36. Non-interactive full-screen interface

J-ottings 36: A Partial Solution to a Partial Problem

by Norman Thomson

A seemingly innocuous post on the J Forum asked if anyone had a general routine for resolving partial fractions. Given that the heavy power-horses of `p.` and `%.` are already present in J, it seemed to require just small extensions of these to solve the problem, and it may be that this is indeed the case. Nevertheless I found it to be quite a tricky exercise, and as the title above suggests, the path to a general partial fraction algorithm as given here is not quite complete. However, some of the J features which turn up on the way are interesting in their own right, notably the opportunities for use of explicit rank. First, the basic problem is relatively simple, namely that of rewriting a quotient of polynomials such as $f(x)/g(x)$ in which $f(x)$ is of lower order than $g(x)$, in a form such as

$$k_1/(a_1-x) + k_2/(a_2-x) + \dots$$

where $a_1, a_2, \dots$ are the roots of $g(x)$. Initially, assume that these are distinct.

Start by defining a "polynomial" as a vector of coefficients in ascending power order which is the meaning implicit in the right argument of `p.`, so that the algebraic function $f(x) = 2x^2 + 5x - 3$ is represented by the polynomial `_3 5 2`. The value returned by monadic `p.` is

```
p. _3 5 2
+-----+
|2|_3 0.5|
+-----+
```

that is the roots `_3 0.5` along with the highest order coefficient 2 which helps distinguish $f(x)$ from say $g(x) = 6x^2 + 15x - 9$, which has the same roots.

To model partial fractions, a generic type of decision has to be made between using boxes or lists. For example, the partial fraction $(1+x)/(1+3x+2x^2)$ could be represented either by a list of boxes

```
1 1;1 3 2
```

or by a list of polynomials

```
1 1 0
1 3 2
```

My general principle is to use lists wherever possible unless data-inherent heterogeneity makes lists too burdensome. Since the contents of boxes are sealed by definition, the only available operations without opening are the relatively simple ones of joining and shaping. With lists, operations specifically appropriate to the data types are fully available, subject to the constraints of structural rectangularity which may force the insertion of fill characters. These can sometimes be benign, as in the case of polynomials where adding a couple of zeros on the right of, say, the polynomial 1 0 2 merely adds two non-contributory terms $0x^3 + 0x^4$ to the function $1 + 2x^2$.

Using this representation of polynomials, a fraction such as $f(x)/g(x)$ is a 2-list of polynomials, for example $(4-x)/(1 + 2x^2)$ is the 2-list

```
4 _1 0
1 _0 2
```

Further, the model is readily extendible by representing a number of such polynomial fractions as a list of 2-lists of lists, for example $\{2/(4+x)\} - \{6/(1+5x)\}$ is

```
      t0
2      0
4      1
-6      0
1      5
```

Next, the dyadic verb **pm** (polynomial multiply) multiplies two polynomials and is often cited as an illustration of the adverb oblique **/**.

```
pm=.+//.@(*) NB. (dyad) polynomial multiplication
4 _1 pm 1 0 2
J4 _1 8 _2
```

(Assume in what follows that defined verbs are monadic unless there is a specific comment to the contrary.)

A useful first step in developing a partial fraction algorithm is to develop an inverse verb, that is one which combines basic (that is 2 by 2) partial fractions into a single composite partial fraction.

```

cp=.(+/@(pm"1|.)),:pm&(1&{)  NB. combine basic partial
fractions
cp/t0
_22 4 0
 4 21 5

```

The roots and multiplier of the denominator of a partial fraction are obtained by

```

pfd=.p.@(1&{)  NB. partial fraction denominator
of=.>@{pfd  NB. (dyad) pick from p. 0=multiplier, 1=roots

```

Thus if u1 represents the partial fraction $(11x + 8)/(3 - 2x - 8x^2)$:

```

u1
11 8 0
 3 _2 _8
   0 of u1  NB. multiplier
_8
 1 of u1  NB. roots of denominator
_0.75 0.5

```

Although having p. return a multiplier (which is already part of the input data) as well as roots adds complexity to its result, this approach is well justified by considering that the factorisation of an expression such as $3 - 5x - 2x^2$ is not unique – it could be the “obvious” factorisation of $(1 - 2x)(3 + x)$ or it could be $(0.5 - x)(6 + 2x)$, or $(2 - 4x)(1.5 + 0.5x)$ and so on. Thus in factorising a polynomial into linear factors (which is always possible because every nth. order polynomial has exactly n roots, allowing for possible complex values), the multiplier delivered by p. must be applied arbitrarily to one of the factors. The choice made here is to apply it to the first. A general verb for multiplying the head only of a list is most clearly expressed as an explicit function:

```

mhd=.dyad define  NB. multiply head of y. by x. with tail unchanged
(x.*{.y.}),.y.
)

```

following which roots are changed into factors by f1, which reverses signs and catenates 1s. and adjusted by the multiplier with facs:

```

f1=.,.&1@-@(1&of)  NB. turn roots into factors ..
facs=.(0&of)mhd f1  NB. ..and adjust for multiplier

```

If there are just two roots, everything is in place to find the factors :

```
      facs u1          NB. factors
    _6 _8
    _0.5 _1
```

.. and then the partial fraction coefficients :

```
      11 8 %.:facs u1  NB. coefficients
    _1.5 _4
```

so that the resolution is $\{-4/(-6 - 8x)\} - \{1.5/(-0.5 + x)\}$ or equivalently $\{2/(3 + 4x)\} + \{3/(1 - 2x)\}$

More generally, the numerator coefficients are equated to the combined coefficients of the denominator polynomial factors following multiplication with one factor at a time omitted. This gives two nice examples of the use of explicit rank, one to exclude each item in turn in the list of factors, then `pm` is used at rank 2 to multiply these factors together. For example, given the fraction $(23 + 55x + 8x^2)/(3 + 13x - 18x^2 - 40x^3)$ to resolve, define:

```
      u2          NB. partial fraction
    23 55 8 0
    3 13 _18 _40
      facs u2      NB. basic polynomial factors of u2
    _30 _40
    _0.5 1
    _0.2 1
```

At this point the power of matrix divide comes into play :

```
      mat=.pm/"2@(-."2 1~)@facs NB. Lin. eqns. for partial fractn. coeffs
      mat u2
    _0.1 _0.3 1
    _6 _38 _40
    15 _10 _40
```

.. and is consolidated in a verb

```
      pfc=.:@{0&{ } %.:@mat  NB. partial fraction coefficients
      pfc u2
    _20 _1.5 0.8
```

which gives the partial fraction coefficients which correspond to `facs u2` in order.

Finally the verb form brings coefficients and factors together in the basic partial fraction representation :

```
form=.(,:~,) "1 0 NB. (dyad) merge factors and coefficients
```

```
pf=.fac form pfc NB. partial fractions
```

```
pf u2
```

```
_20 0
```

```
_30 _40
```

```
_1.5 0
```

```
_0.5 1
```

```
0.8 0
```

```
0.2 1
```

It is easy to confirm that $cp/pf \ u2$ is identical to $u2$ and similarly for $u1$. As further confirmatoin using the first example $pf \ cp/t0$ is the same as $u0$ within constant factors :

```
pf cp/t0
```

```
10 0
```

```
20 5
```

```
_1.2 0
```

```
0.2 1
```

The problem of distinct real denominator roots has thus been fully dealt with, which leaves two matters outstanding, namely complex roots, and repeated roots.

For complex roots, take as an example $1/(1+x^5)$ represented by

```
u3=.2 6$1 0 0 0 0 0 1 0 0 0 0 1
```

```
u3
```

```
1 0 0 0 0 0
```

```
1 0 0 0 0 1
```

pf operates as for real roots

```
pf u3
```

```
_0.161803j_0.117557 0
```

```
_0.809017j_0.587785 1
```

```
_0.161803j0.117557 0
```

```
_0.809017j0.587785 1
```

```
0.2 0
```

```
1 1
```

```

0.0618034j_0.190211 0
0.309017j_0.951057 1
0.0618034j0.190211 0
0.309017j0.951057 1

```

If the implementation dependent assumption is made that imaginary complex pairs occur together, there is no problem in combining basic fractions into partial fractions with real coefficients as in :

```

cp/2{.pf u3          NB. a quadratic partial fraction
0.4 _0.323607 0
1 _1.61803 1

```

```

cp/_2{.pf u3          NB. another quadratic partial fraction
0.4 0.123607 0
1 0.618034 1

```

It is a straightforward matter to write a verb which post-processes the results of pf by combining each complex fraction with its neighbour.

Next, multiple roots, say the resolution of $(6+8x+3x^2)/(1+x)^3$ where the appropriate partial fraction structure is $k_1/(1+x)^3 + k_2/(1+x)^2 + k_3/(1+x)$. This is a relatively easy application of the binomial coefficients and matrix divide :

```

bc=.!/~@i.          NB. binomial coefficients
bc 3
1 1 1
0 1 2
0 0 1

```

and the relevant coefficients are found by

```

6 8 3 %.bc 3
1 2 3

```

that is, $(6+8x+3x^2)/(1+x)^3 = 1/(1+x)^3 + 2/(1+x)^2 + 3/(1+x)$.

More generally the binomial coefficients for the polynomial a b are found by

```

bco=.dyad define      NB. (dyad) coeffs. of (polynomial y.)^x.
r=.(,1),:y. [ i=.2
while. i<x. do.
  r=.r,({:r)pm y. [ i=.i+1 end.

```

so for a partial fraction $(4 + 18x + 9x^2)/(2 + 3x)^3$, the first step is the matrix

```

      3 bco 2 3
1    0 0
2    3 0
4   12 9

```

followed by matrix divide to obtain the coefficients :

```

      4 18 9%.:3 bco 2 3
_4 2 1

```

that is $(4 + 18x + 9x^2)/(2 + 3x)^3 = \{-4/(2 + 3x)^3\} + \{2/(2 + 3x)^2\} + \{1/(2 + 3x)\}$

Notice that bco depends only on pm and not on any of the factorisation verbs. If the denominator is multiplied by a further factor, say resolve $(4 + 14x + 27x^2 + 18x^3)/(x+1)(2 + 3x)^3$ into partial fractions, each of the polynomials listed in 3 bco 2 3 must be multiplied by the new factor (hence pm"1), and a third order set of binomial coefficients added :

```

      m1=.(3 bco 2 3)pm"1(1 1)),{:4 bco 2 3
      4 14 27 18 %.:m1
4 _2 _1 1

```

that is the resolution is $\{4/(2 + 3x)^3\} - \{2/(2 + 3x)^2\} - \{1/(2 + 3x)\} + \{1/(1 + x)\}$.

At the start I said that the journey was not complete, but at least a staging post has been reached from which it is just a matter of conscientious programming to achieve a general partial fraction algorithm. In conclusion here is a list of the verbs contained above :

```

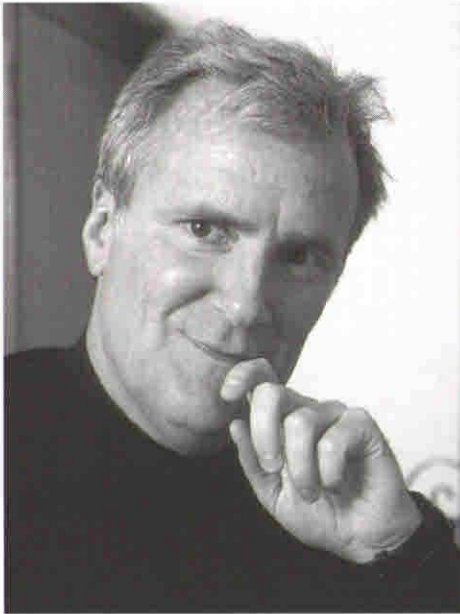
pm=+//. @(*)/      NB. (dyad) polynomial multiplication
cp=.(+/(pm"1|.)).:pm&(1&{) NB. combine basic partial fractions
pfd=.p.@(1&{)      NB. partial fraction denominator
of=>.@(pfd         NB. (dyad) denominator: 0=multiplier, 1=roots
mhd=.dyad define   NB. (dyad) multiply head of y. only by x.
(x.*{.y.)).y.
)
f1=.,. &1@-@(1&of) NB. convert roots to factors ..
facs=.(0&of)mhd f1  NB. .. and adjust for multiplier
mat=.pm/"2@(-."2 1~)@facs NB. lin. eqns for partial.fractn. coeffs
pfc=.:@{0&{) %.:mat NB. partial fraction coefficients
form=.(,,:~)"1 0   NB. (dyad) merge factors and coefficients
pf=.facs form pfc  NB. partial fractions
bco=.dyad define    NB. (dyad) coeffs. of (polynomial y.)^x.
r=.(,1),:y. [ i=.2
while. i<x. do.
  r=.r,({:r)pm y. [ i=.i+1 end.

```

RECENT MEETINGS

K Watch: Arthur Whitney in London

by Steph Johnson for Kx Systems (steph@stephjohnson.com)



Nearly 70 customers, prospects and long-time K supporters braved an unexpectedly icy London night to attend Kx Systems' official European launch party in January. Arthur Whitney gave a short presentation highlighting the strengths of the technology he created and has been enhancing for the past 10 years. Then he opened the floor to questions.

For some in the crowd, Whitney's claims of speed and scalability for K-based products such as kdb and kdb/tick initially bordered on the unbelievable. Fortunately, the Kx CTO's confident and good-humored responses to questions quickly built rapport between him and the audience, which was composed primarily of executives from London financial firms.

Kx executives, including Whitney and CEO Janet Lustgarten, were in London to meet with a number of new UK customers, including JPMorgan, and to talk with journalists about the company's increased commitment to European firms. Whitney and Lustgarten also wanted to express solidarity with their UK-based global sales and support partner, First Derivatives (FD). FD head of operations Michael O'Neill and his staff had invited most of the customers and prospects to the reception. The day before, O'Neill was at the FinExpo financial technology show in Canary Wharf, where he gave a software demonstration of a risk management system built on kdb.

Long-time K watchers and British APL Association members were also present among the crowd that took advantage of the opportunity to hear Whitney and stay for the reception. These included Stephen Taylor and Ray Cannon. In addition to greeting Whitney, they were able to catch up with Simon Garland, who recently joined Kx as SVP of Technical Support.

Kx has witnessed increasing acceptance of its technology, particularly during the past two years, and has achieved significant growth. "Companies today have big data problems they can't solve," Whitney said. "Then they find Kx. There's a new generation of users getting into vectors."

Forests in Finland VII – March 2003

reported by Adrian Smith

Introduction

It all started with a mail which mentioned "The event takes place in the stables of "Svartå Slott". See <http://www.mustionlinna.fi/> ..." and listed Sasha as the keynote presenter. This year, the meeting was around a month later than usual, but as you can see from the picture, the sea remained mostly well frozen, and a sun-tan could only be acquired by the truly dedicated.



The meeting returned to the Svartå Manor, which is a comfortable country hotel, with excellent facilities, about an hour out of Helsinki. As ever, there were lots of useful discussions in the breaks, and some solid APL material in the presentations. My notes inevitably reflect my total ignorance of the Finnish language, so I can only encourage under-represented speakers to make a rough English translation and send it to the editor for publication.

The Presentations

After a brief welcome from Jouko Kangasniemi (see the group photo at the end of the notes) we were shown around the Adaytum Planning 'Cube' editor and 'e-planning' on-line submission tool by Richard Eller, who is the agent for Adaytum in Finland. Now that all this is part of the Cognos empire, I guess that at least the artwork will soon need to change on his slides! There followed a coffee break, and a warm welcome for Sasha (Professor Skomorokhov) who had just managed to defeat an obdurate bureaucracy in time to get his passport renewed.

Sasha, on APL in Co-operation with Matlab and S+



I can only try to use bold and italic here to represent the characteristic 'Russian bear' emphasis that makes Prof. Skomorokhov such a great speaker to listen to.

"Data mining and machine learning is APL a good tool? *Yes!* Are there other good tools? *Yes!* Do we need to use more tools? *Yes!* Let me prove why we need some extra tools, and how it may help us."

Sasha noted that S+ has over 4000 statistical functions built in. All the statistics routines available in the APL world probably amount to less than 1000, including the ASL libraries and many more. MatLab has 500,000 users around the world, and its the standard for technical computing. If you want Wavelet

transforms, APL can do the maths, but visualisation is still an issue.

These tools can be compiled, so they are much faster than APL and have dynamic memory management. They can be run from script files. What is more, the version of S+ called **R** is free under the GNU public licence, has an extremely active development group (over 100 posts every day to the new group), and some routines are already better than those in S+.

So why use **R** from APL? Because "you can also use *APL* from APL!". MatLab users have awful trouble inventing vector code, and **R** needs nearly half a page to implement the APL concept 'take' on a vector. Here is an example from the MatLab newgroup:

I would like to speed up the following code but I am unsure how to convert to vectors. Any help would be appreciated.

```
for i = 1:500
  for j = 1:500
    if rand < 0.5
      matrix(i,j)=1;
    else
      matrix(i,j)=-1;
    end
  end
end
```

As the matrix gets larger this coding becomes very slow! Thanks in advance Harry.

Well, someone stepped in with this:

This should work for you:

```
matrix = rand(500,500);
idx = find(matrix<0.5);
% you can also use idx = matrix<0.5; but it is slower
matrix(idx) = 1;
matrix(~idx) = -1
```

Hope this helps, Jon

Which is much better, but with array thinking, you can do this:

```
mm=x*-.5+(÷m)*?500 500pm~1+2*31
```

or in the language of MatLab:

```
matrix=sign(rand(500:500)-.5)
```

But not everything in APL translates as easily. Sasha showed us the code for 'take' which was posted in reply to his query on the newsgroup. It was around 40 lines of quite dense code! Surprised that it was so hard? *For us, it is in the fingers!* So we must stay with our native language and only use these other tools as tools! Looking for signals in noise – often it is *just array handling* – no higher mathematics! A box of beer to anyone who can do this in anything but APL. (Sasha showed a nice example of data from reactor control rods, and the APL code to find where the rods actually moved).

How to co-operate

You can talk to R with just 4 functions (anyone who has used the J DLL will recognise the style), and because R uses scripts for everything, we can define functions to be executed in the R environment. Of course, it is not convenient to repeatedly call `rexec`, so we prepare a set of commands as a vector of vectors, and pass all the lines at once.

```
[0]  rinit
[1]  '#.R'⎵WC'OleClient' 'StatConnectorSrv.StatConnector'
[2]  '#.R.Init('R')
```

```

[0]  a←(r)revec exp
[1]  ±(0=□NC'r')/'r+0'
[2]  a+0
[3]  :If r
[4]  a+#.R.Evaluate(exp)
[5]  :Else
[6]  #.R.EvaluateNoReturn(exp)
[7]  :EndIf

```

```

[0]  {m)rput a;x1
[1]  ±(0=□NC'm')/'m+'x''
[2]  m+,cm
[3]  a+,ca
[4]  #.R.SetSymbol(m,a)

```

```

[0]  r←rget name;rho;Re;Im
[1]  r←#.R.GetSymbol(name)

```

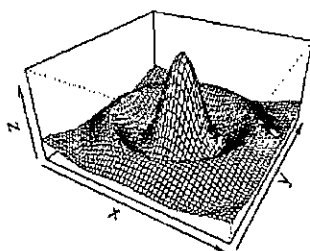
Mexican hat time!

Here is how it looks (note that right arrow, by the way – it creates a new variable as a NL-delimited text vector) ...

```

)ed →script
  pscript
168
      script
x<-seq(-10,10,length=50)
y<-x
foo<-function(x,y){r<-sqrt
  (x^2+y^2);10*sin(r)/r}
z<-outer(x,y,foo)
pers p(x,y,z,theta=30,phi=30,
  expand=0.5)

```



... and to run it

```
revec"{{(∼ω□tc)←ω}script
```

... which makes a classic 'Mexican hat' for us:

Regression

The linear modelling is just like the syntax in ASLGreg, which is hardly surprising, given the common source of both packages. Here is a simple regression model, set up in APL and run for us by the R engine:

```

      +x+9?25
4 19 12 14 6 2 17 24 10
      +/x÷px
12
      'x'rput x
      rexec 'm<-mean(x)'
      rget 'm'
12
      1 rexec 'mean(x)'
12

      rexec 'y<-10+3*x-0.1*x^2'
      rexec 'x1<-x'
      rexec 'x2<-x^2'
      rexec 'mod<-lm(y~x1+x2)'
      rexec 'c<-mod$coeff'

      rget 'c'
10 3 -0.1
      y⊖x°. *0 1 2
10 3 -0.1

```

Comforting that APL gets the same answer! There is more ...

```

      rexec 'yn<-y+rnorm(9)'
      rexec 'mod1<-lm(yn~x1+x2)'
      rexec 'c1<-mod1$coeff'
      rget 'c1'
9.555004062 3.193844447 -0.1062125247
      yn+rget 'yn'
      +a+yn⊖x°. *0 1 2
9.555004062 3.193844447 -0.1062125247
      e+yn-a+.x⊖x°. *0 1 2
      ((+/e*2)÷pe)*.5
0.4995013834

      rexec 's<-sd(yn-mod1$fitted)'
      rget 's'
0.5298012232
      ((+/e*2)÷~1+pe)*.5
0.5298012232

```

Of course, there is much, much more. "In APL I may do terrific things, but 'get random number from Student Distribution' is HARD. Now we just have it, and of course the R stuff is well tested and analysed by 100s or even 1000s of statisticians."

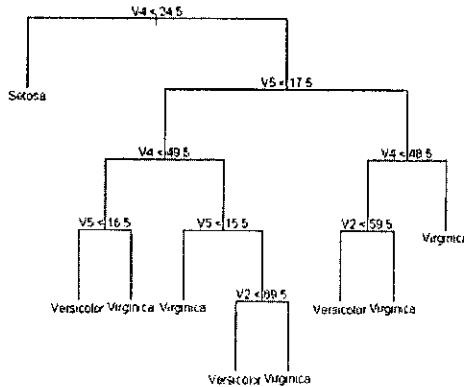
Classification trees

"Why am I not happy with my tested APL code (as shown at APL2001)? In APL it was not easy to write this code. In R it is only this much ...

```

rexec 'library(tree)'
'row'rputtpdata
'col'rput 2>pdata
'data'rput data
rexec'data<-matrix(0,ncol=col,nrow=row)'
rexec'data<-as.data.frame(data)'
formula+ 'V1~.', minsize=2,mindev=0'
rexec't<-tree(data=data,formula=',formula,')'
rexec 'plot(t)'
rexec 'text(t)'

```



... which is some 200 times faster for 4 factors (very few). For 100 factors R is 2000 times faster for the same result. We *have* to analyse by factor – there is no nice array solution here!"

Cooperative Productivity

"APL programmer feels comfortable in MatLab and or SPLUS/R after a 30 minutes' introduction. A couple of weeks are needed for diving in depth.

If we estimate productivity of problem solving in APL, R/SPLUS and in MatLab as 10 arbitrary units for each environment then productivity in cooperative environment may be estimated as multiplication or 100 units for a pair or 1000 for all three."

Lunch and After

We ate well (as ever) and enjoyed the tour around the manor house. Arto Juvonen showed us a nice application in APL+Win with Finnish-Russian dictionaries, and Jouko showed some of his work with RainPro and PowerPoint in co-operation. I talked about the Dyalog .Net compiler, and finally Reima arrived with some interesting work he had done with NewLeaf and PDF. This led directly to the implementation of 'bookmarks as page references' in NewLeaf on the way home.

Then it was the inevitable sauna, dinner and evening seminar with Sasha. The reporting team escaped at around 1:00am, by which time it was very cold indeed for the walk back across the park to our room.

Day 2 (Morning)

Olli Paavola gave an excellent tutorial on the +Win Treeview object, based around the example of a product catalogue. Veli-Matti Jantunen described a multi-dimensional editing tool called PXEdit (now this is a paper that we really could use as a translation – I can't do it justice here). After the lunch break, it was back to Sasha!

Sasha – Why be Interested in APL for Linux?

The prediction is that major developers will use Linux as the primary platform. The German government switched to Linux last year. Whole countries like China and India are Linux. So what are the options for the APL developer?

- SAX and APLX are free for a personal use. SAX has close integration with Java. But – old-style editor. APLX looks pretty nice and workspaces may be used on Mac OS X and Windows and vice versa.
- Dyalog APL for Linux is strictly commercial product. It uses third party library to emulate Windows stuff and is very expensive.
- IBM APL2 – is free for educational purposes, but not for personal use. Very good compatibility – all the APs exist and there is a TCL/TK interface. In Yale, Dave Liebttag was sitting next to me and heard for the first time about this. They already had a REXX AP – Dave said "goodbye Sasha – I am going to bookstore" in 1 hour "Would you like to see the first version of TCL?!" Bad thing about it is the luck of good built-in editor.

BUT ... nothing is free for development except ...

- A+ from Morgan Stanley. You may use it anywhere.

When MS announced it in January 2000, I expected that everybody would switch – now I am really surprised that there seem to be no users of this wonderful product! Let's start with a demonstration!

The screenshot shows the Emacs editor window titled "emacs: *a*". The menu bar includes File, Edit, View, Cnds, Tools, Options, Buffers, A+, and A+Options. The toolbar contains icons for Open, Find, Save, Print, Cut, Copy, Paste, Undo, Redo, Replace, Mail, Info, Compile, Debug, and News. The main text area contains the following text:

```
*a*
x[2]+99

=== now smoothing time series

R HERE WE HAVE TABBED WINDOWS TO EDIT A+ FUNCTIONS AND WRITE SCRIPTS

== Running medians of 3

med3 X:{
Y+a~3IX;
Y+((+Y)-(I/Y))+L/Y;
X[0],Y,-1IX)

explain restructure
3I\6
~3I\6
play with smooth
x*med3 x
A-**-XEmacs: finn.d (Fundamental A)---22%-----
A+
Copyright (c) 1990-2001 Morgan Stanley Dean Witter & Co. All rights reserved.
ed.
This version is Release 4.18
Sun May 18 07:40:34 MSD 2003 on vsuse73
\5
0 1 2 3 4
+/\5
10
+/?10 5p99
492 436 513 477 464
R HERE IS SESSION MANAGER. YOU MAY SCROLL, SEARCH AND EDIT IT

A-**-XEmacs: *a* (A A: run)---Bot-----
```

This is NOT A+ – it is the legend of the Unix world – Emacs. It is not an editor, it is a *religion*. It has LISP as a built-in language. With 10 lines of LISP we make it available to A+ also.

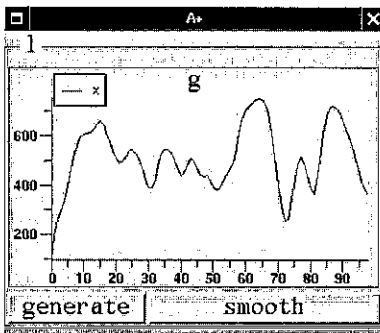
```
+x+10?100
```

```
Load screen management and graph environment
$load s
s.QUIET+2
show `x
```

... and here is a window with x in a grid! Change x[4] to 3 and automatically the grid refreshes. It works both ways [changes the grid and checks x[4] has updated].

```
now see x as graph
show `x is `graph
drag and drop
x[2]
20
x[2]+99

see and explain symbols
t+`one`two`three`four
pt
4
t: `two
1
```



It works both ways on the graph too. If Y has a **dependence** on X (say a moving average) then dragging X automatically redraws the graph of Y.

Here is Hanning, another smoothing algorithm:

```
hann X:(
Y+q-3!X;
Y+.25 .5 .25+.xY;
X[0],Y,-1+X)
```

A few more lines make this simple Gui, where we can experiment with the results.

What else is good about A+? Distribution is simply a saved A+ script and APlus itself. Client-server is simple with the TCP/IP support. There is a very good implementation of memory-mapped files, and a built in database facility which holds columns of tables in separate files, making a 'directory' the equivalent of a 'table'. You can call code in other languages – best of all in C. With Emacs as your editor and session, you have a marvellous experimenting tool at your command.

Summary

Yes, we ran out of time, and I ran out of energy to take notes! I have Sasha's scripts, but I don't have a handy Linux box to play in, and I don't know Emacs. I think he may be right to wonder why the APL world continues on its way, largely ignoring one of the fastest engines available. Just because it is free doesn't mean there is anything wrong with it. Do we need this 'full' build under Windows, or do we just go with Sasha's view that sooner or later Linux will sweep the development world anyway. There is a lot of power out there, if only you have the energy and enthusiasm to find it. Enthusiasm and energy are two qualities our Russian Bear is not short of.

Time to head home – the inevitable group-photo gives you a good idea of the surroundings and some of the last of us to leave:



Arto, Kimmo, Sasha, Adrian, Gill and Jouko

Maybe it will come around again next year? If so, it would be good to be back, and to see how far APL has moved forward in Finland in another 12 months.

Experiences with the Dyalog .NET Compiler

a talk to Forests in Finland VII by Adrian Smith

Author's Note

The proposed talk was a fairly code-intensive walk-through of the potential for using Dyalog's *APLScript* as a primary source for your .net applications, regardless of the current flavour of APL you were using. An initial show of hands showed that almost no-one in the audience really knew anything at all about .net (and APL's relationship with it) so the talk rapidly grew a 20-minute introduction to the whole .net architecture, and the way Dyalog APL fits into it. This is (very approximately) reproduced as the first part of this report, and is followed by the stuff I was expecting to talk about.

Background to the .NET Architecture

I suppose the first thing to say is that Microsoft got the name badly wrong – this has very little to do with the 'net' and everything to do with the Windows API. In fact they have sensibly renamed the next version of Windows NT as 'Windows 2003 Server', probably as the first stage of a rethink on the name.

I think it will quite quickly become the programmer's tool of choice, not because of Microsoft's marketing, but simply because it is a much better way to develop and deploy applications. Java established the principle of a *Virtual Machine* (the JVM) which permits the same compiled code to run on multiple platforms. Well actually, APL did it in 1979 with APL for the IBM System-7, and Paul Chapman did it again with DEGO for I-APL, but that's another story. The snag is that the Java machine was specifically designed to run compiled *Java*, just as the APL IL runs interpreted *APL*. What Microsoft have done is define a virtual machine called the *Common Language Runtime* (CLR from now on) which is designed to be dead easy to write a compiler (or interpreter) for. In fact it is shipped with a demonstration Lisp interpreter, just to show you how to build one.

They then named 7 target languages, carefully chosen to span the known space of current language thinking. This was called *Project-7*, and (as I hope we all know) Dyalog APL was chosen as the APL guest at the party. The work Dyalog have done (which is the reason John Daintree got the BAA Outstanding Achievement Award last year) puts APL on exactly the same footing as all the other .net languages.

This is quite a new situation for us – just contrast it with the current position in some of the APL engines we have today:

- in APL2 I can call out with `⌈NA` (with a lot of technical expertise) and I can be run in batch mode via JCL. As of last week, I can call COBOL subroutines.
- in APL+Win I can call out with `⌈WCALL` (assuming they set the ADF file up correctly) and I can run the interpreter as a COM server. However there is no way a C++ programmer can be shown the details of the functions he/she has access to, or offered any 'properties'.
- in Dyalog APL/W I can call out with `⌈NA` (with a lot of technical expertise) and (with a considerable amount of work) I can expose my application as a proper COM object with genuine Methods and properties.

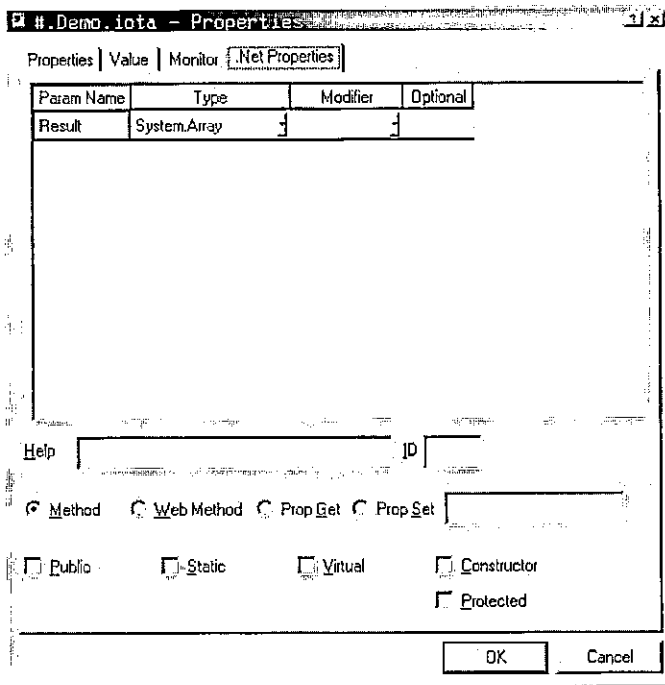
With .net, all this ugly technical stuff goes straight out of the window. If I want to use a C# subroutine from Dyalog APL, I just (well duh) use it. If I want to use an APL function from C#, I just use it. Deployment is as simple as copying the files I need into my installation folder – gone is all that stuff with type libraries and OCX registration and all the things that could so easily fall apart when the user re-installed Windows and trashed the Registry.

Why is it all so easy? Basically because everything is compiled down to CLR, so to the .net engine, it all looks the same. If you open up a DLL that you made from APLScript in IL DASM (the Microsoft Disassembler, free with the .net framework) it is indistinguishable from a DLL that you compiled from C# using the (also free) command-line compiler. We are all playing with the same ball on the same playing field, for the first time that I can remember.

OK, so how do I Kick the Ball?

What I want to do here is just to give a flavour of what is possible, so we will make a simple APLScript, compile it and call it from an equally simple C# application. As a slightly tougher exercise, I will show you how to wrap a typical 'legacy' application (the ASL Regression Library ASLGREG) as a .net component that can easily be called as a subroutine. The only tools you will need are your current flavour of APL, Notepad and the .net framework.

Why use *APLScript* rather than the Gui approach that Dyalog showed in their Madrid workshops? I will try to pick up the advantages as I go along, but basically it is because I have a hate-hate relationship with dialogue boxes like this:



... lifted from John Daintree's article in Vector 19.1. This is fine for demos, but in *RainPro* I have about 60 Methods and 170 properties to work through. I would die of RSI before I got halfway down the list, and I would be sure to make some mistakes along the way. How would you set about globally changing "Int32" to "Float" across 60 functions when each one required about four mouse-clicks to open up the right dropdown? This was definitely not for me, so I cut the workshops and chatted to Dave Liebttag and Eke about control structures instead. Much more fun.

Then I read John's next article in Vector 19.2, and the light dawned. With APLScript, a typical function looks like this:

```

▽ tv←Rotate tv
A This should be pretty simple!
:Access Public
:ParameterList String
:Returns String
tv←ϕtv
▽

```

What do you immediately notice about this function? Yes, that's right – everything it needs to know about being a .net function is defined in *one place*, as easily accessible text. That means I can search for it in an editor, globally replace my systematic errors, and write some tools to help me out. Remember "there's *always* time to sharpen your axe", and if you start with it sharp, so much the better.

The next reason I have for liking this is that I hate changing interpreters. My two main environments are Dyalog 8.2 and APL+Win 3.5, which do everything I need. I have the session how I want it, I like the way the editor behaves, I have Alt+EF hardwired into my brain for 'Edit, ReFormat', and so on. Clearly, I am unlikely to write much APL in Notepad, but I already have lots of APL in my favourite interpreters. Maybe I could just take the existing code, add a few simple comments, and create the script automatically? For example:

```

▽ tv←Rotate tv
[1] A This should be pretty simple!
[2] A:Public
[3] A:Params String
[4] A>Returns String
[5] tv←ϕtv
▽

```

Spot the difference! Well, firstly the lines in the script that define the .net interface must be generated from comments in the original source function. It would be very sensible for Dyalog to permit `:ParameterList` as a valid control structure in normal code, but for the moment we must hide it from the interpreter. But hang on a moment – how many sharp-eyed APLers are there who noticed that it just says `A:Params` here? For that matter it just says `A:Public` instead of `A:Access Public` on line[2]. Yes, you guessed it – Adrian is a lazy typist and would rather write the bare minimum in the function comments. It all gets expanded nicely when the script is written out.

Here is the fragment of C# which tests it:

```
Sample sam = new Sample();
Console.WriteLine("*** String *** " + sam.Rotate("Hello,
world!"));

int[] myints = new int[] {1,2,3};
myints = sam.Rotate(myints);
```

Which brings us very nicely to the next big bonus point for scripts ...

Overloading

Suppose I wanted to be able to rotate numeric vectors in addition to strings? The way you do this in C# (and hence in the CLR) is to define a sibling function which offers the alternative argument type and is associated with the base function using the *Implements* keyword. APLScript follows along the same lines:

```
▽ tv←Rotate_ovl1 tv
:Access Public
:Returns Int32[]
:ParameterList Int32[]
:Implements Method Rotate
  tv←Rotate tv
▽
```

OK, a bad example, as this kind of structural function would probably just take and return *array*, which is the most general type there is. My problem is that functions like *ch.Bar* should be specific about their argument types, and they get rapidly out of hand – in the extreme case a barchart could take an integer scalar; typically it takes an array of either integers or floats, but it can happily accept matrices or vectors of numeric vectors (any flavour). You can clock up a dozen sibling functions here with no real effort, and you do not want these cluttering your workspace, let alone having to be defined with the afore-mentioned dialogue-box one by one! I hope you can see where this is leading:

```
▽ tv←Rotate tv
[1]  A This should be pretty simple!
[2]  A Note the paired argument and result types
[3]  A:Public
[4]  A:Params String
[5]  A:Returns String
[6]  A:Params Int[]
[7]  A:Returns Int[]
[8]  tv←tv
▽
```

Let's just add another pair of comment lines, and have APL generate the siblings when it writes out the script. All the rats stay firmly in one trap, and it is easy to add more definitions as we need them. I am beginning to like this!

As you can see in the following expansion, if there are multiple sets of parameters and results, they will be paired up. Multiple parameters may also be used with a single result, but the C# compiler chooses the correct overload by matching up the parameter lists, so you cannot have a single parameter set, and multiple result types. At the moment, I can't think of an example where I would ever want to do this, so I don't think it is a major oversight.

```

▽ tv←Rotate tv
R This should be pretty simple!
R Note the paired argument and result types
:Access Public
:ParameterList String
:Returns String
:Trap 0
    tv←ϕtv
:Else
    (Exception.New ϵ[AV[2+IO], "2+DM") SIGNAL 90
:EndTrap
▽

▽ tv←Rotate_ovl1 tv
:Access Public
:ParameterList Int32[]
:Returns Int32[]
:Implements Method Rotate
    tv←Rotate tv
▽

```

You may also notice that I have wrapped a standard :Trap block around the body of the main function. At the moment, this is rather redundant, as it simply reproduces the normal Dyalog APL behaviour, however I felt that I might want more control here in future, so let's put the code in now in case we need it later.

Setting and Getting Properties

Again, it is all very simple. To be able to write C# code like this ...

```

grap1.Heading = "Share Price with Moving Averages";
grap1.Subhead = "Buy when Red crosses Cyan!";

```

... I need to define some properties in my script, in this case called *Heading* and *Subhead*, both of type String. In C# (and hence in APLScript) you implicitly define a property by including *Get* and *Set* functions for it. Readonly properties only have the *Get* half of the pair. Easy stuff, but RainPro has (as of today) 171 properties, nearly all read-write. Ug – that means over 300 one-line functions if I do it using the workspace approach. In this case, they all look remarkably similar:

```
:Property String:CaptionFont
  ▽ r←get
    r←##.ch.Query'CF'
  ▽
  ▽ set value
    r←##.ch.Fix 'CF' value
  ▽
:EndProperty
```

RainPro puts everything through the same pipe so it can keep tabs on what you are doing to it – so I can automatically generate the whole script block from `ch.Δchdefaults` without ever seeing any of these ‘getters’ and ‘setters’ in my workspace. In the more general case, you would probably make a simple character array to define the property names, types, and the get/set functions to be called. When I need this (for example for NewLeaf which does it this way) I will add in the code to parse it and output the appropriate calls in the script.

Wrapping a Legacy Workspace

I thought a good example would be the Regression Shelf from the BAA Statistics Library project. This is quite old code, written mostly in APL*PLUS/PC, so we really don’t want to tamper with it if at all possible. The idea would be to create some simple wrapper functions and then bury the entire legacy edifice in a namespace where the C# programmer need never see it. Here is what I needed to build to make this idea possible:

```
↑Δas1
;APLScript definition for ASLGreg.NET

:Namespace as1

A ===== Classes =====
:Class greg
  □IO←1 □ML←1
  InitASL
  !regfns.Δpublic
:EndClass
```

```

A ===== Internal NS =====
*regfns

:EndNamespace

```

This is an easy-to-edit text variable which is like the C-programmer's 'make' file. It has a bit of basic control information, some comments and a few APL lines to be run by the Dyalog compiler at compile time. However ("heads up" as they say in cricket when an unexpected ball is coming your way) there is a small divergence between Dyalog's use of the term *Namespace*, and the CLR meaning of the same term. The compiler hides the issue quite well, but it really can confuse you if you don't follow the rules.

- in .net parlance, a Namespace is very like a DOS path setting. It is a convenient way of omitting the fully qualified part of a name from a function call.
- in Dyalog APL, a Namespace is much more like a C# Class, being an encapsulated collection of functions and data.

What the APLScript compiler does is to take the outer :Namespace blocks in the script and make them into CLR-style namespaces mirrored by APL namespaces – that is what :Namespace ASL does for you in the example. Any deeper-level :Class xxx blocks also become standard APL namespaces, but you cannot put a CLR namespace *inside* a class which is where the analogy breaks down. Now you can refer back 2 pages and see why I needed ##.ch.blaaah in my Get/Set functions for RainPro. These are in a class called GraPL, so Dyalog has implicitly created an APL-style GraPL namespace for them. To see the stuff in the 'real' namespace ch I need to back up one level and look down again. Note that this uses ##.xx rather than #.xx as I have absolutely no idea where I am in the namespace tree that they have built for me, but I am quite sure I am not a child of root. *Be careful* – you have been warned!

So here we are in our ASL namespace and we want to make a collection of functions which the C# programmer will see as a Class with Methods. These lines do the trick:

```

:Class greg
  IO←1 ♦ ML←1
  InitASL
  !regfns.Δpublic
:EndClass

```

Anything in the 'make' variable that starts with a colon or comment, or contains an assignment, just passes straight through into the script. Anything that names a function gets the function thrown into the script, but carefully tidied up as

described above. So in this case, I have an initialisation function which I want run at startup (when an instance of the class is created by the C# program) which looks like:

```

    ▽ InitASL
[1]   A:Access Constructor
[2]   ##.regfns.START
    ▽

```

Because I buried all the real code in the APL namespace `regfns`, I need to walk back one level here. It is quite convenient that Dyalog tolerates this syntax in the ordinary development environment – here `InitASL` is actually in the root namespace but I can still run it quite successfully to test the code before I build it into a script as `##` has nowhere to go but itself!

The next line in the make variable begins with `!` which has no logical meaning whatsoever, except that I am working slowly along the top row of the keyboard as I add in more possibilities here. What `!` does is to take a variable which defines the public functions, and builds the script entries for me. In this case it looks like:

```

Public name:Function:Description:Parameters:Overloads>Returns
Diagnostics:Diagnostics:Useful tables:String cols:vvtv
Display:Display:Results of analysis:String cols:vvtv
Fit:Fit:Model to be fitted:String modeldefn:vvtv
Set:Set:Set named data to value:String name, int[] value;
                                String name, num[] value:txt
Yvar:Yvar:Set Dependent variable:String YVarname:txt
Version:VERSION:What it says::vvtv

```

Taking the last line as the simplest example, this will create a script entry:

```

    ▽ res+Version
A What it says
:Access Public
:Returns String[]
:Trap 0
    res+##.regfns.VERSION
:Else
    (Exception.New €[AV[2+IO], "2+DM] [SIGNAL 90
:EndTrap
    ▽

```

Where overloads are required, I can list multiple parameter sets separated by `;` to have the matching set of generated functions written into the script. The return-type of `'vvtv'` is Causeway-speak for 'vector of text vectors' and translates to 'array of strings' in the generated script; return-type `'txt'` is just a short form of `'String'`.

```

▽ res+Set arg
R Set named data to value
:Access Public
:ParameterList String name,Int32[] value
:Returns String
:Trap 0
  res+##.regfns.Set arg
:Else
  (Exception.New €□AV[2+□IO], "2+□DM) □SIGNAL 90
:EndTrap
▽

▽ res+Set_ovl1 arg
:Access Public
:ParameterList String name,Double[] value
:Returns String
:Implements Method Set
  res+Set arg
▽

```

It is really a matter of taste whether you put this variable in root and have the second column qualify the names, or whether you put it in the namespace where the legacy code lives. Either way, you can conveniently use column-2 in the editor to drill-down to the real, underlying code from the public names.

Finally, we must include the legacy code itself!

```

A ===== Internal NS =====
*regfns

```

As you probably guessed, I used * as a marker to say 'here comes an APL namespace' so this just loops through and emits everything needed to rebuild the namespace (including any subnamespaces unto the nth generation). It is NOT within the :Class block so the C# programmer never sees any of it.

A small enhancement request!

One annoying snag with this is that you can only put variables into the script by carefully creating the expression that (when executed) regenerates the variable. For a deeply nested array, this takes significant time, and for nested arrays with empty bits it is hard to reproduce exactly the array you started from. It would be really nice if (say) qq+1 □OR 'Abstract ' returned the internal representation as a simple byte vector and 2 □OR qq reversed the situation. Then we could create a script from a namespace much more economically. It could be quite handy for

interprocess communication via SOAP, which might encourage them to attempt something similar! I think Morten Kromberg has an interpreter-neutral format?

Anyway, better test it!

We run the 'make' variable to write out the file in APLScript format:

```
'd:\tools\aplc\asl' UPut 'DLL' Make Δasl
34738 bytes written to d:\tools\aplc\asl.apl
```

... and we compile it to a DLL with the Dyalog compiler:

```
D:\Tools\aplc>aplc asl.apl /t:library /runtime /v
Dyalog APLScript compiler. Version 10.0
Copyright Dyadic Systems Ltd 2002
Got Startup Parameters
Reading asl.apl
Processing asl.apl
Processed asl.apl
Emitted File asl.dll
```

... and we hack together some C# code to test it:

```
using System;
namespace asl
{
    class asltest
    {
        static void Main(string[] args) {
            greg test = new greg();
            // greg.VERSION Returns String[]
            foreach (String ss in test.Version()) Console.WriteLine(ss);
            // greg.Set covers data import returning String "xxx defined"
            // Accepts int[] and double[] here
            int[] age = {1, 2, 3, 5, 7, 8, 9};
            double[] wt = {1.4, 2.2, 3, 5.7, 7.1, 8.9, 10.2};
            Console.WriteLine(test.Set("Age",age));
            Console.WriteLine(test.Set("Wt",wt));
            // greg.Yvar names the variable to model
            Console.WriteLine(test.Yvar("Wt"));
            // greg.Fit fits a model and returns some info about it
            foreach (String ss in test.Fit("GM+Age")) Console.WriteLine(ss);
            // greg.Display tells us more about what went on
            foreach (String ss in test.Display("EST")) Console.WriteLine(ss);
            // greg.Diagnostics lists the data and fitted values
            foreach (String ss in test.Diagnostics("FRCL")) Console.WriteLine(ss);
        }
    }
}
```

We compile it (the C# compiler is free, just like all the other tools you need here):

```
D:\Tools\aplc>csc /r:asl.dll tryasl.cs
Microsoft (R) Visual C# .NET Compiler version 7.00.9466
for Microsoft (R) .NET Framework version 1.0.3705
Copyright (C) Microsoft Corporation 2001. All rights reserved.
```

... and we can run it.

```
D:\Tools\aplc>tryasl
Regression Volume 1 Version 1.1
A.M.Sykes, December 1991
Copyright (C) British APL Association 1991
Age defined
Wt defined
```

```
Y variable is Wt
Units set to 7
```

Source	ss	df	ms
-----	--	--	--
Due to model	69.52	1	69.52
Residual	0.6784	5	0.1357
-----	-----	-----	-----
Total Corrected	70.2	6	11.7
Percentage Variation Accounted for = 99.03			
F-statistic =			512.36
p-value =			0.00

Summary

"In the land of the blind, the one-eyed man is king". I can't claim to be an expert on this stuff, but I think I have fallen into most of the Heffalump traps and have understood just enough to make it work. We have GraPL.net customers yelling for it, so I have every reason to make it work as soon as I possibly can.

Will it take over the world? Well it sure beats programming with COM, registering OCXs and all that garbage. I think it will hit the server world quite quickly, which is why I want to be out there with it. As for the desktop – Microsoft missed the boat by not including the runtime support with XP, so give it 5 years and have another look around to see if enough people have upgraded their copy of Office to make it safe to ship. That gives us lots of learning time – don't waste it, because if this does go mainstream it is a real opportunity for APL to rejoin the party as 'just another language'. Anyway, Dyalog deserve our support, don't you think?

GENERAL ARTICLES

A Place for Us

by Stephen Taylor

Email: stj@lambenttechnology.com

(Expanded from remarks extemporised at the open discussion that concluded the APL 2002 conference in Madrid.)

Years ago, a friend asked me what was the longest period I could recall between two cups of coffee. I was startled to be unable to think of one longer than fifteen hours. A year ago I started work in APL after being away from it for fifteen years. This is unusual and I learned some things I'm glad to know. The first is that I can give it up any time I want.

The second is that the toolset is way better than it was fifteen years ago. If you've been working continuously in the language, getting these improvements bit by bit, you could easily overlook how far APL has come as a software development environment. Twenty years ago it was a rogue: APL had almost no communication with other computer systems, and its programming environment was weak. Now it has a full development environment.

The third is very odd. Organisations that own APL systems almost all have 'exit strategies' for APL; or at least an intention to replace them with other technologies. There is something peculiar, and peculiar to APL, that our clients are willing to pay to get rid of our systems. Even more oddly, in the APL world we take this to be normal. We believe our systems are fast and cheap to develop and run. And we accept it as normal that our clients will take trouble to replace them. What is wrong with this picture?

When I was last in the APL world, people expressed hope that APL's virtues would one day enjoy wide recognition. I no longer hear that. In contrast, speakers here have commented on how our numbers shrink year by year while our average age rises.

A shrinking user group and clients who intend to replace our systems: this makes the predictable future of APL very clear. It will disappear. This will not necessarily happen. But right now it is our predictable future. If we continue like this, APL will disappear from use.

I see no signs of J being taken up for commercial use. A+ usage is fading as Morgan Stanley slowly replaces with conventional, more cheaply maintained software the applications it developed rapidly in the 80s and 90s. K appears to be thriving modestly in close proximity to kdb.

I am interested in a different future, in which our technology flourishes. I have a self-interest: my productivity in APL commands a premium for which I cannot charge in other languages. I want to work in APL, or J, or K, or something like them. I am going to assume that all of us here are at least in favour of a future in which these languages flourish, and that the same is true of the larger group of people at past conferences.

The problem with us

A general principle holds that when people espouse something for years and don't get a result, something is not being said. There is something that the group as a whole is unwilling to look at, express, consider or otherwise deal with.

Some of us inquired into this over dinner during the conference. What is so terrible about our technology that our clients want to get rid of it? Our first thoughts sounded like this. Doesn't fit in with the IT culture. APL people's productivity make regular IT folk look bad. Impossible to manage or control.

Which was interesting, because these are not issues about the technology, but about us. Is it possible that we are the problem? That it isn't our software our clients are spending to get rid of, it's us?

Perhaps it's us, not our systems. There are real issues around managing APL software, but I suspect that rather than help IT departments solve them we have for long succumbed to the temptations of being maverick software gurus; and APL's extraordinary productivity enabled us to get away with that for longer than anyone else could. Something like this is the thesis of an article Bob Brown and I are preparing. If that's true, no wonder the IT boys want us out. We APLers have created a brush that I suspect J and K folk are taking care not to get tarred with.

There's a place for us?

What would the future we want look like? I need a new category to even pose the question. It is not the future of APL that precisely concerns me, but of something else, common to at least APL, A+, J and K. When programmers in these languages look at a program in C or Visual Basic, we all react with "it doesn't have to be that laborious". We have a common experience of dense, rigorously general, formal notation that gives us abstractive and expressive power. I don't know a good name for this, so I'll call it Dense General Notation or DGN. What divides DGNs from other languages matters more than differences between individual notations.

What would a future in which the DGNs flourish look like? Here are some suggestions:

1. A growing body of commercial developers working in DGNs
2. World-class research being done in and on DGNs
3. A vigorous annual joint conference held by a DGN SIG
4. A web-based study and coaching programme

Here's something that's inconsistent with that. You used to be able to smell the future at an APL conference. You could hear about work that might change the future use of computers. Some of that was being done *on* the language, some of it *in* the language. An APL conference was an exciting place to be even if you weren't working in APL. You could sniff the future arriving, and some of it looked like arriving coded in APL. That no longer seems true. The people working on language development are now almost all working elsewhere. And if others are using APL to break radical new ground in their own fields, they're not coming to our conferences to tell us about it.

Part of the last point must relate to the nearly complete disappearance of APL from education. We used to get a steady trickle of graduates entering the industry because they'd been exposed to APL and wanted more. That seems to have stopped in the UK. The slight use of APL or J in education in the UK is inconsistent with the claims we make for their benefits in exploring mathematics and theories in the natural sciences.

The separation between APL and J users cannot have helped this. As far as I can tell, the educators and the language developers now work with J, leaving a rump of software developers improving their tools (the current APL software development environments really are good) and managing the code base of commercial APL systems. This would explain something about our conferences.

Stories of successful systems development can be instructive, but it's hard to generate much excitement from them.

To address (1) I've started on a short series of articles arguing the case in systems development for maverick groups using DGNs to solve pressing problems. Morgan Stanley's use of A+ to create complex risk-management software would be exemplary. I intend these articles for the IT sections of leading papers in London and New York.

More exposure and commercial credibility for DGNs as tools for smart software developers should stimulate interest within educational institutions and thus (2). The BAA is partly addressing (2) through its Schools Project, which is likely also to produce (4).

Conclusion

Bob Brown wonders whether we really are a community, and not just a minority. If we are, we never defined and promoted to the IT majority our place in the mainstream architectures. Delivering impressive systems and delighting end users has not saved and is not saving us from exclusion by the majority.

Life is never static. APL as an industry is contracting and will disappear unless it expands. Expansion requires a rapprochement with mainstream IT that defines the role we play in their architectures and strategies, and distinguishes when and why to use DGNs rather than scalar technologies.

It's long past time we showed up as the partners of scalar technologists, not the people who make them look foolish.

The Simple Art of Software

by Stephen Taylor
Email: sjt@lambenttechnology.com

The Extreme Programming movement is reviving and extending our tradition of fast and lively software development. APL programmers should hop aboard this bandwagon.

In "The Simple Art of Murder"¹, Raymond Chandler wrote of Dashiell Hammett, master of the 'hard-boiled' detective story, that he "took the art of murder out of the country vicarage and gave it back to the people on the street, to whom it really belonged anyway."

Kent Beck and his colleagues in the Extreme Programming (XP) movement have performed a similar service for software development. XP aims to take software development out of the hands of methodologists and give it back to the programmers.

Critics accuse XP of being a licence for programmers to hack away without supervision. Such accusations overlook XP's exacting disciplines. But the bottom line is that XP practitioners report startling improvements in satisfying customers by quickly getting the software they want into production. It also seems to be a barrel of fun: few programmers who've tried it want to work any other way afterwards.

XP aims to focus ruthlessly on producing working code and to avoid distractions. It has radical views on what supports and what distracts from software development. Change is constant, so XP says develop in iterations and plan just for the next release. It's cheaper and faster to refactor than to do Big Design Up Front, so write the simplest code that could possibly work. Don't build infrastructure until the iteration you need it. The future is hard to predict; the odds are You Ain't Gonna Need It. Stay agile.

Heady stuff. XP can sound like a hacker's paradise. But it also introduces radical disciplines. XP asks *What if we took the best programming practices we know and turned all the dials up to ten?* Code reviews are the best way we know to improve code quality; so we'll program in pairs for continuous review. Testing is good even though we hate it; so we'll do lots. In fact, we'll write the tests before the

¹ *The Atlantic Magazine*, December 1944.

programs. Integration is important; we'll integrate new code every few hours. Did we ever work like this?

Perhaps we did. Some XP practices are eerily reminiscent of APL projects I've worked on. For example, the impatience that ditches formal requirements analysis and specification in favour of close communication with the customer in a conversation that allows the system to evolve. XP practitioners report the sense of exhilaration and momentum that I remember from APL projects in the 1970s and 1980s.

In the 1980s software engineers persuaded me that our loose, fast way of producing systems, for all its early successes, was bankrupt, a dead end. But my initial enthusiasm for the promises of structured analysis and design gave way to frustration and despondency as I saw projects trudge into delay, disappointment or cancellation. If this was the right way to make software, I was going to find another way to make a living.

XP implies the way I used to write software in APL projects in the 70s and 80s is valuable and worth recovering, and that new practices identified by XP can make it safer and more predictable, as I had once hoped software engineering would.

This will interest every commercial APL programmer. The software engineering project was part of a professionalisation of IT in the 1980s that sought, with limited success, to bring the notoriously risky activity of software development under management control. That movement brought APL into disrepute as an exotic technology that should be eschewed in favour of 'standard' and 'strategic' technologies or 'platforms'. (Is there anything more poignant than out-of-fashion buzzwords?)

The Extreme Programming movement implies the possible rehabilitation of the way APL programmers have always written software. Instead of being seen as maverick and eccentric, we might be seen as the vanguard of software development. If you write APL programs for money, you will want to find out about XP.

In this article, we look firstly at what XP is, secondly at what APL software development might look like with XP added, and lastly at what APL has to offer the XP movement. (XP emerged from the Smalltalk community but now seems to be based among Java, C# and C++ programmers.) This last topic is the subject of a research report I'm presenting at the XP 2003 conference in Genoa in May.

It's programming, Jim, but not as we know it

What follows is the briefest summary of XP principles and practices that could possibly make the rest of the article work. I'm not going to argue here their strengths and weaknesses. That is discussed at length in McBreen (2003) and all over the XP wiki. (See Links at the end of this article.)

A better introduction to XP principles and practices can be found on the Web as the Gentle Introduction, also listed in the Links.

XP was first articulated by Kent Beck in a now legendary 1990s Smalltalk project that built a payroll system for Chrysler. Beck's 'sound bites' enjoy wide currency in the XP community; one of the better-known is "All methodologies are based on fear." Organisations that have been burned by vague and informal requirements, and coding activities that never seem to finish, learned to insist on formal statements of requirements, and detailed planning and control of code construction.

This kicked off in the 1980s with Barry Boehm's *Software Economics*. Boehm compared the work of software developers unfavourably with the work of civil engineers. Engineers routinely construct bridges and buildings to within 2-3% of their budgets and schedules; but 13% of all software projects used all of their time and money and then fail without delivering anything. Why were we so bad at what we did? Boehm proposed we adopt some of the routine disciplines of engineering.

A key premise of his work and the Software Engineering movement that it founded was that the cost of change in a project rises over time. This is so obvious in civil engineering as not to require examination. It's cheaper to change the shape of a bridge before you've built any of it than when it's half built. The lesson drawn for software development was to plan ahead. Get the requirements defined exactly, design for those and you'll deliver the right software, with a predictable construction process. One of the implications is that, when you've defined and agreed those requirements, you freeze them. In practice, freezing requirements never seemed to work. But it made sense to make it harder for the users to change the requirements the later it got in the project.

It turned out there were several things wrong with Software Engineering. The worst of them was the results. Inexplicably, all this process didn't seem to make developing software much more predictable or successful. Indefatigable methodologists looked for what was missing so they could add it to their processes. Major projects routinely produced awe-inspiring quantities of design

artefacts, including formal requirement definitions of extravagant length. (Had the business users truly read and understood all *that*?)

Software Engineering is undermined by two Great Lies. The first is that the users can tell us what they want. That's not true: they can't. Usually they can't do it because they don't know. Only when they use some of the software will they start to see what it makes possible for their business. And even what they're sure of they can't tell us because we don't have a language for communicating the requirements precisely. Writing a thousand pages of requirement narratives and diagrams doesn't help if the business users can't master it. Instead they trust that we've understood them and sign off the requirements. (Also, they know we won't build anything for them until they do.) But they hate it, because they know that anything we build that they don't like, we're going to show them described in the requirements with their signatures underneath.

And even if the business users know what they want and were able to tell us precisely and accurately, the rapid change of business conditions would make them wrong before we've finished building it. We'd have met the requirements as stated, but failed to deliver what the business needs. And isn't *that* the point?

The second Great Lie is that we know how to build the system to the requirements. That's not true; we rarely know how to do that. Engineers might know how to put up buildings and bridges to a blueprint, but we don't know that much about computer systems. Oh, we know *some* of most systems, but most software development entails novelty. We estimate as best we can and solve the problems as we come to them.

The process of a running project then works to protect the two lies. Further contributions from users are deflected into Change Management over the protests of users that they are not changing but clarifying requirements. Project managers hide delays and problems in building the software for as long as they can assure customers that construction is going "according to plan".

XP offers a heroic escape from this impasse. It begins with acknowledging the two Great Lies. XP then offers a Hopeful Insight.

The Hopeful Insight is that the analogy with engineering has been misapplied. In engineering, expensive resources (architects and engineers) are used to plan construction. Construction uses cheaper resources (building materials, labourers, craftsmen and foremen) but is the biggest project cost. So it's worth using valuable time and people to get it right. Mistakes in construction are potentially very expensive.

Software Engineers applied the analogy from civil engineering like this: system and database designers correspond to engineers and architects, and programmers churn out code instead of concrete.

XP says that's wrong. Executable code is produced by compilers and, given the speed and cost of PCs now, is cheap and immediate. Software development is all design activity these days; and the construction process is for all practical purposes free.

That makes it possible that Boehm's famous cost-of-change curve might not always rise over time. What if it didn't? Then, given the net present value of your project funds, you would rather spend money on design as late as possible. You would try to avoid Big Design Up Front and solve your problems as late as you could.

XP practitioners experimented with treating the cost-of-change curve as flat. They report that mostly, technical problems turn out to have simple solutions. Some don't, but it costs *more* to identify and solve them in advance than to solve them as you find them.

Two keys to doing that are to keep the system as simple (and thus as flexible) as possible and to have very full communication between the developers, and between the developers and the customers.

A number of virtuous circles emerge from this. For example, if we do Big Design Up Front, then a change in requirements entails changing requirements and design documents even if the change is to a part of the system that hasn't been built yet. Under XP, if you haven't built it, you haven't done the design either, so the change in requirements is free.

Values

XP urges four cardinal virtues:

- Communication
- Simplicity
- Feedback
- Courage

Practices

Here is a summary of all the practices, lifted straight from Beck (2000, p. 54). Some of them have gained fame or notoriety; others are less well-known.

- *The Planning Game*—Quickly determine the scope of the next release by combining business priorities and technical estimates. As reality overtakes the plan, update the plan.
- *Small releases*—Put a simple system into production quickly, then release new versions on a very short cycle.
- *Metaphor*—Guide all development with a simple shared story of how the whole system works.
- *Simple design*—The system should be designed as simply as possible at any given moment. Extra complexity is removed as soon as it is discovered.
- *Testing*—Programmers continually write unit tests, which must run flawlessly for development to continue. Customers write tests demonstrating that features are finished.
- *Refactoring*—Programmers restructure the system without changing its behaviour to remove duplication, improve communication, simplify, or add flexibility.
- *Pair programming*—All production code is written with two programmers at one machine.
- *Collective ownership*—Anyone can change any code anywhere in the system at any time.
- *Continuous integration*—Integrate and build the system many times a day, every time a task is completed.
- *40-hour week*—Work no more than 40 hours a week as a rule. Never work overtime a second week in a row.
- *On-site customer*—Include a real, live user on the team, available full-time to answer questions.
- *Coding standards*—Programmers write all code in accordance with rules emphasizing communication through the code.

This list will provoke a variety of responses, from “no big deal; we do most of that” to “couldn’t possibly work”. I’m not going to discuss that here. There are excellent books that do; I recommend them and list them at the end of the article.

Applying XP to APL projects

McBreen (2003) discusses to which projects you can apply XP. Four to six programmers seems to be a minimum size. I know few APL projects with teams this large. So just this one criterion by itself seems to limit XP's application to APL. Doing a 'pure XP' project in APL looks problematic right away.

XP writers warn of the pitfalls of selecting and applying just the XP practices you like. The full set of practices reinforce and support each other in multiple ways. That said, they also recommend *adopting* them piecemeal, bearing in mind that you won't see most of the value until you have most of the practices in place.

So, what XP practices could I adopt in my 2-man APL project?

Some of the practices listed above seemed immediately familiar to me. Wary of ambitious projects that go nowhere, I have long built in small increments and coded "the simplest thing that could possibly work", both for APL systems and web sites.

When introduced to APL, I loved the simplicity of the text book functions, and (in the days before GUIs) the application-specific languages we constructed in immediate-execution mode. I loved the further simplification of Iverson's early designs for direct definition, but was already struggling to preserve such simplicities in systems that communicated complex states through improbably named globals.

I fondly remember from the days before spreadsheets, sitting beside a customer and coding functions as he described what he wanted. What I wrote was then immediately usable; integration and development proceeded hand in hand. XP talk of an *on-site customer* and *continuous integration* brought back happy memories.

It's been a long time since problems of memory management or processor speed occupied much of my attention. Instead I've learned to devote much of my coding effort to clarity; that is, writing code that communicates to a human reader its intent clearly and without duplication. (See "Three Principles of Coding Clarity", *Vector* 18.4). XP's practices of *simplicity* and *refactoring*, and its Once And Once Only standard for coding, immediately sounded like home territory.

Other XP practices sounded quite alien. Hating testing just as much as the next guy, I'd tried with some success to compensate with code clarity for the absence of formal tests. Doing *test-first design* sounded at different times either radical or perverse.

I'd had success teaching APL as Ken Iverson taught me to do it, pairing students for explorations at the keyboard. And I've always found pairing myself with a fellow programmer improved code. But I'd never considered routinely pairing for *all* production coding. This is XP 'turning all the dials up to ten'.

What have I been able to try out in my current project?

Testing

Testing seemed like a good idea until I started analysis. Then it became indispensable.

The project entails automating complex and elaborate procedures for calculating the surrender value of a pension policy. New clerks typically train for six months before they are proficient in all the different procedures. The trainers scheduled a couple of weeks to train me in the most important ones.

We quickly ran into difficulties. The trainers can *show* me how to process any given case. But discussing and reasoning about the rules only produced confusion. I foresaw frustration, followed by them resignedly nodding through my error-filled notes and diagrams in the hope that I had surely 'got it'. Communication wasn't happening: what we were trying to do exceeded our linguistic abilities.

So I changed tactics. I had my partner, the trainer, take me through a single example, showing me the mainframe screens consulted. In immediate-execution mode I was able quickly to reconstruct data from the mainframe screens. My partner then explained what to do with it; and I coded the steps as a function, which we referred to as our 'script'. Writing each step of the script in APL usually took less time than speaking it in English, so there was no occasion to interrupt our conversation; coding substituted for taking notes. I used my partner's vocabulary to name objects, and only simple control structures. I also subordinated into Dfns any primitive code of distracting length, and moved the Dfn definitions into the script's initial declarations. So my partner was able to review the APL code with me.

When we had reviewed the process, I ran our 'script' and compared my results with my partner's. If we differed, it was a few minutes' work to step through the script to identify and resolve the divergence.

I had nested within Dyalog namespaces all the data used. I now nested that into another, along with a comment and an executable expression `1x` which compared the results of the script with model answers:

```

)ns c
#.c
c.comment+ 'Transfer val when suspended and status is 98'
c.lx+ 'TV = Data #.Script ValueDate'
c.(TV ValueDate Data)+34093.51 20030307 data
c □FAPPEND 1

```

The namespace c became the first test case on the file tied to 1—my test suite. A simple function sufficed to run a test.

```
rt+{0:-1 ◊ ω.(lx)}
```

This would return 1 if the test passed; 0 if it failed, and -1 if it broke. RunTest made this explicit to my partner.

```
RunTest+{>'Passed' 'Failed' 'Broke'[1 0 -1rt ω]}
```

I resisted the temptation to give my test rig any more capability unless and until I should need it. This conformed both to the XP principle of doing the simplest thing that could possibly work, and to my love of simple, textbook programming.

My trainer partner then took me through another worked example to show some different rules. I extended our script where necessary to accommodate new rules, confirmed the result and resolved differences as before until

```

RunTest c
Passed
c □FAPPEND 1

```

Now I needed to check that our first test case still passed. A simple function rs exercises the whole suite

```
rs+{{0:-1 ◊ (□FREAD ω).lx}}ω,">+◊1/□IO-~2t□FSIZE ω}
```

but needs expanding for my partner:

```

RunSuite+{
  p f b+1 0 -1◊.=rs ω
  t+(p/p), '/', (p), ' passed'
  cs+{1φ' ; case 's' /~1<+/ω}
  rpt+{v/ω:(cs ω), (p/ω), ' ', α ◊ ''}
  t, ('failed'rpt f), 'broke'rpt b
}

```

```

RunSuite 1
2/2 passed

```

We got up to 5 cases before

```
RunSuite 1
3/5 passed; case 2 failed; case 3 broke
```

led us back to resolve conflicts with earlier expressions of the rules.

Some observations.

- This conforms roughly to the XP practice of *test-first design*. If we'd conformed strictly, I would have run the test before amending the code, to demonstrate that the code needed changing. (Sometimes this will surprise you.) Once the script produced the right answer without breaking earlier cases, we'd done with the case and could move on. This gave us *certainty* and *speed*. Our task was to produce a script that would pass a set of test cases; when it did that we were done. (We are free in the project to add any more test cases we think of later.)
- I'd completely elided analysis. When we finished, I was *starting* to get the hang of the logic for myself. I'd produced no notes, no flow charts or logic diagrams and no documentation, and only partially understood the process we were automating. In terms of a traditional analysis session, it was a disaster. Instead, I had a set of formal acceptance tests and working code that passed the tests. I was *way* ahead of my schedule. I also had a user who was certain that I had correctly coded the rules I'd been shown, and that we could easily accommodate any further variation. This conforms to the XP practice of *communicating through the code*, and its focus on producing code rather than 'design artefacts'.
- At one or two points we'd sketched quick diagrams or listed some points. In accordance with XP practice, we destroyed these notes as soon as the code passed the tests. The *acceptance tests are the best documentation*. I wrote display functions for the tests and we had them cross-checked by other trainers: see Technical Notes.
- The code I had sketched so quickly benefited from *refactoring* towards the end. I removed duplications and condensed the primitive APL expressions as much as I could to leave the control structure plain to my partner. I was able to do this with *speed and confidence*: every time I recoded something, I reran the test suite.

Pair programming/On-site customer

At first I thought I was doing a new kind of *pair programming*, but with the customer. Not so. The point of pair programming is to raise code quality. What I was doing with my trainer partner was about requirements, not code quality.

However, it was a very intensive way of working with the *on-site customer*. Developing software raises questions which need answers and decisions from the business users. Having an on-site customer means fast answers when you need them, which in turn means you can jump straight into stuff and work things out as you go.

Planning Game

The principle behind the Planning Game is that the business users should make all the business decisions and the programmers all the technical ones. That sounds too obvious to bear saying, but it has radical consequences.

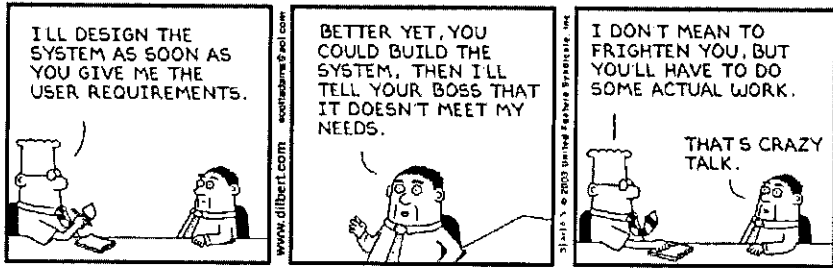
At the beginning of an iteration the business users write *user stories* on index cards. These describe in a sentence or two some desired behaviour of the system. The programmers estimate how long it would take to produce that in ideal conditions—with no distractions and so on. Life isn't ideal like that, so we use the *velocity* of the team to calculate how many ideal days work we'll get done in the iteration, given that we're programming in pairs and will also need to help and talk to each other. XP recommends guessing an initial velocity of 1 ideal day for 3 calendar days; experience will quickly show what actual velocity is.

The business users then get to pick the stories for the iteration. They can pick any combination that adds up to the number of ideal days available. Our initial estimates are only guesses, but they are the best we've got. After a couple of iterations, estimates should get reliable enough. If we underestimate and can't get it all done the business users choose what gets deferred. If we get through faster than we thought, they can pick another story or two to fit in. What we *don't* do is shorten or lengthen the iteration; because we need the feedback on how good our estimates were, else we'll have no better information when planning later iterations.

XP commits me to writing the simplest code that could possibly work, and pairs me with a programming partner to reduce my chances of straying. So I have no scope for adding neat stuff to the system. I am free to suggest it as a story to the business users in the Planning Game, but unless they include it in the iteration, I don't build it.

I've done this for one iteration now in my current project, and notice how much easier my life is. Previously I allowed myself freedom to add neat stuff to the system and fretted over what to do and in what order and would it all be ready in time. Now I can spend time only on the simplest possible solution to the user stories; my users know exactly what is coming, and hey, the next iteration is only

a week or so away, we can pick up the neat idea then, or not. XP says I've been working way too hard.



Copyright © 2003 United Feature Syndicate, Inc.

XP + APL

XP originated in a Smalltalk project and seems these days to be used mainly by Java, C# and C++ programmers. What would XP be like with APL added?

The most dramatic extension of the XP practices I can see is the way I was able to work with the on-site customer. To be more precise, we had an *on-site programmer*. Instead of an on-site customer sitting with the programmers, the two programmers work on-site with the customers. This has some inconveniences. But they are outweighed by being able to resolve questions in the environment in which the software will be used. The moral effect on our customers is dramatic. They know what we're doing, what they're going to get and when. And they're pulling more weight on the project than I've seen since, oh, APL projects in the 70s...

Coding with the customer seems to be an APL tradition not easily replicated in more verbose languages.

Communication is one of the four cardinal values of XP, especially communication between programmers through code. *Coding with the customer* extends this to the customer.

XP emphasises talking as an alternative to slower and more formal writing. A virtuous circle emerges in which questions are resolved almost immediately instead of being documented and managed. *Coding with the customer* tightens this still further, producing another turn of the circle.

Conclusion

Extreme Programming revives a fast and lively style of software development that will be familiar to APL programmers. XP adds disciplines to make software development predictable as well as satisfying and productive. Implementing a test rig in APL is almost trivially simple.

XP aims to take good programming practices and “turn the dials up to ten”. On one of those dials—communication—APL’s tradition of *coding with the customer* extends the dial.

APL programmers should look into XP practices. They are an opportunity to rehabilitate our tradition of fast and lively software production.

Some languages are more agile than others. Extreme Programming practitioners will want to explore how much further APL’s immediate-execution mode and extreme terseness can take XP practices.

Bibliography

Extreme Programming Explained: Embrace Change, Kent Beck, Addison-Wesley, 2000

Planning Extreme Programming, Kent Beck & Martin Fowler, Addison-Wesley, 2001

Extreme Programming Applied: Playing to Win, Ken Auer & Roy Miller, Addison-Wesley, 2002

Extreme Programming Explored, William C. Wake, Addison-Wesley, 2002

Questioning Extreme Programming, Pete McBreen, Addison-Wesley, 2003

Links

Addison-Wesley publishes The XP Series - <http://www.awl.com/cseng/xp>

The Agile Alliance is a coalition of Extreme Programming and other agile development styles - <http://www.agilealliance.org/home>

Extreme Programming is the Wiki home of XP lore on the Web - <http://c2.com/cgi/wiki?ExtremeProgramming>

Extreme Programming: A Gentle Introduction is the best overview on the Web of XP principles - <http://www.extremeprogramming.org/>

Extreme Programming in APL is my web log of a current APL project in which I have been exploring the application of XP practices - <http://aplxp.blogspot.com/>

XP 2003 conference web site - <http://www.xp2003.org>

XP Developer is a British XP site with its own Wiki and details of London's Extreme Tuesday Club - <http://www.xpdeveloper.com>

XProgramming.com holds many articles on XP and is the home of XP Magazine - <http://www.xprogramming.org/>

Technical Notes

```

▽ ShowSuite cases;case
[1]  A cases is overloaded: [1] tno [2] start [3] end
[2]
[3]  ⍵←⍵FNUMS[⍵FNUMS≠cases;]
[4]
[5]  :For case :In cns cases
[6]      ⍵←'[',(?2>case),']'
[7]      ⍵←sn ⍵FREAD case
[8]  :EndFor
▽

```

```

sn←{
0ερw.⍵NL 9:w.({w fv⍵w})⍵NL 2)
{wρ≠⍵1,ρw}(cw.({w fv⍵w})⍵NL 2)),w.((,∘'>'⍵NL 9){α w}''∇∘⍵'⍵NL 9)
}

```

sn contains awkward repetitions in order to avoid internal assignments, which its recursions would leave defined inside the namespaces it displays.

```

fv←{
1≥ρw:(',',⍵α)(⍵w)
⍵{w^.=1⍵w}ρ∘,⍵:(θ)((α~'' ')⍵q↑w)
(↑α,⍵':')((⍵1,ρw)ρ,⍵w)
}

```

The last function fv (format variables) exploits the frequency with which the test data incorporated variables of the same length, often taken from columns on mainframe data. For example:

```
Data.MF1.(PHASE FUND UNITS)←(2 3 4)(159 168 159)(0 0 21345.68)
```

So fv displays equilinear vectors as a table of headed columns.

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hacker's Corner: from APL 2 APL2C	Adrian Smith	96
At Play with J: Pick a Card, Any Card	McDonnell & Smith	101
Tools, Part 1. Basics	Dan Baronet	106
Web Services with Dyalog.NET	John Daintree	116
Getting to the Bottom of Things	Phil Chastney	132

Hacker's Corner: from APL 2 APL2C

by Adrian Smith (adrian@causeway.co.uk)

Has Adrian Gone Completely Potty?

That heading reminds me of the comment "it is really easy to spell banananana – it's just hard to know when to stop ..." which was attributed to a Camacho granddaughter. We know what APL2C is – it is Tilman Otto's excellent translator from APL2 to ANSI C for input to any compiler you can name. Why might I want to translate from some other APL to this dialect? To run RainPro (aka GraPL.exe) from the \$prompt on Unix, that's why.

Easy enough, you would think. 10 years ago, it would have been really easy as Rain was written in APL*PLUS/PC and would obviously run on APL2 with almost no modification. Today, there is one major stumbling block – Control Structures! Tilman has implemented Eke's 'Del' form and RainPro has been gently eliminating the right-arrow to the point where it has almost vanished from the workspace. So all I needed to try out my workspace in Tilman's translator was a little something to map from the keyword-style to the Del-style control code.

And here it is, all 210 lines of it. The comments flag up the limitations – if you can live with these (or don't mind extending it) feel free to mail me and demand a copy. My objective was compiled RainPro, so I know what I don't do (see the XP approach etc., etc.) so I just left it out.

So What Does It Do, Exactly?

Let's look at a simple test function in the 'before' and 'after' state.

```

      □vr 'ifelseif'
      ▽ r←ifelseif x;qwe
[1]  A Simple IF/ELSEIF block example
[2]  :if x>12
[3]    'x>12'
[4]  :elseif x>4
[5]    'x>4'
[6]  :elseif x>0
[7]    'At least it was +ve'
[8]  :end
[9]  'The End'
      ▽

```

```

      [vr] [fx] a2c 'ifelseif'
    ▽ r←ifelseif x;qwe
[1]   ▽1 0=(x>12) A If
[2]   : 'x>12'
[3]   : ▽1 0=(x>4) A ElseIf
[4]   : 'x>4'
[5]   : ▽(x>0) A ElseIf
[6]   : 'At least it was +ve'
[7]   ▽ A EndIf
[8]   ▽ A EndIf
[9]   ▽ A EndIf
[10]  'The End'
    ▽

```

You will have to refer back to Eke's article for the exact specification, but you can see the general idea. I am careful to preserve the original name of the structure as a comment, which makes debugging the results in APL2C a whole lot easier.

And How Does It Do It?

Here we go, with very little more comment. The comments in the code should be enough to go on:

```

      [vr] 'a2c'
    ▽ nr←clp a2c fn;cr;chk;dlb;dtb;uc;sink;lv1;targets;open;tgt;top;topline;end;
      pos;chk6;blk;ct;rep;items;code;forvar;header;ic;tmp;leco;AEE
[1]   A Convert control structures in 'fn' to APL2C ▽ format, returning CR
[2]   A Limitations - Repeat/Until not handled (recode with While and an extra test)
[3]   A - While/Continue combination not handled (While/Leave is OK)
[4]   A - If(x)/OrIf(y) is rendered as If(y)v(x) which could be a gotcha
[5]   A - combinations like If/elseif/andif are not all handled - recode
[6]   A - If blocks with empty 'then' sections are not handled!
[7]   A - Caselist 'abc' not handled - recode as Caselist 'a' 'b' 'c'
[8]   cr←[CR] fn
[9]   :if 0=pcr 0 [ERROR] 'Cannot take [CR] of ',fn 0 :end
[10]  A Little helpers
[11]  sink←[FX] 0'r+dlb s' 'r+(+/^\\'' ''=s)ts'
[12]  sink←[FX] 0'r+dtb s' 's+~1+((s='A')^~x\\s='''')1)ts 0 r+(-+/^\\'' ''=s)s'
[13]  sink←[FX] 0'r+uc v' 'r-[AV]([~32=ve[AV[97+;26]])+[AV+;v]'
[14]  nr←dtb'dlb''c[2] cr
[15]  A Break out all unquoted diamonds as these will spoil our day later on.
[16]  nr←0'0',nr 0 nr←1i''(nr=0')^(-x\\''=nr)) [PENCLOSE] nr
[17]  A Clear leading blanks and trailing blanks+comments
[18]  nr←dtb'dlb''nr
[19]  A Make an upper-case version for searching
[20]  chk←uc''3t''nr 0 chk6←uc''6t''nr
[21]  A Loops will need unique temporary variables
[22]  forvar←0 0 header←10nr
[23]  A The first thing is to make a matching level vector
[24]  A This assumes we NEVER use :Repeat .... :Until in RainPro!
[25]  lv1←(pch)0 0 targets←':FO' ':If' ':WH' ':SE'
[26]  :for tgt :in targets
[27]  lv1←lv1+(tgt)≡chk
[28]  :end

```

```

[29] 1vl+1vl-(c':EN')=chk o 1vl+1vl
[30] A Now to work through our target list, until none left
[31] targets+':UN' ':GO' ':RE',targets
[32] A Hopefully, we will never hit these as the loop conversion clears them
[33] targets=targets,':LE' ':CO'
[34] A Work from the bottom up so we handle inner loops before outer
[35] Next:tgt+1>targets o top+{<\$chk<tgt):1
[36] :If top>pchk
[37]   targets+1>targets
[38] :Else
[39]   topline+top>nr o end+top+((top+1vl)i-1+top>1vl)
[40]   blk+top>endchk6 A Possible structures inside this one
[41]   blk=(6*(top>end+1vl)=top>1vl)i-1blk A at the right level
[42]
[43] :Select tgt
[44] :Case ':IF' A Lots of options here - enumerate the ones we can do
[45]   AEE=(+/blk<':ANDIF'),(+/blk<':ELSEI'),(+/blk<':ELSE ')
[46]   topline+'(',(dlb 3+topline),')' A Main If test
[47] A Accumulate any 'OrIf' tests into the first test
[48] pos+top+((top>endchk)<':OR')/tpblk
[49] :for code :in pos A Preserve execution order here
[50]   topline+'(',(dlb 5+code>nr),')v',topline
[51]   (code>nr)+' ' o (code>chk)+' '
[52] :end
[53] :Select AEE
[54] :Case 0 0 0 A Simple case (no AndIf, ElseIf, Else here)
[55]   (top>nr)+'v',topline,' A If '
[56]   (end>nr)+'v A EndIf'
[57] :Case 0 0 1 A Plain If/Else structure
[58]   (top>nr)+'v1 0=',topline,' A If/else '
[59]   pos+top+1 o (pos>nr)+' ':pos>nr
[60]   pos+top+blk<':ELSE ' o (pos>nr)+' ':A Else'
[61]   (end>nr)+'v A EndIf'
[62] :CaseList ~3t~2t~12 A If/Elseif/... with no fallthrough
[63]   (top>nr)+'v1 0=',topline,' A If ' o rep+2>AEE
[64]   pos+top+1 o (pos>nr)+' ':pos>nr
[65]   :for ct :in ~1+irep
[66]     pos+blk<':ELSEI' o (pos>blk)+' ' o pos+top+pos
[67]     (pos>nr)+'v1 0=',(dlb 7+pos>nr),') A ElseIf'
[68]     pos+pos+1 o (pos>nr)+' ':pos>nr
[69]   :End
[70]   pos+top+blk<':ELSEI'
[71]   (pos>nr)+'v(',(dlb 7+pos>nr),') A ElseIf'
[72]   (end>nr)+'~1+e(rep+1)/c'v A EndIf o'
[73] :CaseList ~3t~12,~1 A If/Elseif/... /Else
[74]   (top>nr)+'v1 0=',topline,' A If ' o rep+2>AEE
[75]   pos+top+1 o (pos>nr)+' ':pos>nr
[76]   :For ct :in irep
[77]     pos+blk<':ELSEI' o (pos>blk)+' ' o pos+top+pos
[78]     (pos>nr)+'v1 0=',(dlb 7+pos>nr),') A ElseIf'
[79]     pos+pos+1 o (pos>nr)+' ':pos>nr
[80]   :End
[81] A We just need one fallthrough clause here
[82] pos+top+blk<':ELSE ' o (pos>nr)+' ':A Else'
[83] (end>nr)+'~1+e(rep+1)/c'v A EndIf o'
[84] :CaseList 3t~12 A If/AndIf/... with no Else block
[85] (top>nr)+'v',topline,' A If ' o rep+1>AEE
[86] :For ct :in irep
[87]   ((top>ct)>nr)+'v(',(dlb 6+(top>ct)>nr),') A AndIf'
[88] :End

```

```

[89]      (end>nr)+~1!e(ct+1)/<' A EndIf o'
[90] :CaseList (112),~c(0 1) A If/AndIf/... with Else block
[91]      (top>nr)+~v1 0=,topline,' A If ' o rep+IAEE
[92]      :For ct :in trep
[93]      ((top+ct)>nr)+~v1 0=(, (dlb 64(top+ct)>nr),') A AndIf'
[94]      :End
[95]      pos=top+rep+1 o (pos>nr)+~:,pos>nr
[96]      pos=top+blk+~:ELSE '
[97] A Extract Else clause, diamond up and repeat as needed
[98]      code=(end-pos+1)tpos+tpnr
[99]      (pos>nr)+e(rep+1)/<:', (~1!e(nr[code]),~c'o'),'o'o'
[100]      nr[code]+c' ' o (end>nr)+~'
[101] :Else A die horribly
[102]      ERROR 'a2c[62] Cannot handle pattern: ',e(AEE/'AndIf ' 'ElseIf ' 'Else ')
[103] :End
[104]
[105] :Case ':SE' A Select will need Caselist repeating
[106]      pos=top+(blk+~:CASE ' ':CASEL' ':ELSE ')/tpblk
[107]      topline+~c', (dlb 74topline),')~'
[108]      :for ct :in tppos
[109]      :if ~:ELSE '= (pos[ct]-top)>blk A No condition for :Else clause
[110]      :if ':CASEL'= (pos[ct]-top)>blk A break out items
[111]      items+dlb 94pos[ct]>nr
[112]      items+11'(1, (' '=items)^(~<\'' '=items)) [PENCLOSE ' ',items
[113]      topline+topline,e'('','items','') o rep+items
[114] A Find the code block, diamond up and repeat for each item
[115]      code=(pos,end)[ct+1]-pos[ct]+1tpos[ct]+tpnr
[116]      :if 0<pcode
[117]      ((1+pos[ct])>nr)+~1!erep/<:', (~1!e(nr[code]),~c'o'),'o'
[118]      nr[1+code]+c' '
[119]      (pos[ct]>nr)+~A ',pos[ct]>nr
[120]      :else A Empty code block is legal!
[121]      ((pos[ct])>nr)+~1!erep/<:'o'),'A ',pos[ct]>nr
[122]      :end
[123]
[124]      :Continue
[125]      :else A single item only
[126]      topline+topline,'(',(dlb 54pos[ct]>nr),')'
[127]      :end
[128]      ((pos[ct]-top)>blk)+~'
[129]      :end
[130]      (pos[ct]>nr)+~: A ',pos[ct]>nr
[131]      :end
[132] A Allow for fallthrough clause if present
[133]      :if (<:ELSE ')+~blk
[134]      (top>nr)+~v<\(' ,topline,') ,1 A :SelectElse'
[135]      :else
[136]      (top>nr)+~v<\(' ,topline,') A :Select'
[137]      :end
[138]      (end>nr)+~v A EndSelect'
[139]
[140] :Case ':FO' A For loop uses a temporary variable to save repeated evaluation
[141]      forvar=forvar+1 o tmp='ivar',(*forvar) o leco+0 0
[142]      ic=dlb 44topline o ic=dtb (~1+ict:')tic
[143]      items+dlb (3+(1+topline)1:')tpos+tpnr
[144]      (top>nr)+tmp,~',items,'o'o(p',tmp,')o',ic,~(1+IC)=',tmp
[145] A Handle :Leave and :Continue in the context of this loop
[146]      pos=top+((top!end!chk)+~:LE',tmp o leco[1]+1 o (pos>chk)+~'***' o :end
[147]      :if 0<ppos o (pos>nr)+~LE',tmp o leco[1]+1 o (pos>chk)+~'***' o :end
[148]      pos=top+((top!end!chk)+~:CO')/tpblk

```

```

[149]      :if 0<ppos ◊ nr[pos]~ε~CO',tmp ◊ leco[2]~1 ◊ chk[pos]~ε~***' ◊ :end
[150]      (end>nr)~(leco[2]~CO',tmp,'◊'),'∇ A EndFor',(leco[1]~LE',tmp,':')
[151]      header~header,';',tmp
[152]
[153]      :Case ':WH' A While is like :If but with -ve test!
[154]      (top>nr)~'∇~',(dlb 6+topline),' A While '
[155]      A Handle :Leave in the context of this loop (forget :Continue for now)
[156]      pos~top~((top!end+chk)ε~LE')/tpblk
[157]      :if 0<ppos
[158]          forvar~forvar+1 ◊ tmp~'whl',(*forvar)
[159]          (pos>nr)~LE',tmp ◊ (pos>chk)~***'
[160]          (end>nr)~'∇ A EndWhile◊LE',tmp,':'
[161]      :else
[162]          (end>nr)~'∇ A EndWhile'
[163]      :end
[164]
[165]      :Case ':RE' A Return assumed (will already have failed on :Until)
[166]      (top>nr)~'~0 A :Return'
[167]      :CaseList ':LE' ':CO'
[168]          □ERROR 'Undetected exit tag in: ',topline
[169]      :Case ':GO'
[170]          (top>nr)~'~',(dlb 6+topline),' A :GoTo'
[171]      :Case ':UN'
[172]          □ERROR 'Unhandled Repeat/Until structure: ',topline
[173]      :EndSelect
[174]      A Kill the lookup entry we just completed
[175]      (top>chk)~***'
[176]      :End
[177]
[178]      ~<0<ptargets)†Next
[179]
[180]      A Return function ready to fix (resplit new multiple lines)
[181]      A Header may have new locals and we can dump empty lines
[182]      (1>nr)~header
[183]      nr~(0<†'p'nr)/nr
[184]      nr~ε~o',nr
[185]
[186]      A We cannot handle † here so use ,[1] instead
[187]      nr~nr Δssr '†' ',[1]'

.... and a few more of the same .....

[207]
[208]      A Break out at diamonds and erect as a matrix to make □CR equivalent
[209]      nr~1†~((nr~o')^~ε~'~nr))□PENCLOSE nr
[210]      nr~nr
    ∇

```

Remaining Gotchas

The only real nasty is □ELX which cannot be emulated in an APL2-style of error trap. I use very few of these, and I think the answer is to replace all of them by a utility called ΔEA which I can then simply replace with □EA at conversion time. This will give me a decent chance of running on APLX also, which has to make it worth the effort. Apart from that, it all works rather well. It is nice to see RainPro running as a single-file EXE – makes it a very easy product to ship!

Pick a Card, Any Card

by Gene McDonnell & Richard Smith (rcs36@cam.ac.uk)

Introduction

The crowd sits and waits, eagerly anticipating the showman's grand entrance. Eventually he arrives, bringing his glamorous assistant and a pack of cards with him. He selects a volunteer from the audience and asks them to pick five cards out of the pack and give them to the assistant, without of course seeing them himself. The assistant then shows him four of the cards, and, after a suitable dramatic pause, the showman identifies the fifth. The crowd applauds, and the magician and his assistant leave after a few repeats to show it wasn't a fluke.

Those of you who were at last month's Finnish conference will already have seen this spectacle, know that the showman in question is actually Adrian, and moreover will know the twist, which is that the assistant is not a 5'6" blonde but a 4" by 2" grey box which comes with a screen and a stylus.

I'm sure that some of you asked him afterwards how he did it, and I suspect that instead of the usual "Magic!" he said "Wait for the next Vector"; this article shows you how the trick works and how the digital Esmerelda is written. (Of course, the assistant is far more important than the magician.)

The Trick

Anyone who reads *New Scientist* can skim-read this section, as the trick follows the same principle as that described in one of its recent articles. However, it is obviously vital to understand the trick before trying to understand the implementation.

The interesting nature of this trick stems from the fact that using a simple analysis, it would seem to be impossible. 4 cards can only encode $4!$, or 24, combinations, while there are 48 options for the fifth card. We can narrow it down by using one of the cards to pin down the suit of the fifth (there must be at least 2 cards which share a suit in the five), but then we only have 3 cards, giving $3!$ or 6 combinations, to account for 12 possibilities.

The secret, of course, lies in the ordering of the cards. Because the assistant gets to choose which of the five cards is hidden, she can choose such that the hidden card is within 6 of the visible card in that suit. For example, if two of the cards were the

10 and 2 of spades, hiding the 10 would not work ($10 - 2 = 8$, which is more than 6), but hiding the 3 gives a difference of 5 (J-Q-K-A-2). There is always a way to arrange two cards in a suit so this is true.

Now we can use the 6 combinations of the other three cards, combined with a suit card, to find the missing card. We take the relative sizes of the cards, and use their order to generate a number: 1 for small-medium-large, 2 for small-large-medium all the way up to 6 for large-medium-small. Then add this number to the number on the exposed card of the suit to find the missing card.

There is one small complication – what if we have, say, two 3's? We define the suits to have an order, so that the 3 of spades is 'higher' than hearts, diamonds or clubs.

The Implementations

Both of us have produced an implementation of Esme; Gene's is written in J and Richard's is in Dyalog APL. (Richard's is the one you may have seen in Finland, running under Pocket APL.) Both are very simple and easy to follow.

Gene's Version

```
suits =: '♠♥♦♣'
values=: 'A23456789TJQK'
```

A♠	2♠	3♠	4♠	5♠	6♠	7♠	8♠	9♠	10♠	J♠	Q♠	K♠
A♥	2♥	3♥	4♥	5♥	6♥	7♥	8♥	9♥	10♥	J♥	Q♥	K♥
A♦	2♦	3♦	4♦	5♦	6♦	7♦	8♦	9♦	10♦	J♦	Q♦	K♦
A♣	2♣	3♣	4♣	5♣	6♣	7♣	8♣	9♣	10♣	J♣	Q♣	K♣

Here is a sketch showing the results of the steps in performing the magical trick.

The argument is list of 5 distinct integers from i.52.

```
y. =: 45 24 49 20 40
vsi y.
+-----+
|7S|QD|JS|8D|2S|
+-----+
```

Sort the integers and append them below a 2 x 5 table of suits and values.

```
sv =. (|) , ~ [: |: 4 13 " _ #: 1) @ /: ~) y.
sv
1 1 3 3 3
7 11 1 6 10
20 24 40 45 49
```

Get a list of sublists, each sublist giving the indices of cards having the same suit.

```
ta =. ({. < /. 0 1 2 3 4 " _) sv
ta
+-----+
|0 1|2 3 4|
+-----+
```

Select the sublists having more than one card.

```
tb =. (] # ~ 1: < [: # & > ]) ta
tb
+-----+
|0 1|2 3 4|
+-----+
```

Select one of these sublists at random.

```
tc =. (> @ ({ ~ ? @ #)) tb
tc
2 3 4
```

Select two of its indices at random, preserving order.

```
td =. (] { - /: ~ @ (2 & ?) @ #) tc
td
2 4
```

Move the two columns with the selected indices to the front.

```
pc =. td({[:{[ , ] -. ~ 0 1 2 3 4 " _) [ { " 1 [: ] . )sv
pc
1 10 7 11 6
40 49 20 24 45
```

Find the commuted difference of the first two columns of the value row. This is positive, in the range 1-12. Take top row only of result, and append difference.

```
cd =. ((([: - - / 2: { . {-) , ~ {:) @ ). )pc
cd
40 49 20 24 45 9
```

If the tail is less than 7, delete the second item, otherwise the first.

```
dc =. ((([: < [: < [: < { : < 7:) { ]) cd
dc
49 20 24 45 9
```

If the tail is less than 7, leave it unaltered; otherwise replace it by 13 - tail.

```
ce =. ([ ^ (13 & | @: -) @. (7 & <:)) { : dc)4 } dc
ce
49 20 24 45 4
```

Determine which special permutation to apply, using the tail value as determinant; drop the tail; apply the permutation.

```

rs =: (((5 3 & p. - -. @ (2 & |)) {: ce) A. i. 4) { } : ce
rs
24 45 49 20

vsi rs
+-----+
|QD|7S|JS|8D|
+-----+

```

The magician sees that the third card is a spade, and that the other cards are in the order 1 2 0, which is the fourth permutation of order 3. The fourth card beyond JS is 2S. QEF.

Richard's Version

This implementation is very much in the same mould, with `IO` set to 0 so as to match the J. Here is the code for the core algorithm, together with the (totally minimal) user-interface to make it workable in the field on Pocket APL:

```

▽ Go;inp;deck
[1]  A Run ESME simulator
[2]  'Tell me the 5 cards ...'
[3]  'Suits are CDHS and cards'
[4]  'range from A,2 to 9,TJQKA'
[5]  'e.g. 5s 8d 3c 4h as'
[6]
[7]  Next:⎕←>'
[8]  inp←1⊥,⎕ ⋄ →(pinp)⊥Done
[9]  deck←vCards2Nums inp
[10] →(5#pdeck)⊥Badboy
[11] →(⌵/deck<52)⊥Badboy
[12] 'Say these 4 ...' ⋄ ' '
[13] Nums2Cards Esme deck
[14] ' ' ⋄ '(in this order!)' ⋄ ' '
[15] →Next
[16] Badboy:'Try to fool me eh!'
[17] 'We need 5 distinct cards here ...' ⋄ ' '
[18] →Next
[19] Done:'Easy, for a PocketAPL'
▽

▽ nv←Cards2Nums str;lkp;vtv
[1]  A Look up names and return card index
[2]  lkp←,⊆names°.suits
[3]  vtv←1⊥''(⌵1,stre', /;)c',',toupper str
[4]  nv←lkp⌈vtv
▽

```

```

▽ r←Esme cards;sv;ta;tb;tc;td;pc;cd;diff;dc;⌈IO;⌈ML
[1]  A Do the sorting. See MagicCD.doc
[2]  ⌈IO←0
[3]  ⌈ML←3 A for partition enclose
[4]  sv←{3 5p{({w÷13},{13|w},w)w[4w]}cards
[5]  ta←(1+sv[0;])÷5
[6]  tb←({1≤1<pw}ta)/ta
[7]  tc←0>tb[?ptb]
[8]  td←tc[{w[4w]}2?ptc]
[9]  pc←sv[1 2;td,((15)-td)]
[10] cd←pc[1 0;] ∘ diff+,-,-/1 2↑pc
[11] dc←((6≥diff),(6<diff),1 1 1)/cd
[12] :If 6<diff ∘ diff+13-diff ∘ :End
[13] dc←dc[0;0,1+4dc[1;1 2 3]] A Reorder numerically
[14] r←dc[(diff-1)Δperms]
▽

```

```

▽ vtv←Nums2Cards nv;lkp
[1]  A Report names
[2]  lkp←(names.,suits),c'??'
[3]  vtv←tolower$lkp[nv]
▽

```

```

names suits
A23456789TJQK CDHS
Δperms
1 2 0 3 1 3 0 2 2 1 0 3 2 3 0 1 3 1 0 2 3 2 0 1
Go

```

Tell me the 5 cards ...
Suits are CDHS and cards
range from A,2 to 9,TJQKA
e.g. 5s 8d 3c 4h as

>5c 9d ad th 2c
Say these 4 ...

9d ad 2c th

(in this order!)

>

Easy, for a PocketAPL

There is still a small amount of brain-work left for the magician – in this case the thought process would be “It’s a club. *Nine, Ace, Ten* is *Medium, Small, Large* (Ace is low) which is 3 on our scale of 1-6 (SML->LMS) so 2+3 is the Five of Clubs.” As for the ‘inverse’ function, it could be an exercise for the reader, but it would take longer to enter the data than it takes to do the logic in your head. Anyway, people would suspect a WiFi network!

Tools, Part 1. Basics.

by Dan Baronet (dbaronet@yahoo.com)

In the following article I use terms specific to our trade. You won't find them in the dictionary but I assume the reader is familiar with words such as 'monad', 'global' (as a noun) and 'default' (verb). I also use quotes and angle brackets to help determine the type of the object I am referring to. 'Quotes' denote a variable or workspace and <angle brackets> refer to a function/operator or file. Often, the context is sufficient to remove ambiguities. Italicized words have a special meaning and often precede their definition.

Introduction

Programmers get used to some tools, often their own tools. Most APL vendors come with tools, which are not usually compatible with other APLs.

I have been working with different APLs and I have had a need for cross platform tools. Rewriting them for each APL is cumbersome. When you make modifications to a version on one of the APLs you have to keep track of it in order to change the other versions in the other APLs. I eventually put everything under one platform and made the change there, distributing the changes across the various APLs. This will be the subject of another article, if time permits...

This is the first of a series of articles on tools for APL.

A function to parse command lines

This function is the first of my tools to have been ported across platforms. That's because I use it almost everywhere. It's one of my favourite functions. It allows parsing a string following specific rules. I wrote it when I started using user commands¹. I was tired of having to write up to 20 lines before doing anything useful every time I had to write a user command. It also allowed standardizing my code by using a uniform syntax for all my user commands and all my code. Eventually it was incorporated in my code everywhere else.

Now, the way software is written is not always by careful analysis of the problem. More than often it is the result of experimentation. Some code is written to study

¹ User commands are specific to the APL+Win family of APLs

the problem. The prototype becomes the program and is refined until it reaches, in an asymptotical fashion, the desired result... Until the requirements change again...!

This case is no different. I wrote the program specifically for user commands in APL/PC a while back when nested arrays weren't implemented. Over time it proved useful for other things and I adapted the code accordingly. It has doubled in size since I first wrote it. Typical.

In the following text I give the user commands case as an example but remember it could apply to any other program that takes a string to be parsed as argument. Here it is:

PARSE utility.

Keywords: USER COMMAND, ARGUMENT, TOKEN, SWITCH, DELIMITER

This function will return the parsed *tokens* of a string argument given to a *user command*. It will also set a series of globals, one per allowable command *switch*.

To understand what this means we must first know that the user command processor does not perform any parsing of its own. The processor will only take whatever is to the right of the command name and supply it to the APL function performing the command. For example, assuming the user command 'bingo' has been defined. If we enter

```
]bingo a/b=x O41
```

the processor will start the program <cmdbingo> with the *argument* ' a/b=x O41', all spaces included.

That's the way it works. It is general and allows us to do whatever we want. It is our problem to make some sense of the string given as argument. We can adopt a convention. Consider DOS or Linux: commands in these systems follow a loose convention using *switches*². Take the Linux command line 'cp', which copies a file to another:

```
cp filea fileb -R -v
```

Here 'filea fileb' is the *argument* list and 'R' and 'v' are *switches*. There are other acceptable switches to this command but they are not used nor shown here. In this

² you'll sometimes also hear the term *modifier*

case the argument list is made of 2 tokens: *filea* and *fileb*. Here, the character identifying a switch is the hyphen (-). DOS uses '/' as the switch delimiter.

Another system closer to APL is *Logos*, a SHARP APL product for the mainframe. Commands in *Logos* use a similar convention but much tighter. The switch delimiter in *Logos* is the '+'.³

A switch alters the behaviour of the command. A token is what the command needs to do its job.³

Conventions

We can use our own convention. In this case the convention I use is

- All commands take 0 or more *tokens* and accept 0 or more *switches*.
- All tokens come first (all switches come last).
- *Tokens* are separated by one or more spaces. A *delimiter* identifies and precedes a switch.
- *Switches* may be absent, present and may take a value (a string).
- *Switches* and their values end at the beginning of the next switch or at the last non-blank character.
- *Switches* can be entered in any order.
- *Tokens* and switch *values* may be surrounded by quotes to include spaces and switch delimiters.
- *Switches*, once prefixed, must produce a valid variable name (case sensitive).

Example:

```
]MYCOMMAND 1ST-ARG 2ND-ARG MEBBE-3RD /SWITCH1 /SWITCH2=abc
```

Here '1ST-ARG', '2ND-ARG' and 'MEBBE-3RD' are the *tokens*. They are separated by spaces.

'SWITCH1' and 'SWITCH2' are switches that modify the behaviour of 'MYCOMMAND'. 'SWITCH1' is present and 'SWITCH2' is not only present but

³ it really is also an *argument*. I need to use a different term as 'argument' is already used for the user command function and the `ΔPARSE` function.

has the *value* "abc". There may be other acceptable switches that have not been specified. Here, the character starting a switch, its *delimiter*, is the slash (/).

The function

The APL $\langle \Delta \text{PARSE} \rangle$ function will return (in this case) the string '*1ST-ARG*2ND-ARG*MEBBE-3RD' and set the global variables ' $\Delta \text{SWITCH1}$ ' to 1 and ' $\Delta \text{SWITCH2}$ ' to 'abc'. The star character (*) is an example. In practice the function should use a token *delimiter* that cannot be entered from the keyboard such as $\square \text{TCDEL}$.

Example. Here's the function associated to the user command 'MYCOMMAND':

```
[0] CMDMYCOMMAND ARG;TOK;ΔSWITCH1; ΔSWITCH2; ΔSW3 ;ΔSW4 ;DELIMITER
[1] A THIS COMMAND WILL SHOW WHAT IS PARSED BY <ΔPARSE>
[2] TOK+ '/SWITCH1 /SWITCH2= /SW3 /SW4 [=]' ΔPARSE ARG
[3] (≠+/TOK=DELIMITER+1≠TOK), ' TOKENS ENTERED'
[4] 'VALUE OF SWITCH1: ', ≠ΔSWITCH1
[5] 'VALUE OF SWITCH2: ', ≠ΔSWITCH2
[6] 'VALUE OF SW3: ', ≠ΔSW3
[7] 'VALUE OF SW4: ', ≠ΔSW4
```

The user command associated with this function will only accept 4 different switches: SWITCH1, SWITCH2, SW3 and SW4. The syntax of the command is specified on line 2. It is the left argument to $\langle \Delta \text{PARSE} \rangle$.

Here SWITCH1 and SW3 can only appear without any value assigned to them. If /SWITCH1 appears in the command line (which is passed as argument to this function) then variable $\Delta \text{SWITCH1}$ will contain the boolean scalar value 1. If not, it will contain the boolean scalar value 0.

Same thing for /SW3 and ΔSW3 .

/SWITCH2 can only appear with a value assigned to it, that is followed by an '=' sign and a non-empty string. If it appears in 'ARG' (the command line), variable $\Delta \text{SWITCH2}$ will contain the assigned string, that is the string to the right of the '=' sign, otherwise it will contain 0.

/SW4 is ambiguous; if it appears in the command line without a value then ΔSW4 will be set to 1. If it is assigned (with the '=' character) then ΔSW4 will be set to the string next to it and if NOT present on the command line ΔSW4 will be assigned 0. The '[=]' meaning 'assignment not compulsory'.

These are the rules of this particular command. If someone enters

```
]MYCOMMAND now I know /SW4 /SWITCH2=Oh! Really?
```

then we'll have $\Delta\text{SWITCH1}=0$, $\Delta\text{SWITCH2}=\text{'Oh! Really?'}$, $\Delta\text{SW3}=0$, $\Delta\text{SW4}=1$ and TOK will contain an 11 character string whose 3 tokens ('now', 'T' and 'know') are each preceded by the same character ⁴.

Problems with tokens and switches arise when they must contain spaces or switch delimiters. Using quotes around the tokens or the switch values will circumvent this problem. $\langle\Delta\text{PARSE}\rangle$ will recognize empty strings and strings with quotes which must be doubled. Here's another example:

```
]xxx OK ' ' 's/he is' 'I'M HERE' /S1=' ' /DT='3/12/27'
```

Here the command 'xxx' has 4 tokens: 'OK', an empty string, 's/he is' (with '/') and a string with embedded spaces and (doubled) quotes. Switch S1's value is a single quote and DT's value contains '/s'.

The syntax specification

The left argument specifies how the right argument to $\langle\Delta\text{PARSE}\rangle$ must be parsed. It contains

1. the switch *delimiter*
2. all the acceptable *switches* and their application and possibly domain
3. the character used to separate the *tokens* (defaulted)

1. $\langle\Delta\text{PARSE}\rangle$ uses the first character of the left argument as switch delimiter. In other words if '/a /b=' is the left argument then '/' is the delimiter. If '+a +b=' is the left argument then it is '+'.

2. Each allowed switch is described. It starts with the delimiter then the name. If it may take a value it is followed immediately by '[=']. If it requires a value it is followed by '='. Moreover, if only specific strings are accepted they can be listed here. For example if only the cities of PARIS, LONDON and ROME are acceptable then

```
 '/city= LONDON PARIS ROME'  $\Delta\text{PARSE}$  arg
```

would do it. Note that this is not a foolproof test. The verification relies loosely on $\underline{\epsilon}$.⁵

⁴ This character depends on the APL platform. It is $\square$ TCDEL for APL2000

If only a set of characters is acceptable then '€' should be used instead of '='. As in
'/code€XYZ' ΔPARSE arg

In this case any combination of the letters X, Y and Z will be accepted (minimum 1 character).

The complete specification is

/sw	accept only sw without a value
/sw=	accept only sw with a value
/sw[=]	accept sw with or without a value
/sw=s1 s2	accept sw only with s1 or s2 as value
/sw[=] s1 s2	accept sw with or without a value; if one is supplied accept only s1 or s2
/sw€xyz	accept sw only with a combination of the characters x, y or z as value
/sw[€]xyz	accept sw with or without a value; if one is supplied accept only a combination of the characters x, y or z as value

3. The character used to separate the argument's tokens.

By default tokens are returned with surrounding quotes stripped (same for switch values) and separated by □TCDEL. This can be changed. The left argument to <ΔPARSE> may contain a special *empty* switch name to specify this character. As in (there are no other switches specified in this example).

```
'/=*' ΔPARSE 'a b dsa'
```

which specifies the '*' as the token delimiter and will return

```
*a*b*dsa
```

⁵ For example, /city=PARIS ROME would be accepted

Why Δ ? Why globals?

When I first wrote $\langle \Delta \text{PARSE} \rangle$ it was for user commands. ' Δ ' is the character used throughout the user command processor to limit name clashing. I was following the convention in use.

Also, because APL/PC (for which this was written for) wasn't using nested arrays I could not return arguments and switches together. The natural thing to do was to set globals. The scheme I used was to set global names with the same name as switches proceeded by, you guessed it, ' Δ '. So switch X was going to be stored in variable ΔX^6 .

Now, the first thing my first user wanted to do was to change this prefix so I made my first (quick) modification: if there exist a local variable named ' Δ ' in the immediately calling environment to $\langle \Delta \text{PARSE} \rangle$ I use its contents as prefix. See here; ' Δ ' is assigned ' $\Delta\Delta$ ', the prefix to use:

```
[0] CMDMYCOMMAND ARG;TOK;ΔΔSWITCH1;ΔΔSWITCH2;ΔΔSW3 ;ΔΔSW4 ;Δ
[1] A THIS COMMAND WILL SHOW WHAT IS PARSED BY <ΔPARSE>; 'Δ' must be local
[2] Δ+'ΔΔ' ♦ TOK←'/SWITCH1 /SWITCH2= /SW3 /SW4 [=]' ΔPARSE ARG
[3] (*+/TOK=1+TOK),' ARGUMENTS ENTERED'
[4] 'VALUE OF SWITCH1: ',*ΔΔSWITCH1
[5] 'VALUE OF SWITCH2: ',*ΔΔSWITCH2
[6] 'VALUE OF SW3: ',*ΔΔSW3
[7] 'VALUE OF SW4: ',*ΔΔSW4
```

This is ugly, I concede. I did with what I had.

Exceptions

If $\langle \Delta \text{PARSE} \rangle$ detects something wrong it will signal an exception. Under the user command processor the error is *resignalled* outside (unless you've disabled that "feature").

Validation

Obviously once these variables are set you may need to validate them. You need to verify that if, for example, switch 'DATE' is expecting a date, that it was, indeed, given a date. Or that /AGE=xx was given a single valid positive integer. And so on. Since $\langle \Delta \text{PARSE} \rangle$ always returns values as character strings you must at least $\square$ FI them when you really want numbers. This is pretty much beyond the

⁶ since then APL2000 has come up with a limited function, $\langle \Delta \text{GETOPT} \rangle$, to fetch switches values, but it's very different

scope of <ΔPARSE>. It does only provide string and character set verification. And that is already beyond the scope of parsing.

Default

You may also want to default some values. For example, if /FILE isn't set you may want to default it to 'MYFILE'. Or if /LEVEL wasn't specified you may want to default it to the integer 3. In the first case you'd have to do something like

```
*(0<ΔFILE)/'ΔFILE←"MYFILE"'
```

and in the second case

```
*(0<ΔLEVEL)/'ΔLEVEL←3' ⋄ ΔLEVEL←⊞FI⊗ΔLEVEL
```

the ⊞FI⊗ is to cover the case where /LEVEL was indeed set to a string representing a number.

This default value thing is also annoying. It would be nice to have something like

```
ΔFILE←ΔFILE ΔDEFAULT 'MYFILE' or
```

```
ΔLEVEL←ΔLEVEL ΔDEFAULT 3
```

So I created such a function. It is smart enough to convert the switch value (the left argument) to numeric if it "sees" that the default is also numeric.

Where do I find these functions?

<ΔPARSE> and <ΔDEFAULT> are both part of my user command files. You can find the file <DANBCMDS.SF> for APL+Win and for SAX⁷ at <http://www.milinta.com>. They also exist in the workspace 'utils' under the section of your favourite APL at the same location.

How do I use them with ucmds?

If you've used user commands before you know that you must regroup all the items a user command needs under the name GRPCMD... The functions <ΔPARSE> and <ΔDEFAULT>, if needed, must be specified there. They must, of course, also exist in your user command file. For example, if command ABC requires functions <sub1> and <ΔPARSE> you would do, to make it effective:

⁷ a function to simulate ⊞UCMD exists under SAX in the STASK workspace at the same www location

```
GRPCMDABC+ 'CMDABC sub1  ΔPARSE'
]usave .GRPCMDABC
```

User commands already have <ΔGETOPT>. What's wrong with it?

Nothing. It's different. It handles simple cases as well as <ΔPARSE>. <ΔPARSE> is standalone and sets globals. It can be used anywhere. <ΔGETOPT> uses globals and subroutines and returns a result. It can only be used with user commands. It's smaller but does less. It does not handle unknown switches, for example.

Features:

- standalone, no subroutine or globals needed
- the token delimiter may be changed using the null switch
- no need to enter the full name of a switch; the minimum to distinguish it from the others is sufficient
- unspecified switches are refused
- the prefix for the variables to contain the switch values may be changed
- the switch delimiter can be chosen
- minimal string validation can be performed by specifying the acceptable strings or characters
- a separate default function returns a specific value if a switch hasn't been specified

An example

This user command runs under APL/II. It translates a string (the argument to the command) into an argument suitable for □SOUND to play the string in MORSE code. The string can be made out of several words (tokens), each returned preceded by a space (/ =). It accepts 3 switches: FREQ to specify the frequency to use (default 1024), PER to specify the duration of a dot and boolean TEST to also perform a test using □sound:

```
▽ CMDMORSE STR;F;LD;LP;LS;□IO;LIST;ΔFREQ;ΔPER;ΔTEST
[1] R USE □SOUND TO PLAY STR IN MORSE CODE
[2] STR←1+ '/' = /TEST/FREQ= /PER= ' ΔPARSE STR
[3] F←ΔFREQ ΔDEFAULT 1024
[4] LP←ΔPER ΔDEFAULT 50
[5] LS←[0.5×LD~[2.5×LP R LENGTH OF SPACE,DASH,DOT (PERIOD)
[6] STR←UPPERCASE STR ◊ LIST←Morse[;□io+1]
[7] F←(3 2 ρF,LP,F,LD,0,LS)[,3,[1.1]'.- 't(1+,Morse[LISTtSTR;])~' ';]
[8] □SOUND ΔTESTΔRESULT←F
▽
```

It's a bit overkill for the task at hand but it gives an idea on how useful it can be (imagine a fn with 12 switches!)

By contrast lines [2,3,8] would be, using <ΔGETOPT>

```
[2] STR←ΔSQUEEZE (⌈1+STR1 '/'⌋)↑copy←STR
[3] F←1024 ⋄ →(2=pp↑copy ΔGETOPT '/FREQ')/⌈1c+1 ⋄ F←⌈FI t
[8] ⌈SOUND (''=copy ΔGETOPR '/TEST')/ΔRESULT+F
```

Wish list⁸

I would like another pseudo switch in the left argument to specify the prefix. It might work by using /Δ= as in /Δ=ΔΔ (for "use ΔΔ as prefix"). See:

```
'/Δ=ΔΔ /SWITCH1 /SWITCH2= /SW3 /SW4 [=]' ΔPARSE ARG
```

Another thing I'd like the function to do is return command argument(s) followed by switches as a nested array. This would allow performing multiple assignments to random variables and avoiding the 'globals' specification. Something like:

```
(tokens sw1 sw2)← '/Δ=+ /sw1 /sw2=' Δparse cmdarg
```

The ability to ignore the case of switches might also be interesting. Switch /sa and /SA would be considered the line. This would be in line with the APL+Win version of user commands.

Limitations of this particular version

A switch may not be the exact beginning of another switch. For example, /SW and /SWITCH may not coexist, but /SW1 and /SWITCH may. It doesn't also verify that your prefix is valid. If you set it to something that produces an invalid variable name it will stop ungracefully.

Note

This is not a foolproof function. It's a tool to help saving time parsing an argument line. You must do your own checking from there.

Finally, I wish STSC (and subsequent owners) had made something like this available right at the beginning. I think it would have made user commands easier to adopt ...

⁸ these have already been solved in a test version

Dyalog.NET: Putting it all Together

by John Daintree, Senior Programmer, Dyalog Ltd
email: johnd@dyalog.com mailing list: dotnet@dyalog.com

Introduction

This is the third of a number of papers discussing Dyalog's Microsoft .Net Framework compatible product, Dyalog.Net. This paper brings together the topics raised in the first two papers, and uses them to write APL web services and web pages. The reader may benefit from re-reading the first two Dyalog.Net papers. All of the features discussed here are also available in the newly released Dyalog APL version 10.0.

Dyalog has set up a mailing list to discuss Dyalog.Net related issues. To subscribe to this list, send an email to dotnet@dyalog.com with SUBSCRIBE as the subject.

Hello (The Whole) World

This is the sample "Hello World" program that we introduced in the last paper.

```
vhello
[1] ⍵←'Hello World'
▽
```

With a couple of minor "tweaks" we can enable this function to be run from within a web page:

```
vhello args
[1] :Access Public
[2] :ParameterList Object,EventArgs
[3]
[4] Response.Write 'Hello World!'
▽
```

The most important change is on line [4]. The assignment to quad has been replaced with a function call to something called `Response.Write`. At the moment we will just say that this will cause the string "Hello World!" to be displayed on the client's browser. But how is this achieved? Where does `Response` come from? What are `EventArgs` and `Object`?

Anatomy of a Dyalog.NET Web Page

Initialisation

Typically the above piece of script will be embedded in an .aspx file. This file contains both the script that controls the variable content of the page and the html that defines the static content of the page, the control layout and so on. Here is the full content of the .aspx file for the hello example:

```
<script language="apl" runat="server">

vhello args
:Access Public
:ParameterList Object,EventArgs

Response.Write 'Hello World!'
v
</script>

<body>
<form runat=server>
<asp:Button id="Pressme" Text="Press Me" OnClick="hello"/>
</form>
</body>
```

We should examine this text fairly closely.

```
<script language="apl" runat="server">
```

This line specifies a number of things. First, `<script` introduces the section of the text that contains the program that will control the other elements of the page. `language="apl"` specifies that the script is written in the language apl, which has been registered on the machine. `runat="server"` determines that the script will be executed on the server machine. This ensures that there is zero APL footprint on the client machine. There are no downloads necessary to access a Dyalog.Net web site.

We will examine the hello function shortly.

`</script>` specifies that we have ended the script section of the page, and

`<body>` begins the body of the page. This is the section that defines the layout of the page.

Within the body, we define a single form that contains a single button, which will initially display the text "Press Me". The button has an id, or name, of "Pressme".

Notice that the form definition also includes a `runat=server` tag. This ensures that the state of the page (and all its children) is maintained on the server, again minimizing the footprint on the client.

A crucial feature of the button contained on the page is provided by the expression `OnClick="hello"`. This statement indicates that when the user presses the button, the function "hello" should be executed. The function "hello" is defined, in APLScript, in the script portion of the page.

Compilation

The first time our web page is accessed, the *APLCodeGenerator* processes the .aspx page. The purpose of this tool is to convert the content of the file to a single piece of APLScript.

Here's a snippet of the output from the *APLCodeGenerator* (the line wrap is for the convenience in this document):

```
:Class hello_aspx: System.Web.UI.Page,
  System.Web.SessionState.IRequiresSessionState
```

The script defines a class called "hello_aspx" (this name is derived from the original file name).

This new Type derives from a Type called "System.Web.UI.Page". The new Type also provides the functionality of the `System.Web.SessionState.IRequiresSessionState` interface. The use of interfaces is beyond the scope of this paper, but may be discussed in a future article.

The generated script contains code to create and initialise the "Pressme" button:

```
__ctrl+=(System.Web.UI.WebControls.Button.New 0)
Pressme+__ctrl
__ctrl.ID+'Pressme'
__ctrl.Text+'Press Me'
__ctrl.OnClick+[]OR 'hello'
```

The last line of this code attaches the "hello" function to the `onClick` event of the newly created button.

In addition the generated code includes our “hello” function

```
∇hello args
:Access Public
:ParameterList Object,EventArgs

Response.Write 'Hello World!'
∇
```

In order for the “hello” function to be callable at runtime we need to use the `APLScript :Access` statement to make this a `Public` method. In addition we need to specify the types of each of the parameters that will be passed to the method. The first parameter is declared as being of type “`Object`”, in fact it will be a reference to the button that raised the event. The second parameter is of type `EventArgs`, and will include information pertinent to the specific Click event. In this example we will use neither piece of information, but we need to provide the correct metadata for the function.

As mentioned earlier, `Response.Write` outputs its argument `String` to the output stream of this web page. Our class (`hello_aspx`) derives from `System.Web.UI.Page`. It is this base class that defines the `Response` property, and because our class has derived from it we can use `Response` directly, as if it were a member of our current namespace.

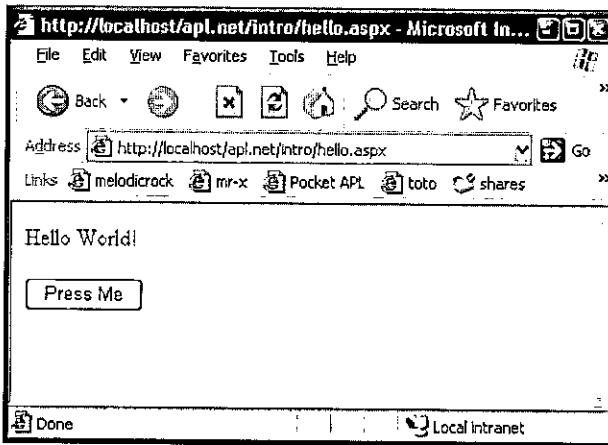
The automatically generated `APLScript` is saved to a temporary file. This file is then passed to the `APLScript` compiler, which generates a .Net assembly.

Runtime

Once the page has been compiled the created assembly is loaded into the ASP.NET process on the server machine. This process creates an instance of our `hello_aspx` class and some of the automatically created APL code is executed (i.e. that code which creates the button). Finally the appropriate HTML is sent "down the wire" to the client. This displays the correct user interface in the client browser. It is important to realize that all the APL code is executed on the server machine. The only data sent to the client is the HTML that defines how the page should appear.

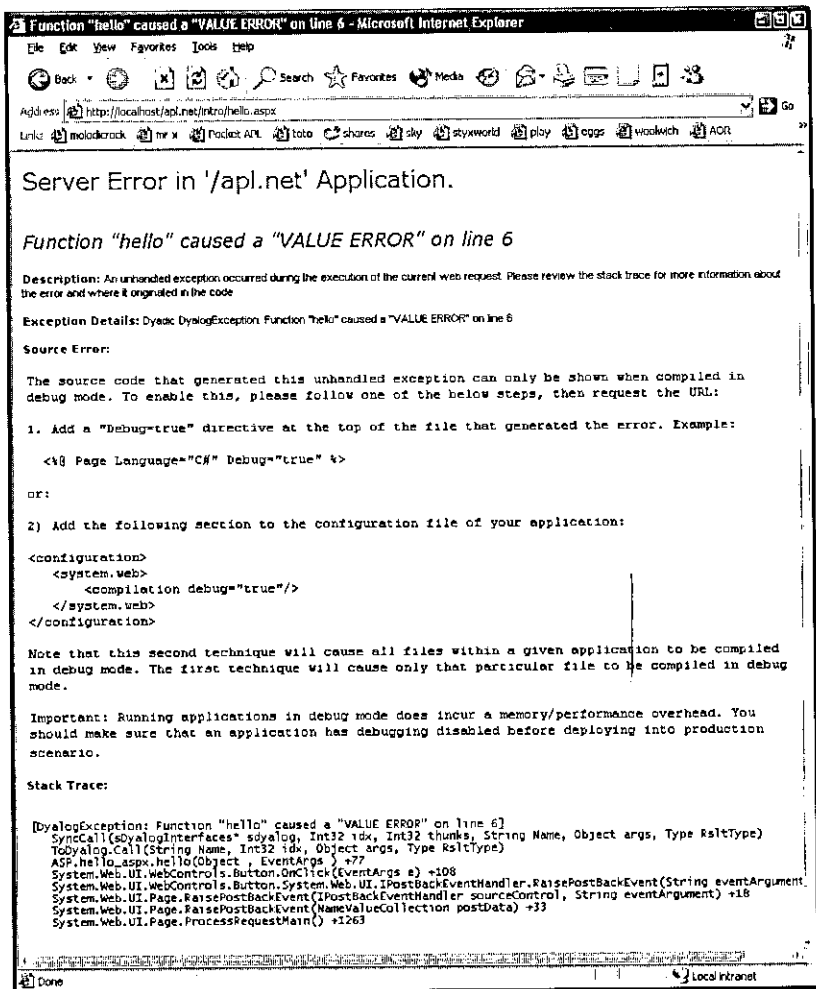
This HTML should be displayed correctly in any browser, not just Microsoft Internet Explorer. When the user presses the "Press Me" button, our function "hello" is called (on the server), and "Hello World!" is included in the HTML that is sent down the response stream.

The client browser appears as follows:



Debugging

We should consider the case where our "hello" function may contain an error. For example if we refer to a non-existent variable in our callback function. There are two possible outcomes. If we have not yet enabled debugging of our ASP.NET scripts, the client will see an error message describing the cause of the error.

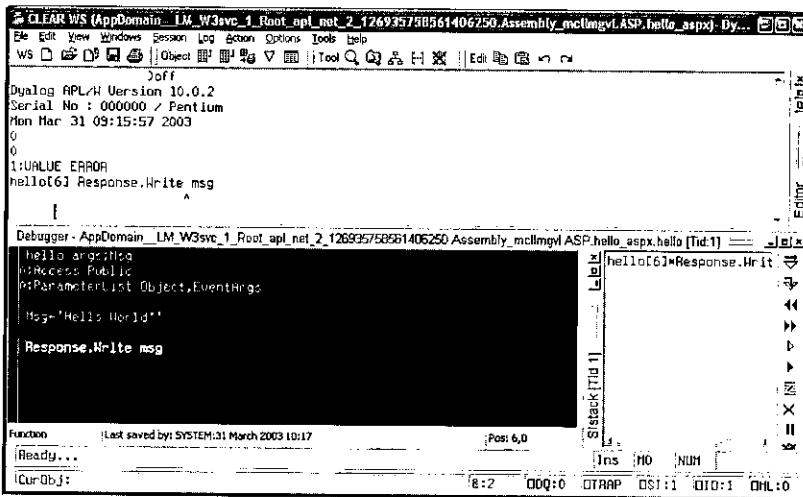


However, if we enable debugging of our ASP.NET web pages we can use the Dyalog APL debugger to debug the APL program in the normal way. We enable debugging by modifying the debug attribute in our web.config file:

```
<configuration>
  <system.web>
    <!-- DYNAMIC DEBUG COMPILATION
      Set debug="true" to enable debugging and bind to
      dyalog.net.dll. Otherwise, setting this value to false will bind
      to dyalog.net.runtime.dll which will prohibit debugging.
    -->
    <compilation debug="true"/>

    <!-- EXECUTION TIMEOUT
      This controls the time in seconds for which a resource
      is allowed to try to execute before ASP.NET cancels (times out)
      the execution of the request. 90 seconds is the default value. If
      you know that a certain process will take longer than 90 seconds
      to execute you should increase this value. Note that if debug is
      true, ASP.NET automatically resets this parameter to a very high
      number.
    -->
    <httpRuntime executionTimeout="90"/>
  </system.web>
</configuration>
```

Once we have made this change, we can re-execute the page. This time we get the Dyalog.Net debugger, neatly displaying the cause of the error.



Once we are stopped in the debugger, we can interrogate the system a little.

First, let's look at)SI.

```
)si
· AppDomain__LM_W3svc_1_Root_apl_net_2_126935779779687500.Assembly_b17d6m12.ASP.hello_aspx.hello[6]*
· System.Web.UI.WebControls.Button.OnClick[]
· System.Web.UI.WebControls.Button.System.Web.UI.IPostBackEventHandler.RaisePostBackEvent[]
· System.Web.UI.Page.RaisePostBackEvent[]
· System.Web.UI.Page.RaisePostBackEvent[]
· System.Web.UI.Page.ProcessRequestMain[]
· System.Web.UI.Page.ProcessRequest[]
· System.Web.UI.Page.ProcessRequest[]
· System.Web.CallHandlerExecutionStep.Execute[]
· System.Web.HttpApplication.ExecuteStep[]
· System.Web.HttpApplication.ResumeSteps[]
· System.Web.HttpApplication.System.Web.IHttpAsyncHandler.BeginProcessRequest[]
· System.Web.HttpRuntime.ProcessRequestInternal[]
· System.Web.HttpRuntime.ProcessRequest[]
· System.Web.Hosting.ISAPIRuntime.ProcessRequest[]
&1 (system thread:1708)
```

We can see that)SI includes the sections of the call stack that are "outside" APL. (⌈SI will still return just the APL portion of the stack). In addition we can see from)SI that Dyalog.Net is running the incoming requests on different threads. The current call has been raised on system thread number 1708, and this has been mapped to APL thread number 1. Additional system threads will be mapped to additional APL threads. The use of threads in this way allows for a very efficient management of multiple, simultaneously connected users.

Also of interest is the name of the current namespace in which we are executing the "hello" callback.

```
{↑'#',1↓('.'=ω)<ω} '' ⌈ns ''
#
AppDomain__LM_W3svc_1_Root_apl_net_2_126935779779687500
Assembly_b17d6m12
ASP
hello_aspx
```

For a number of reasons, Dyalog.Net has taken our original namespace (hello_aspx, from the name of the file hello.aspx) and instantiated it within other namespaces. The reason for having these outer namespaces is to provide isolation from other APL web applications that may be running on this server, in the same workspace. This isolation, along with the concept of AppDomains may be discussed in a future paper.

When a web page in APLScript is compiled the APLCodeGenerator inserts a niladic function called `this` into the source code. `this` returns a reference to the current object.

```

    ⍵vr 'this'
  ▽ r←this
[1]
[2]      ⍺Automatically generated function
[3]      r←((⍵NS'')).##)
[4]
  ▽

this.GetType
ASP.hello_aspx

this.GetType.BaseType
System.Web.UI.Page

```

We can see that our Type is indeed derived from "System.Web.UI.Page".

So, to fix our bug and continue execution, we repair our hello function, by changing `msg` to `Msg`, and then continue execution with `→⍵1c`.

Web Pages from Workspaces

As an alternative to using APLScript we could define our `hello_aspx` Type inside a traditional workspace. In this example we'll call the Type `hello2_aspx` for clarity.

```

)CLEAR
)WSID hello2_ws

```

We need to specify which elements of the .Net Framework we wish to use.

```

⍵using←'System' 'System.Web.UI, System.Web.dll'

```

We now define our new Type. 'Page' specifies the base class for our Type, and because of our current value for `⍵USING` it will be resolved to `System.Web.UI.Page`.

```

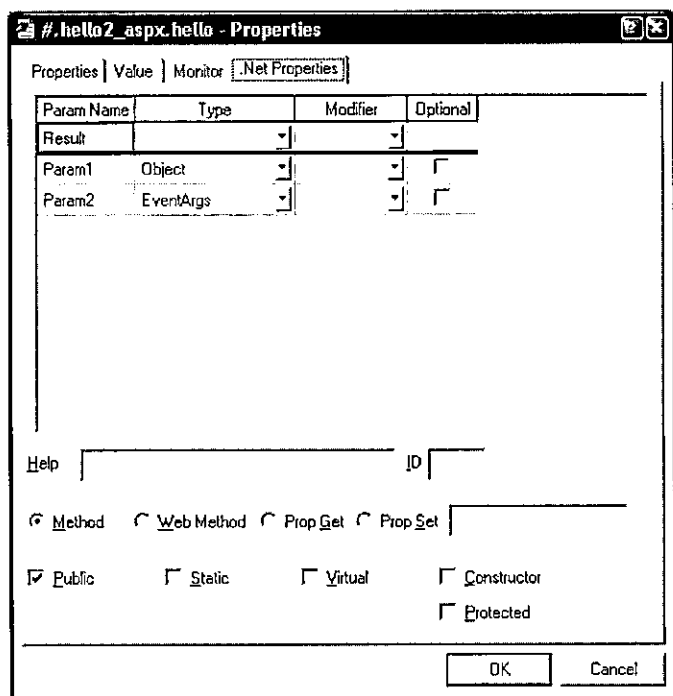
'hello2_aspx'⍵WC'nettype' 'Page'
)cs hello2_aspx

```

And we can define our "Hello" function as

```
var 'hello'  
    v hello args  
[1]  
[2]    Response.Write'Welcome World!!!'  
    v
```

Once we have our hello function we can use the appropriate dialog box to set the metadata for the function:



Finally we can save our workspace.

ASP.NET cannot access the workspace directly. We still need a .aspx file that references the workspace.

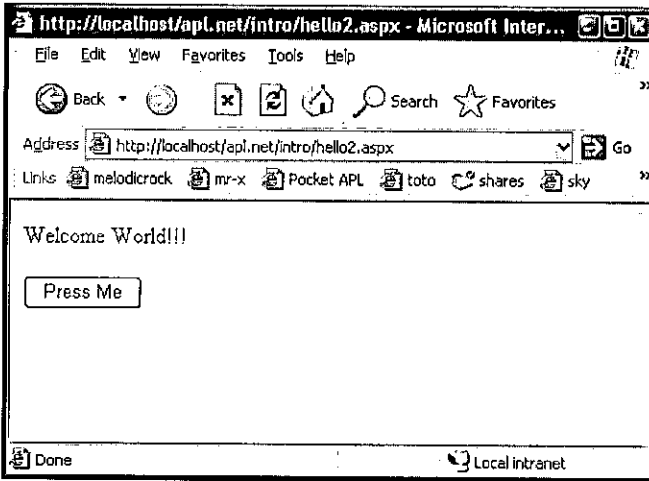
Here's hello2.aspx that does just that:

```
<%@Page Language="apl" Debug="true" Inherits="hello2_aspx"
src="hello2_ws.dws" %>

<body>
<form runat=server>
<asp:Button id="Pressme" Text="Press Me" OnClick="hello" />
</form>
</body>
```

The body of the page is identical to hello.aspx, but now we specify that this page inherits from a Type (called hello2_aspx) defined inside a file called hello2_ws.dws, the workspace that we have just saved.

Pointing our browser at hello2.aspx, and pressing the button gives:



Using this method, the layout of the namespaces at runtime is very similar to that when using APLScript. However, to many programmers, or for larger projects this may be a more convenient way to package the application.

Web Services

Web Services are programmatic elements that may be called over the internet. Dyalog.Net is a perfect tool to write web services, both with APLScript or from workspaces.

As with Dyalog.Net web pages, all the calculation behind the web service is performed on the server machine. Only XML strings that contain the parameters to, and the results of the web service are passed from client to server and vice-versa.

A Simple Web Service

Within the Microsoft .Net Framework Web Services typically reside within files with a .asmx extension. Here is the content of a trivial web service written in APLScript:

```
<%@ WebService Language="apl" Class="APLExample" %>

:Class APLExample:System.Web.Services.WebService

    ▽ R←Add arg
    [1] :Access WebMethod
    [2] :ParameterList Int32 arg1,Int32 arg2
    [3] :Returns Int32
    [4] R←+/arg
    ▽

:EndClass
```

The layout of this file is similar to that of the .aspx files that are used to define web pages. In the case of web services however, we have no <body> section, because a web service has no user interface of its own. The <script> section is implied because it is the only section in the file. In addition the opening declaration in the file declares that this file contains a web service definition.

```
<%@ WebService Language="apl" Class="APLExample" %>
```

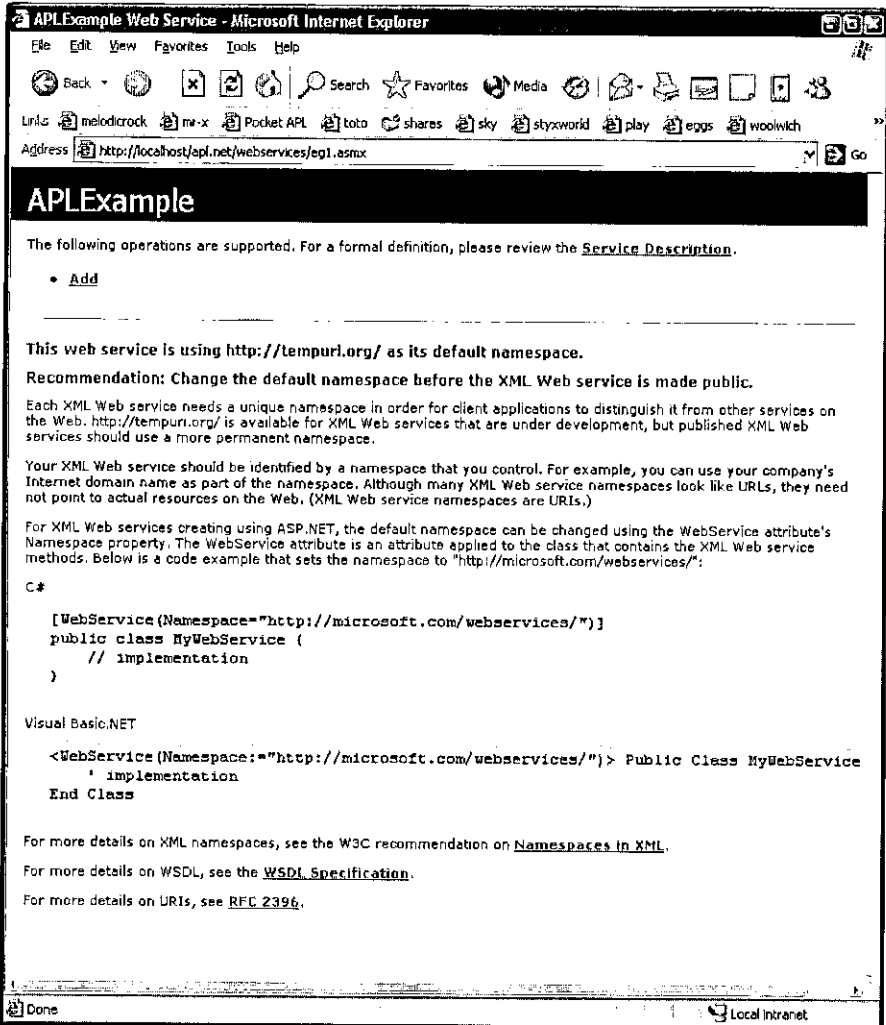
We are defining a web service, that is written in the APL language. This specific file contains a web service called "APLExample". This is the Type of the web service that we are implementing

Our web service contains a single function called Add, and it adds two integral arguments. As we have seen before in APLScript, the function defines its parameter list and return type. In addition, and specific to web services, is the

:Access WebMethod declaration. This is similar to the :Access Public declaration that we have seen before, but additional metadata is emitted for methods marked with the WebMethod declaration.

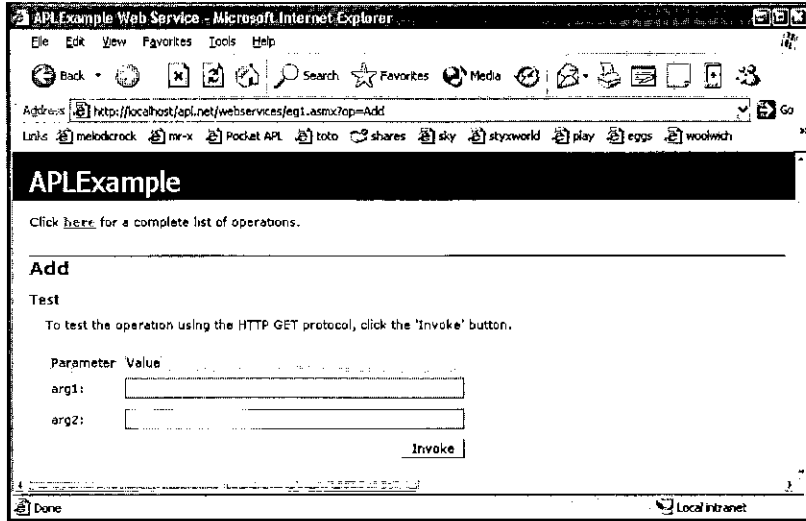
Accessing the Web Service from Internet Explorer

When we point our client browser at eg1.asmx we see:

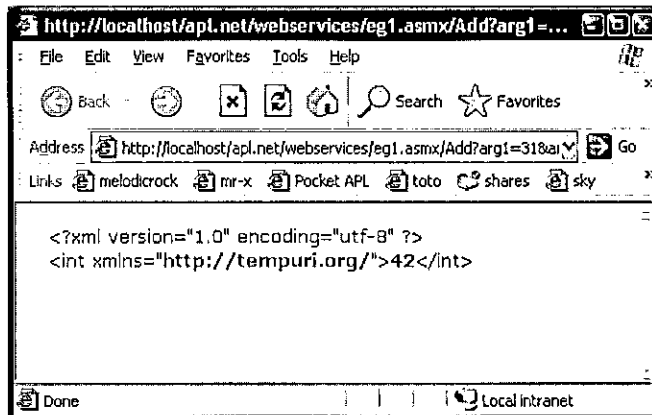


This is the default GUI that Internet Explorer gives us when we access any web service.

If we select our chosen function from the top of the page we see:



The contents of this page have been dynamically created from the metadata that we provided when we declared the Add function. For example, the names of the parameters have been retrieved from that metadata. We can add values to the two input fields and we obtain the result:



The result from the web service is returned as an XML string. This is not very useful to us as it stands, but a web service can be called from a number of sources which may be able to decode the XML stream and present the result in a more meaningful manner.

Accessing the Web Service from APL

Accessing a web service is typically a two-stage process. First we need a "proxy class" that connects to the web service. This class is essentially a thin layer of code that presents a functional interface to the XML streams that pass arguments to, and retrieve results back from, the web service. Fortunately Microsoft ships a tool that creates this proxy class for us. The `wsdl.exe` utility that is part of the .Net Framework can generate a proxy class from the URL of the web service. Dyalog.Net ships with a workspace that contains a cover function for `wsdl`. The utility `wsdl` actually generates C# source code, so the APL cover function also invokes the C# compiler to produce an assembly that we can use.

The proxy class is the only code that runs on the client machine. The web service itself runs only on the server.

Here's the relevant code and result from the APL session, (again this is wrapped for this document).

```
)load c:\dyalog10\samples\asp.net\webservices\
webservices.dws
  MakeProxy 'http://localhost/apl.net/webservices\eg1.asm
C:\devt10\aplsrc\APLExample.dll
```

The file `C:\devt10\aplsrc\APLExample.dll` is the assembly that contains the proxy class to access the web service. We can use `⎕USING` to load this assembly.

```
⎕using←',C:\devt10\aplsrc\APLExample.dll'
```

and create an instance of the proxy class:

```
example←APLExample.New 0
```

which we can subsequently use:

```
example.Add 31 11
42
```

We can, of course create arbitrarily complex web services. Dyalog.Net and Dyalog APL Version 10.0 ship with a web service example to manage golf course

bookings. Dyalog.Net web services may be written in APLScript, as in the above example, or created and saved in traditional workspaces.

And, of course we can use the MakeProxy function to access web services written in any language with any accessible URL. We are not restricted to only those written in Dyalog.Net.

There are a number of interactive tutorials for both web services and web pages which can be accessed from <http://www.dyalog.com>.

Getting to the Bottom of Things

by Phil Chastney (*philip_chastney@yahoo.com*)

introduction

What should the fill value be if we overtake a vector of arrays?

One possible answer is to allow the null elements that occur in index lists, so that, e.g.,

$3 \uparrow (A;B)$

would return $(A;B;)$, which looks promising, but what is that third value?

Some pointers to an answer came up in the consideration of possible maximal and minimal elements in a domain.

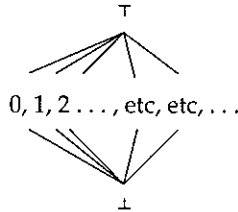
top and bottom

1.1 Thus far, we have had domains, and expressions involving domains, and these have been carefully quarantined from the rest of APL. At the moment, their only point of contact is the membership tests like $x \in N$, where the left argument is an APL expression and the right argument is a domain expression.

There are, however, other interesting attributes of a domain which (sometimes) yield ordinary APL values. Among these is the question of bounds. The concepts will be familiar to many from a study of real analysis, where an interval or a series may have a least upper bound (lub, supremum, or sup) and/or a greatest lower bound (glb, infimum, or inf).

When discussing domains, we shall call these values the **top** and the **bottom** of the domain, and denote the top and bottom of domain D by $\top D$ and $\perp D$. From intervals, we are also familiar with the idea that the glb and lub may or may not be part of the interval. The interval $(0 \leq x < 3)$, for instance, includes its lower limit 0 but not its upper limit of 3. Although bounded above, there is no greatest number *within* that interval. A similar situation is found with the natural numbers, N . The natural numbers start with 0, and carry on from there: there is no largest natural number. The series as a whole, however, is bounded above by the first of the transfinite ordinals, commonly denoted by ω . If top and bottom were incorporated into APL, we would be able to write an expression whose result would be an infinite quantity — interesting, perhaps, but not obviously useful.

1.2 It should be pointed out that the terms "top" and "bottom" are not used here with the same meanings as are employed in mathematical semantics and domain theory. In mathematical semantics, the domain $\mathbb{N}$ includes both top and bottom elements, and there is no ordering between the natural numbers, giving a particularly simple type of lattice, thus:



1.3 The domain $\mathbb{Z}$ consists of all the integers: $\dots, -3, -2, -1, 0, 1, 2, 3, \dots$, etc, etc, ad infinitum. Clearly, this series too is bounded above by ω . It would also appear to be bounded below by some sort of negative infinity. However, if we re-order the series as follows: $0, -1, 1, -2, 2, -3, 3, \dots$ we can see that for every element of $\mathbb{Z}$ there corresponds exactly one element of $\mathbb{N}$, and vice versa. Ergo, there are just as many elements in $\mathbb{Z}$ as there are in $\mathbb{N}$.

Perhaps unsurprisingly, the values of top and bottom depend on the way the elements are ordered. At least for numeric domains, there appears little value in permitting user-defined ordering, so we shall use the ordinary arithmetic ordering.

1.4 The domain of real numbers, $\mathbb{R}$, is a little more difficult. It has been known for a long time that it has a cardinality greater than ω , and is therefore the first uncountable infinite domain most of us encounter. This quantity, usually known as the "cardinality of the continuum", is commonly denoted by c .

The domain $\mathbb{C}$, the complex numbers, introduces another factor: its elements can only be partially ordered, there being no obvious sense in which $3j4$ can be said to be numerically greater than or less than $4j3$, for instance.

"fill" elements

2.1 We, however, are looking for suitable "fill elements" (or "characteristic values") which can act as "prototypes" or "archetypes". It is desirable, but not compulsory, that a domain should have a fill element defined, something after the fashion of DEFAULT values in SQL. Where one is defined it must be an element

of that domain, meeting the type and other constraints of the domain, but leaving other attributes as free as possible.

2.2 A suitable symbol for the fill element can be made up by combining the symbols for top and bottom, thus:

$$\perp D$$

which encapsulates the idea that the fill element must be greater than or equal to the bottom limit and less than or equal to the top limit. If it helps, $\perp D$ may be thought of as the "initial value" for the domain.

2.3 Where $(\perp D) \in D$, for a domain D , the bottom value makes a good choice of fill element. This is the case with $\mathbb{Z}$ and $\mathbb{N}$, and for good practical reasons, the same fill element is used for the other numeric domains. To find a suitable fill element for a given domain, we need to look at the ordering (often a partial ordering) within that domain. Strings, for instance, can be ordered lexicographically, and the space character sorts ahead of all others in almost all collating sequences, which makes it as close to a "bottom element" as we are likely to get, making it an excellent choice for the fill element for $\mathbb{H}$.

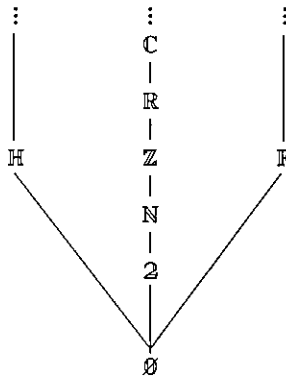
Note that it is not just primitive domains which can have fill elements. The domain $(\mathbb{N}^* \mathbb{N}^* \mathbb{N})$ of all rectangular arrays of natural numbers has a bottom element of the scalar 0, the vector domain $(\mathbb{N}^* \mathbb{N})$ has a bottom element of $\perp 0$, while for a specifically matrix domain, $(\perp \mathbb{N}^* \mathbb{N}^* 2) = (0 \ 0 \rho \ 0)$.

So, if we want to identify a fill element for $\mathbb{U}^* \mathbb{N}$ (vectors of arrays), the first question we need to ask is: what is $\perp \mathbb{U}$?

3. a lower bound for $\mathbb{U}$

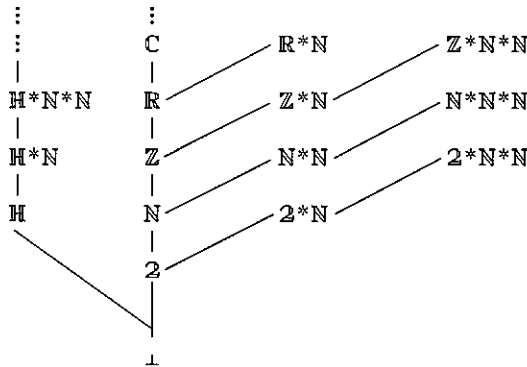
$\mathbb{U}$ is the universal data domain, containing all the admissible values within the language. This includes all the scalar elements from the primitive data domains. The primitive domains themselves are members of $\mathbb{D}$, and include the character and numeric domains. These primitive domains may themselves be partially ordered: if we use $<$ to denote "precedes" in some sense, then we can say $\mathbb{Z} < \mathbb{N} < \mathbb{Z} < \mathbb{R} < \mathbb{C}$. (A totally ordered subset like this is often referred to as a **partial chain**.)

But $\mathbb{H}$ does not fit into that chain. Neither does $\mathbb{F}$, the set of applicable functions. So far, then, our domain $\mathbb{D}$ looks like this:



although this picture is still incomplete.

The scalar elements of $\mathcal{U}$ can be ordered¹ the same way their domains are order in $\mathcal{D}$. Arrays constructed therefrom can ordered by rank and length of dimension:



and so we see our partially sorted universal data domain growing. But this isn't helping with the problem at the root. We do not need to grow this tree to see that no ordering relation holds between character scalars and numeric scalars. The same would be true of any other primitive data domains we might introduce ($\mathcal{F}$, for instance, if functions were made first class objects), so that $(\perp \mathcal{U})$ cannot have any structure, and cannot be a member of a primitive domain. In short, $(\perp \mathcal{U})$ has no type, no shape and no value.

¹ note that, in the upper diagram, the elements shown are *members* of $\mathcal{D}$, while in the lower diagram they are *subsets* of $\mathcal{U}$

4. *null* values in APL

4.1 Actually, it does have a shape, it is just that its shape vector is empty. For reasons we can go into later, it is perfectly consistent to think of $(\perp\mathbb{U})$ as being scalar.

Not unreasonably, $(\perp\mathbb{U})$ is called "*null*". Some care is needed in the use of the term "*null*": too often it is used as a synonym for "*empty*", but it is best equated with the adjective "*undefined*". It is also helpful to treat expressions like "*the null object*" as oxymorons.

4.2 So, on to the next question: is $(\perp\mathbb{U})$ an element of $\mathbb{U}$? To quote Sam Goldwyn, "That's a definite maybe".

There are no functions which can act on undefined values, and there is no way of defining such functions. In the normal course of events, we cannot even view undefined quantities:

```

      A
VALUE ERROR
      A
      ^

```

and we would expect the same sort of treatment here:

```

      ⊥U
VALUE ERROR
      ⊥U
      ^

```

4.3 Null values are not admissible values, but they are nonetheless a part of the language, since, in a limited number of contexts, an element of a list may be null – the most interesting instance of this phenomenon being that of index lists.

Each dimension of an array may be indexed by a sequence of non-negative integers: $\mathbb{N}^*\mathbb{N}$, in the simplest case. These indices may also be structured as arrays, which means that each dimension may be indexed by an element of the domain $\mathbb{N}^*\mathbb{N}^*\mathbb{N}$. If the target array has k dimensions, then the index list also has k elements, and is therefore of the form $(\mathbb{N}^*\mathbb{N}^*\mathbb{N})^*k$, which is a member of $(\mathbb{N}^*\mathbb{N}^*\mathbb{N})^*\mathbb{N}$, our first approximation to the domain of all index lists.

But the story does not end there. We are also permitted to write $V[\]$. The syntactic element between the two brackets is a null string, indicating that this element of the list is null. Now $V[\]$ and $V[\]0$ deliver different results (for non-empty V). In the latter case, the index list is an empty vector; in the former case, the index list is *null* (or "*undefined*"), and they are clearly different things.

In $M[;]$ we have a 2-element index list, each of whose elements is *null*.

4.4 Null elements could also occur in right argument to $\square FMT$, and were ignored, as were null elements in old-style function headers. More interesting, though, is the example of mixed output, where the command

```
'→' ; ; '←'
```

used to print out a null string between the arrowheads, back in the good old days, when such things were considered acceptable. These null strings do not have to be part of a list, either. In immediate execution mode, try typing a null string, followed by Return. The interpreter will respond by displaying your null input as null output.

4.5 It would appear that "explicit" null values (those found in the input at translate time) are treated differently from null values occurring accidentally (those found at run time). This will be because an exception is raised immediately if an undefined value is found whilst dereferencing a variable or initialising an anonymous temporary. (This effectively outlaws $A \leftarrow \perp U$, which is a pity, because it would have been a nifty way to erase a variable.)

5. the "unit domain"

5.1 Clearly, $(N^*N^*N)^*N$ is inadequate as a description of index lists. The problem is: how to extend it? It will be noted that *null* occurs as $\perp U$, while $\perp D$ is $\emptyset$. But $(\perp U) \notin \emptyset$ because $\emptyset$ is empty, so it must be part of some other, previously unidentified, domain.

A number of different approaches have been taken to this boundary value problem². Tennent (p36), for instance, says, "we adopt the convention that $D^0 = \{null\}$, that is to say, the singleton whose only element is *null*", but Bird and Walder (p33) say, "there is no concept of a one-tuple". Watt (p54) defines a domain "Unit – whose only element is the 0-tuple $()$ ", while Gordon (p15) introduces an unnamed domain $\{unbound\}$.

APL has always distinguished scalars from single-element vectors, and the language is the better for it. Can we find a candidate for $\mathbf{1}$, our own "unit domain", which preserves this distinction?

5.2 The first thing we have to do is resolve an identity crisis. If we use $(\vee D)$ to denote the cardinality of a domain D , we would expect

$$\begin{aligned}(\vee A \oplus B) &= (\vee A) + (\vee B) \\ (\vee A \otimes B) &= (\vee A) \times (\vee B) .\end{aligned}$$

² the early page numbers indicate that it is a fundamental issue

We would also expect

$$(\vee A \otimes 1) = (\vee A)$$

which means that 1 must have cardinality 1, i.e, the domain must contain a single element.

Likewise, in the domain of pairs, $\langle A; B \rangle$, the cardinality is $(\vee A) \times (\vee B)$, and we would expect the domain of triples $\langle A; ; B \rangle$ to have precisely as many members, given that the middle element is a constant.

For scalar and/or vector domains A and B , in $(A \otimes B)$ we would expect

$$(\forall x \in A) (\forall y \in B) (\rho x, y) = (\rho, x) + (\rho, y)$$

If B is the unit domain, then the elements of A are extended in length by 1, by the catenation of an undefined value.

So we know that 1 has a single element, and that single element has a shape of $(\vee 0)$ or $(, 1)$. The candidates, then are the domains $\{null\}$ and $\{<null>\}$. Because we want something with no more structure than $\perp \mathbb{H}$ or $\perp \mathbb{Z}$, the simpler of these is preferred, making 1 the domain whose only element is an undefined scalar value.

5.3 Exponentiation (domain repetition) can be applied to 1 , thus:

- 1 contains an single undefined value
- $1 * 0$ contains a zero-length tuple
- $1 * 1$ contains a unit-length tuple, whose contents are *null*
- $1 * 2$ contains an ordered pair, whose elements are both *null*
- $1 * 3$ contains an ordered triple, with all elements *null*

(The elements of $1 * \mathbb{N}$ are vectors whose shape has been defined, but whose values have not.)

We can now define the domain of index lists in all its glory:

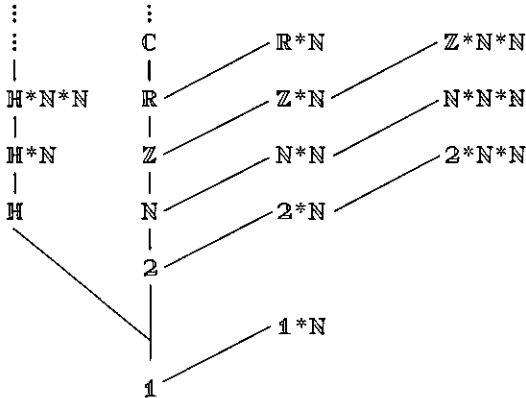
$$(1 \oplus \mathbb{N} * \mathbb{N} * \mathbb{N}) * \mathbb{N}$$

which says that an index list is a one-dimensional list each of whose elements is either null or a rectangular array of natural numbers. (Note that this domain contains the element $(1 \oplus \mathbb{N} * \mathbb{N} * \mathbb{N}) * 0$, a 0-length list suitable for indexing scalars – a facility not normally included in APL implementations.)

6. a “fill” element for $\mathbb{U}$

6.1 To return to our earlier question: is *null* an element of $\mathbb{U}$?

Assuredly so, but only as an element of an array. It is a permissible right argument to (the indexing part of) the primitive function "[", so we can enhance our earlier diagram for $\mathbb{U}$, as follows:



6.2 This might seem to imply that null values are a part of the domains of other primitives. A similar situation obtains with the reciprocal function, whose domain is commonly cited as $\mathbb{R}$, when in fact the reciprocal of zero is not defined — there is no requirement that a function be defined for all the values in its domain. The exclusion of *null* from a domain, however, is such a common requirement that it is more convenient to redefine $\mathbb{U}$. We can exclude scalar *null*, because null elements are admissible values only when they exist as elements of nested arrays. Unfortunately, the simplest rewrite:

```

ARRAY_OF ← λx∈D: (x*N*N)
SIMPLE    ← (ARRAY_OF H)⊕(ARRAY_OF R)
NESTED    ← (ARRAY_OF*N)SIMPLE⊕1

```

will not work, because $(\text{ARRAY_OF}*0)$ would admit scalar *null*. We have to enforce a minimum length of 1 for the rank vector of the nested arrays, and ensure that the constructor is applied at least once, thus:

```

SIMPLE    ← (H*N*N)⊕(R*N*N)
ENCLOSE   ← λx∈D: (x*N⊗N*N)
NESTED    ← (ENCLOSE∘ENCLOSE*N)SIMPLE⊕1

```

Our universal data domain is given by $\mathbb{U} = (\text{SIMPLE} \oplus \text{NESTED})$, and when we want to admit the possibility of a given semantic unit being an unstructured *null*, we can use $\mathbb{U} \oplus 1$.

7. treatment of null

SQL, which has, after all, been dealing with nulls for years³, insists that nulls lie outside the domains of all arithmetic and aggregation functions, and (with one exception) all comparisons. This seems like a reasonable stance to take with APL as well.

In practice, the interpreter traps inappropriate nulls, preventing them from being passed through to most functions. This behaviour is not always convenient, however. Instead of

$$T \leftarrow 3 \uparrow (B;C) \text{ so that } T \leftrightarrow (;B;C)$$

$$A \leftarrow \uparrow T$$

VALUE ERROR

$$A \leftarrow \uparrow T$$

$$\wedge$$

it would be nicer to be able to detect these things before they blow up in our faces.

There are two similar approaches to this:

- (i) testing for nulls explicitly, with an expression like $(x \in 1)$, which would mean the interpreter detecting domain membership tests *before* raising an exception — this would be somewhat similar to the SQL predicate “IS NULL” (the space is important);
- (ii) adding in a primitive function equivalent to SQL92’s ISNULL function. Its syntax is

$$\text{ISNULL}(\text{check_expression} , \text{replacement_value})$$

and it would be employed approximately thus:

$$T \leftarrow 3 \uparrow (; B; C)$$

$$A \leftarrow \text{ISNULL}(\uparrow T; 10) .$$

8. related matters

8.1 There is a related problem involving another level of detail:

Can we distinguish the following items?

³ Certain database luminaries, such as Chris Date and David McGovern, have argued against the inclusion of nulls in databases. They may have a point when it comes to relational databases, but database languages do not normally have to deal with dynamically changing type: for them, a value in a column will always be of the same type.

- a numeric vector
- a length-1 numeric vector
- an empty numeric vector
- a numeric scalar
- an uninitialised variable, constrained to numeric values

The answer to that last one is: not yet, but maybe some day soon.

8.2 There remains a problem with syntax: since *null* is denoted by the null string, how do we interpret the expressions $()$, $[]$ and $\{\}$? Are they strictly empty, or do they enclose a *null*?

brackets, $[]$

This is the easy case: these are used for indexing, and the length of the enclosed list is always equal to the rank of the left argument. For vector left arguments, then, the brackets enclosed a list of length 1, and therefore cannot be empty.

parentheses, $()$

By analogy with the treatment of strings, $()$ should be used to denote the 0-length vector of arrays, as illustrated below:

$M^*\emptyset$	e.g: 'a'	$U^*\emptyset$	e.g: (A)
M^*0	' '	U^*0	()
M^*1	e.g: , 'a'	U^*1	e.g: ↓(; A)
M^*2	e.g: 'ab'	U^*2	e.g: (A; B)

braces, $\{\}$

Usage here is inherently ambiguous, and could denote either $\emptyset$ or 1 . It may be that $\emptyset$ will ultimately be proved unnecessary, but in the meantime, we can avoid the use of $\{\}$, and always refer to the domains by name.

8.3 More importantly, in the meantime, some of the earlier writings will need amendment, so that 1 is used where $\emptyset$ had previously been mis-used.

8.4 Finally, the problem of finding fill elements has its own lower limit, and this is close to it:

$THREE \leftarrow \{3\}$ $\mathbf{A}$ define a domain containing a single element
 $\perp THREE$ $\mathbf{A}$ the bottom element of that domain

If we then, by some method so far unspecified, constrain $V3$ to vectors of 3s:

$V3 \in \text{THREE} * \mathbb{N}$ $\wedge$ confirm that the constraint is operative
 1
 $1 \uparrow V3 + 0 \rho V3$ $\wedge$ the result of overtaking an empty vector
 3

9. references

9.1 Scott's use of "top" and "bottom" is described fairly informally in the appropriate entry (p.74) of

ACM Turing Award Lectures, *The First Twenty Years*, ACM Press [1987]
 and much more formally in

C A Gunter, D S Scott, *Semantic Domains*

[in] J van Leeuwen [ed], *Handbook of Theoretical computer Science*, Volume B
 North Holland [1990]

9.2 The full references for the sources cited in §6 are:

R D Tennent, *Principles of Programming Languages*, Prentice Hall [1981]

Richard Bird, Philip Wadler, *Introduction to Functional Programming*
 Prentice Hall [1988]

David Watt, *Programming Language Syntax and Semantics*
 Prentice Hall [1991]

Michael J C Gordon, *The Denotational Description of Programming Languages*
 Springer-Verlag, New York [1979]

Index to Advertisers

Dyadic Systems Ltd	6
MicroAPL Ltd	2
Strand Software	10
Vector Back Numbers	7

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (hardcopy with diskette as appropriate) to the Vector Working Group via:

Vector Administration, c/o Gill Smith
Brook House
Gilling East
YORK, YO62 4JJ
Tel: +44 (0) 1439-788385
Email: apl385@compuserve.com

Authors wishing to use Word for Windows should contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template. These may also be downloaded from the Vector web site at www.vector.org.uk

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

Subscribing to Vector

Your Vector subscription includes membership of the British APL Association, which is open to anyone interested in APL or related languages. The membership year runs from 1st May to 30th April. The British APL Association is a special interest group of the British Computer Society, Reg. Charity No. 292,786

Name: _____
 Address: _____

 Postcode / Country: _____
 Telephone Number: _____
 Email Address: _____

UK private membership	£20	☐
Overseas private membership	£22	☐
Airmail supplement (not needed for Europe)	£4	☐
UK Corporate membership	£100	☐
Corporate membership overseas	£135	☐
Sustaining membership	£500	☐
Non-voting UK member (student/OAP/unemployed)	£10	☐

PAYMENT – in Sterling or by Visa/Mastercard/JCB only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard, Visa or JCB number.

I authorise you to debit my Visa/Mastercard/JCB account

Number: Expiry date: /

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

Data Protection Act:
The information supplied may be stored on computer and processed in accordance with the registration of the British Computer Society.

Signature: _____

Send the completed form to:

BAA, c/o Rowena Small, 12 Cambridge Road, Waterbeach, CAMBRIDGE CB5 9NJ, UK

Fax: +44 (0) 1653 697719

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" – an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
The Maltings, Castlegate,
MALTON, North Yorks YO17 7DP
Tel: 01653-696760
Fax: 01653-697719
Email: sales@causeway.co.uk
Web: www.causeway.co.uk

Compass Ltd
Compass House
60 Priestley Road
GUILDFORD, Surrey GU2 5YU
Tel: 01483-514500

Dyalog Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants, RG24 0AL
Tel: 01256-811125
Fax: 01256-811130
Email: sales@dyalog.com
Web: www.dyalog.com

APL2000 Inc
One Research Court
Suite 140
Rockville MD 20850
USA
Tel: +1 (301) 208 7150
Email: sales@apl2000.com
Web: www.apl2000.com

Insight Systems ApS
Nordre Strandvej 119G
DK-3150 Hellebæk
Denmark
Tel: +45 70 26 13 26
Fax: +45 70 26 13 25
Email: info@insight.dk
Web: www.insight.dk

HMW Computing
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 0870-1010-469
Email: HMW@4xtra.com

MicroAPL Ltd
The Roller Mill, Mill Lane
Uckfield
E Sussex TN22 5AA
Tel: 01825-768050, Fax: 01825-749472
Email: MicroAPL@microapl.demon.co.uk
Web: www.microapl.co.uk/apl

Soliton Associates Ltd
Groot Blankenberg 53
1082 AC Amsterdam
Netherlands
Tel: +31 20 646 4475
Fax: +31 20 644 1206
Email: sales@soliton.com

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

2002/2003 Committee

Chairman	Adrian Smith 01439-788385 apl385@compuserve.com	Brook House Gilling East YORK YO62 4JJ
Secretary	Anthony Camacho 0117-973-0036 acam@blueyonder.co.uk	11 Auburn Road Redland BRISTOL, BS6 6LS
Treasurer	Nicholas Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ
Journal Editor	Stefano Lanzavecchia stf@apl.it	c/o APL Italiana Corso Vercelli 54 20145 - Milano Italy
Projects and Publicity	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Webmaster	Ray Cannon 01252-874697 ray_cannon@compuserve.com	21 Woodbridge Road, Blackwater, Camberley, Surrey GU17 0BS
Activities	Jon Sandles 01904-612882 jon_sandles@csi.com	138 Burton Stone Lane, YORK YO30 6DF
School Project	Stephen Taylor stj@lambenttechnology.com	
Administration	Rowena Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ

Journal Working Group

Editor:	Stefano Lanzavecchia	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Anthony & Sylvia Camacho (0117-973-0036), Ray Cannon (01252-874697), Stephen Taylor (077-1340-0832), Bob Hoekstra (01483-771028), Jon Sandles (01904-612882), Marc Griffiths	

Typeset by APL-385 with MS Word for Windows

Printed in England by Short-Run Press Ltd, Exeter